eMadrasa Management System

Team ID: 4

Session: BSCS Fall 2021 - 2025

Project Advisor: Mr. Fazeel uddin Faraz

Submitted By:

Name of Student	Roll Number
Huzaifa Zafar	071262

Statement of Submission

This is to certify that the following student have successfully completed the final year project titled **“eMadrasa Management System”** at **FG Sir Syed College, The Mall Rawalpindi Cantt**, in partial fulfillment of the requirements for the degree of **Bachelor of Science in Computer Science**:

Name of Student	Roll Number
Huzaifa Zafar	071262

Project Co Supervisor

Miss Rafia Khalil

Lecturer Computer Science

FG Sir Syed college, The Mall Rawalpindi

Project Supervisor

Mr Fazeel ud din Faraz Siddiqi

Head of Department Computer Science

FG Sir Syed college, The Mall Rawalpindi

Acknowledgement

I sincerely acknowledge the invaluable cooperation and support provided by **Fazeel uddin Faraz**, **HOD Computer Science**, **FG Sir Syed College, Rawalpindi**. Their continuous guidance has been a great source of motivation throughout the course of this project.

I would also like to extend our gratitude to **Rafia Khalil**, **Co Supervisor**, **FG Sir Syed College, Rawalpindi**. for their helpful insights and encouragement during the development of this project.

My heartfelt thanks go to my friends and families, whose silent support and encouragement played a vital role in helping and completing this project successfully.

Special Thanks:

Mr. Fazeel uddin Faraz
Miss Rafia Khalil
Miss Kainat Mehak
Mr. Arshad
Mr. Atif Javed

Date: 01-11-2024

Abstract

This project is about making a website for an eMadrasa Management System. It helps in doing the daily work of a madrasa in a very easy and fast way. In the past, all the work like attendance, diaries, sabaq, and student records were done using paper and pen. This took a lot of time and sometimes had mistakes. But now, with this system, all the work is done on computer or mobile in a proper and organized way.

This system is for different people who work in a madrasa, like admin, accountant, teacher, student, and parent. Each person has a separate login and a separate panel. The admin can add and manage accountants. The accountant can add and manage students, teachers, and parents. The teacher can give sabaq, mark attendance, write diary notes, and add student progress. The parent can see their child's diary, sabaq, attendance, and progress. The student can also read and listen to the Quran from this system.

All these features help to make the work easier. No one needs to write on paper again and again. Everything is saved safely in the system. The data can be seen anytime by the users. This makes it fast, safe, and less confusing.

The system is made using the latest technology, used React for the front side, Node.js and Express for the back side (server), and MongoDB to save the data.

The main goal of this project is to bring digital tools into the madrasa. It saves time, reduces mistakes, and keeps all records in one place. This system helps all users admin, accountant, teacher, parent, and student to do their work easily without any problem. It also helps madrasas to move from paper work to computer work in a smart way.

Table of Contents

List of Tables ..... **xii**
List of Figures ..... **xv**
CHAPTER 1 ..... **1**
INTRODUCTION **1**
1.1 Background **2**
1.2 Problem Statement **2**
1.2.1 Attendance Management 2
1.2.2 Progress Tracking 2
1.2.3 Communication Gaps 2
1.2.4 Resource Constraints 2
1.3 Objectives **3**
1.3.1 General Objectives 3
1.3.2 Specific Objectives 3
1.3.3 Technical Objectives 3
1.4 Scope of the Project **3**
1.4.1 Included Scope 3
1.4.2 Excluded Scope 4
1.4.3 Technical Scope 4
1.4.4 Limitations 4
1.4.5 Constraints 4
1.5 Methodology Summary **5**
1.6 Tools and Technologies Used **5**
1.6.1 Frontend Development 5
1.6.2 Backend Development 5
1.6.3 Database 5
1.6.4 APIs & Third-Party Services 6
1.6.5 Testing & Debugging 6
1.6.8 Other Tools 6
CHAPTER 2 ..... **7**
LITERATURE REVIEW **7**
2.1 Summary of Similar Existing Systems or Research **8**
2.1.1 Existing Management Systems 8
2.2 Analysis of Gaps or Limitations in Current Solutions **8**
2.2.1 How EMMS Fills the Gaps 9
2.3 Justification for your Approach **9**
2.3.1 Focus on Madrasa-Specific Needs 9
2.3.2 Cost-Effective Technology 10
2.3.3 Simplicity for Non-Tech Users 10

2.3.4 Security Without Complexity 10
2.3.5 Scalability for Growth 10
CHAPTER 3 11
SYSTEM ANALYSIS 11
3.1 Requirements Gathering 12
3.2 Functional Requirements 12
3.2.1 FR01: Sign Up 13
3.2.2 FR02: Reset Password 13
3.2.3 FR03: Reset Password OTP 13
3.2.4 FR04: Change Password 13
3.2.5 FR05: Login Account 13
3.2.6 FR06: Dashboard 13
3.2.7 FR07: Create/View/ Update/Delete Details 14
3.2.8 FR08: Read and Listen Quran 14
3.2.9 FR09: Admin Portal 14
3.2.10 FR 10: Accountant Portal 14
3.2.11 FR 11: Teacher Portal 15
3.2.12 FR 12: Parent Portal 15
3.2.13 FR 13: Logout 15
3.3 Non-Functional Requirements 15
3.3.1 Performance 15
3.3.2 Security 15
3.3.3 Usability 16
3.3.4 Reliability 16
3.3.5 Maintainability 16
3.4 Gantt Chart 16
3.5 Use Case Diagrams 17
3.5.1 Extended Admin Use Case 18
3.5.1.1 Admin Login 19
3.5.1.2 Admin View Dashboard 20
3.5.1.3 Admin Create Accountant 21
3.5.1.4 Admin Update Accountant 22
3.5.1.5 Admin Delete Accountant 23
3.5.1.6 Admin Monitor System Performance 24
3.5.1.7 Admin Logout 25
3.5.2 Extended Accountant Use Case 26
3.5.2.1 Accountant Login 27
3.5.2.2 Accountant View Dashboard 28
3.5.2.3 Accountant Create Student 29
3.5.2.4 Accountant Update Student 30

3.5.2.5 Accountant Delete Student 31
3.5.2.6 Accountant Create Teacher 32
3.5.2.7 Accountant Update Teacher 33
3.5.2.8 Accountant Delete Student 34
3.5.2.9 Accountant Create Parent 35
3.5.2.10 Accountant Update Parent 36
3.5.2.11 Accountant Delete Parent 37
3.5.2.12 Accountant view Progress 38
3.5.2.13 Accountant Logout 39
3.5.3 Extended Teacher Use Case 40
3.5.3.1 Teacher Login 41
3.5.3.2 Teacher Mark Attendance 42
3.5.3.3 Update Daily Dairy 43
3.5.3.4 Update Student Progress 44
3.5.3.6 Teacher reads and listen Quran 45
3.5.3.6 Teacher Logout 46
3.5.4 Extended Parent Use Case 47
3.5.4.1 Parent Login 48
3.5.4.2 View Attendance 49
3.5.4.3 Read Daily Diary 50
3.5.4.4 Check Progress 51
3.5.4.5 Read and listen Quran 52
3.5.4.6 Parent Logout 53
3.6 Domain Model Diagram 54
3.7 Data Flow Diagram 55
3.7.1 Level 0 Data Flow Diagram 55
3.7.2 Level 1 Data Flow Diagram 56
CHAPTER 4 57
SYSTEM DESIGN 57
4.1 System Architecture 58
4.2 Database Design 59
4.2.1 Admin Collection 59
4.2.2 Accountant Collection 59
4.2.3 Teacher Collection 59
4.2.4 Parent Collection 60
4.2.5 Student Collection 60
4.2.6 Diary Collection 60
4.2.7 Attendance Collection 61
4.2.8 Progress Collection 61
4.3 ER Diagrams 61

4.3.1 Admin Entity 61
4.3.2 Accountant Entity 62
4.3.3 Diary Entity 62
4.3.4 Student Entity 62
4.3.5 Teacher Entity 63
4.3.6 Attendance Entity 63
4.3.7 Parent Entity 63
4.3.8 Progress Entity 64
4.3.9 Admin Accountant Relationship 64
4.3.10 Accountant Student Relationship 64
4.3.11 Accountant Teacher Relationship 65
4.3.12 Accountant Parent Relationship 65
4.3.13 Accountant Progress Relationship 65
4.3.14 Teacher Diary Relationship 66
4.3.15 Teacher Attendance Relationship: 66
4.3.16 Teacher Progress Relationship 66
4.3.17 Teacher Progress Relationship 67
4.3.18 Parent Diary Relationship 67
4.3.19 Parent Attendance Relationship 67
4.3.20 Parent Progress Relationship 68
4.3.21 Student Diary Relationship 68
4.3.22 Student Attendance Relationship 68
4.3.23 Student Progress Relationship 69
4.3.24 Complete System ER Diagram 70
4.4 Interface Design 71
4.4.1 Welcome Page Design 71
4.4.2 Admin Dashboard Interface Design 71
4.4.3 Accountant Dashboard Interface Design 72
4.4.4 Teacher Dashboard Interface Design 72
4.4.5 Parent Interface Design 73
4.5 Component Design 74
4.5.1 Login Component 74
4.5.2 Forget Password Component 74
4.5.3 Reset Password Component 75
4.5.4 Register Accountant Component 75
4.5.5 Update Accountant Component 76
4.5.6 Delete Accountant Component 76
4.5.7 All Users Component 77
4.5.8 Add Student Component 77
4.5.9 Update Student Component 78

4.5.10 Delete Student Component 78
4.5.11 All Students Component 79
4.5.12 Create Teacher Component 79
4.5.13 Update Teacher Component 80
4.5.14 Delete Teacher Component 80
4.5.15 All Teacher Component 81
4.5.16 Create Parent Component 81
4.5.17 Update Parent Component 82
4.5.18 Delete Parent Component 82
4.5.19 All Parents Component 83
4.5.20 Attendance Component 83
4.5.21 Update Sabaq Component 84
4.5.22 Add Diary Component 84
4.5.23 Progress Report Component 85
4.6 Class Diagram 86
4.7 Object Diagram 87
4.8 Sequence Diagram 88
4.8.1 Admin Register Accountant Sequence Diagram 89
4.8.2 Admin Update Accountant Sequence Diagram 89
4.8.3 Admin Delete Accountant Sequence Diagram 90
4.8.4 Admin View Accountant List Sequence Diagram 90
4.8.5 Accountant Create Student Sequence Diagram 91
4.8.6 Accountant Update Student Sequence Diagram 91
4.8.7 Accountant Delete Student Sequence Diagram 92
4.8.8 Accountant View Student List Sequence Diagram 92
4.8.9 Accountant Create Teacher Sequence Diagram 93
4.8.10 Accountant View Teacher List Sequence Diagram 93
4.8.11 Accountant Update Teacher Sequence Diagram 94
4.8.12 Accountant Delete Teacher Sequence Diagram 94
4.8.13 Accountant Create Parent Sequence Diagram 95
4.8.14 Accountant Update Parent Sequence Diagram 95
4.8.15 Accountant Delete Parent Sequence Diagram 96
4.8.16 Accountant View Parent List Sequence Diagram 96
4.8.17 Accountant View Progress Sequence Diagram 97
4.8.18 Teacher Marks Attendance Sequence Diagram 97
4.8.19 Teacher Add Diary Sequence Diagram 98
4.8.20 Teacher Update Sabaq Sequence Diagram 98
4.8.22 Parent View Attendance Sequence Diagram 99
4.8.23 Parent View Diary Sequence Diagram 100
4.8.24 Parent View Progress Sequence Diagram 100

CHAPTER 5 101
IMPLEMENTATION 101
5.1 Development Tools and Environment 102
5.2 Modules Implemented 103
CHAPTER 6 104
SYSTEM TESTING 104
6.1 Testing Techniques Used 105
6.2 Test Plans 106
6.2.1 GUI Testing 106
6.2.3 Performance Testing 106
6.2.4 Load Testing 107
6.2.5 Configuration Testing 107
6.2.6 Browser Testing 107
6.3 Test Cases 108
6.3.1 Login Test Case 108
6.3.2 OTP Verification Test Case: 108
6.3.3 Reset Password Test Case 108
6.3.4 Register User Test Case 109
6.3.5 Update User Test Case 109
6.3.6 Delete User Test Case 110
6.3.7 Get All Users Test Case 110
6.3.8 Add Diary Test Case 110
6.3.9 Mark Attendance Test Case 111
6.3.10 Add Daily Sabaq Test Case 111
6.3.11 Read and Listen Quran Test Case 111
6.3.12 View Diary Test Case 112
6.3.13 View Attendance Test Case 112
6.3.14 View Progress Test Case 112
6.3.15 Logout Test Case 112
6.4 Results and Bug Reports 113
CHAPTER 7 114
RESULTS AND DISCUSSION 114
7.1 Performance Metrics 115
7.1.1 Page Load Time 115
7.1.2 Data Saving Time 115
7.1.3 System Uptime 115
7.1.4 Multiple Users at the Same Time 116
7.1.5 Database Speed 116
7.1.6 Mobile and Desktop Compatibility 116
7.1.7 Low Internet Handling 116

7.2 Evaluation of Results 117
CHAPTER 8 118
CONCLUSION & FUTURE WORK 118
References 120

List of Tables

Table 1 Analysis Gap 9
Table 2 FR01 Sign Up 13
Table 3 FR02 Reset Password 13
Table 4 FR03 Reset Password OTP 13
Table 5 FR04 Change Password 13
Table 6 FR05 Login Account 13
Table 7 FR06 Dashboard 13
Table 8 FR07 CRUD Operations 14
Table 9 FR08 Read and Listen Quran 14
Table 10 FR09 Admin Portal 14
Table 11 FR10 Accountant Portal 14
Table 12 FR11 Teacher Portal 15
Table 13 FR12 Parent Portal 15
Table 14 FR13 Logout 15
Table 15 UC1-01 19
Table 16 UC1-02 20
Table 17 UC1-03 21
Table 18 UC1-04 22
Table 19 UC1-05 23
Table 20 UC1-06 24
Table 21 UC1-07 25
Table 22 UC2-01 27
Table 23 UC2-02 28
Table 24 UC2-03 29
Table 25 UC2-04 30
Table 26 UC2-05 31
Table 27 UC2-06 32
Table 28 UC2-07 33
Table 29 UC2-08 34
Table 30 UC2-09 35
Table 31 UC2-10 36
Table 32 UC2-11 37

Table 33 UC2-12 38
Table 34 UC2-13 39
Table 35 UC3-01 41
Table 36 UC3-02 42
Table 37 UC3-03 43
Table 38 UC3-04 44
Table 39 UC3-05 45
Table 40 UC3-06 46
Table 41 UC4-01 48
Table 42 UC4-02 49
Table 43 UC4-03 50
Table 44 UC4-04 51
Table 45 UC4-05 52
Table 46 UC4-06 53
Table 47 Admin Collection 59
Table 48 Accountant Collection 59
Table 49 Teacher Collection 60
Table 50 Parent Collection 60
Table 51 Student Collection 60
Table 52 Diary Collection 61
Table 53 Attendance Collection 61
Table 54 Progress Collection 61
Table 55 Modules Implemented 103
Table 56 GUI Testing 106
Table 57 Functional Testing 106
Table 58 Performance Testing 107
Table 59 Load Testing 107
Table 60 Configuration Testing 107
Table 61 Browser Testing 107
Table 62 Login Test Case 108
Table 63 OTP Verify Test Case 108
Table 64 Reset Password Test Case 109
Table 65 Register User Test Case 109
Table 66 Update User Test Case 110

Table 67 Delete User Test Case 110
Table 68 Get All Users Test Case 110
Table 69 Add Diary Test Case 111
Table 70 Mark Attendance Test Case 111
Table 71 Add Daily Sabaq Test Case 111
Table 72 Read and Listen Quran Test Case 111
Table 73 View Diary Test Case 112
Table 74 View Attendance Test Case.....112
Table 75 View Progress Test Case 112
Table 76 Logout Test Case 112
Table 77 Bug Report Table 113
Table 78 Page Load Time.....115
Table 79 Data Saving Time.....115
Table 80 System Uptime 115
Table 81 Multiple User at same Time.....116
Table 82 Multiple User at same Time.....116
Table 83 Database Speed 116

List of Figures

Figure 1 Gantt Chart 16
Figure 2 Complete System Use Case Diagram 17
Figure 3 Admin Use Case Diagram 18
Figure 4 UC1-01 19
Figure 5 UC1-02 20
Figure 6 UC1-03 21
Figure 7 UC1-04 22
Figure 8 UC1-05 23
Figure 9 UC1-06 24
Figure 10 UC1-07 25
Figure 11 UC2-01 27
Figure 12 UC2-02 28
Figure 13 UC2-03 29
Figure 14 UC2-04 30
Figure 15 UC2-05 31
Figure 16 UC2-06 32
Figure 17 UC2-07 33
Figure 18 UC2-08 34
Figure 19 UC2-09 35
Figure 20 UC2-10 36
Figure 21 UC2-11 37
Figure 22 UC2-12 38
Figure 23 UC2-13 39
Figure 24 Teacher Use Case Diagram 40
Figure 25 UC3-01 41
Figure 26 UC3-02 42
Figure 27 UC3-03 43
Figure 28 UC3-04 44
Figure 29 UC3-05 45
Figure 30 UC3-06 46
Figure 31 Parent Use Case Diagram 47
Figure 32 UC4-01 48

Figure 33 UC4-02.....49
Figure 34 UC4-03.....50
Figure 35 UC4-04.....51
Figure 36 UC4-05.....52
Figure 37 UC4-06.....53
Figure 38 Domain Model Diagram.....54
Figure 39 Level 0 Data Flow Diagram.....55
Figure 40 Level 1 Data Flow Diagram.....56
Figure 41 System Architecture Diagram.....58
Figure 42 Admin Entity.....61
Figure 43 Accountant Entity.....62
Figure 44 Diary Entity.....62
Figure 45 Student Entity.....63
Figure 46 Teacher Entity.....63
Figure 47 Attendance Entity.....63
Figure 48 Parent Entity.....63
Figure 49 Progress Entity.....64
Figure 50 Admin Accountant Relationship.....64
Figure 51 Accountant Student Relationship.....65
Figure 52 Accountant Teacher Relationship.....65
Figure 53 Accountant Parent Relationship.....65
Figure 54 Accountant Progress Relationship.....65
Figure 55 Teacher Diary Relationship.....66
Figure 56 Teacher Attendance Relationship.....66
Figure 57 Teacher Progress Relationship.....67
Figure 58 Teacher view Progress Relationship.....67
Figure 59 Parent Diary Relationship.....67
Figure 60 Parent Attendance Relationship.....67
Figure 61 Parent Progress Relationship.....68
Figure 62 Student Diary Relationship.....68
Figure 63 Student Attendance Relationship.....68
Figure 64 Student Progress Relationship.....69
Figure 65 Complete System ER Diagram.....70
Figure 66 Welcome Page Interface Design.....71

Figure 67 Admin Dashboard Interface Design 71
Figure 68 Accountant Dashboard Interface Design 72
Figure 69 Teacher Dashboard Interface Design 72
Figure 70 Parent Dashboard Interface Design 73
Figure 71 Login Component Design 74
Figure 72 Forget Password Component Design 74
Figure 73 Reset Password Component Design 75
Figure 74 Register Accountant Component Design 75
Figure 75 Update Accountant Component Design 76
Figure 76 Delete Accountant Component Design 76
Figure 77 All Users Component Design 77
Figure 78 Add Student Component Design 77
Figure 79 Update Student Component Design 78
Figure 80 Delete Student Component Design 78
Figure 81 All Students Component Design 79
Figure 82 Create Teacher Component Design 79
Figure 83 Update Teacher Component Design 80
Figure 84 Delete Teacher Component Design 80
Figure 85 All Teacher Component Design 81
Figure 86 Create Parent Component Design 81
Figure 87 Update Parent Component Design 82
Figure 88 Delete Parent Component Design 82
Figure 89 All Parents Component Design 83
Figure 90 Attendance Component Design 83
Figure 91 Update Sabaq Component Design 84
Figure 92 Add Diary Component Design 84
Figure 93 Progress Report Component Design 85
Figure 94 Class Diagram 86
Figure 95 Object Diagram 87
Figure 96 Sequence Diagram 88
Figure 97 Admin Register Accountant Sequence Diagram 89
Figure 98 Admin Update Accountant Sequence Diagram 89
Figure 99 Admin Delete Accountant Sequence Diagram 90
Figure 100 Admin View Accountant List Sequence Diagram 91

Figure 101 Accountant Create Student Sequence Diagram.....91
Figure 104 Accountant View Student List Sequence Diagram 92
Figure 105 Accountant Create Teacher Sequence Diagram 93
Figure 106 Accountant View Teacher List Sequence Diagram 93
Figure 107 Accountant Update Teacher Sequence Diagram 94
Figure 108 Accountant Delete Teacher Sequence Diagram.....94
Figure 109 Accountant Create Parent Sequence Diagram 95
Figure 110 Accountant Update Parent Sequence Diagram.....96
Figure 111 Accountant Delete Parent Sequence Diagram.....96
Figure 112 Accountant View Parent List Sequence Diagram 97
Figure 113 Accountant View Progress Sequence Diagram 97
Figure 114 Teachers Marks Attendance Sequence Diagram.....98
Figure 115 Teachers Add Diary Sequence Diagram 98
Figure 116 Teachers Update Sabaq Sequence Diagram.....99
Figure 117 Teachers View Progress Sequence Diagram.....99
Figure 118 Parent View Attendance Sequence Diagram 99
Figure 119 Parent View Diary Sequence Diagram 100
Figure 120 Parent View Progress Sequence Diagram.....100
Figure 121 Test Flow Diagram..... **Error! Bookmark not defined.**

CHAPTER 1
INTRODUCTION

1.1 Background

Madrasas are places where children learn the Quran, Arabic, and Islamic teachings. But most madrasas still use paper and pen to manage their work.

Currently, teachers manually mark attendance on physical registers, which can lead to data being lost or misplaced due to damage or mismanagement. Parents do not receive regular reports or updates about their child's learning progress, creating a communication gap. Additionally, madrasa managers have lack access to report, making it difficult to identify students who are struggling or need additional support.

Other schools use apps like Google Classroom, but cannot directly relate with madrasa needs. The **eMadrasa System** is a free and simple app made just for madrasas. It is easy to use for non tech users and works on phones and computers.

1.2 Problem Statement

There is no proper online system that connects admins, accountants, teachers, and parents in one place, especially in madrasas. This gap makes it hard to manage student information smoothly and quickly. Also, with no use of modern tools like data analysis or automation, understanding overall student performance becomes even more difficult including:

1.2.1 Attendance Management

Teachers mark students attendance manually in registers, which takes a lot of time and effort. This method is not safe because the register can get lost, damaged, or misplaced. Also, parents don't get timely or regular updates about their child's attendance, which creates a communication gap.

1.2.2 Progress Tracking

Students daily progress like *Sabqi* and *Manzil* is written down in notebooks by teachers. This makes it difficult to track or analyze a student's learning progress over time. Parents also don't get proper combined reports, which causes confusion about how well their child is doing.

1.2.3 Communication Gaps

Madrasa managers find it difficult to keep track of the madrasa's overall performance because there are no automated reports. They have to check everything manually, which takes time and increases the chances of errors.

1.2.4 Resource Constraints

Most existing school management systems are either too expensive or too complicated for madrasas to use. On the other hand, the free apps available on the Play Store usually only offer basic features like attendance marking. There is no proper system that focuses on the specific needs of madrasas like tracking Quranic progress or involving parents in their child's Islamic education. This creates a big gap in effective learning and communication.

Because of these problems, there is a need for a simple and easy-to-use system that can help madrasas move from manual work to an online platform where everything is managed in a better, faster, and smarter way.

1.3 Objectives

The main objective of this project is to build an online system that makes the daily management of a madrasa easier and faster. The system is designed in a way that every user like admin, accountant, teacher, and parent can use it easily without any confusion. Let's break different types of objectives for this project in detail:

1.3.1 General Objectives

The primary objective of this project is to replace the traditional, manual, and paper-based processes used in madrasas with a user-friendly digital platform. This system aims to establish real-time communication between teachers, parents, and administrators to improve coordination and ensure transparency. By automating key tasks such as attendance tracking and student progress reporting, the system helps save valuable time and significantly reduces the chances of human error.

1.3.2 Specific Objectives

One of the key objectives of this system is to implement Role-Based Access Control (RBAC), allowing secure and customized access for different users. The platform will provide four separate dashboards, each tailored to the specific needs of the admin, accountant, teacher, and parent. This ensures that each user only views and interacts with the information relevant to their role.

1.3.3 Technical Objectives

The technical goals of this project focus on building a secure, responsive, and scalable system. To ensure secure user access, the system will implement JWT (JSON Web Tokens) for role-based authentication and authorization. The frontend will be developed using React.js to provide a clean, efficient, and user-friendly interface for all users. For data management, MongoDB Atlas, a reliable cloud-based NoSQL database, will be used to store and manage all records securely, ensuring high availability and real-time access.

1.4 Scope of the Project

This project is mainly focused on making the daily working of a madrasa easier through an online system. The system will work for four main types of users: admin, accountant, teacher, and parent. Each user will have their own logins and can perform specific tasks based on their role.

1.4.1 Included Scope

The proposed system includes four separate portals based on user roles to ensure smooth operations and clarity of responsibilities. The Admin Portal allows the admin to manage accountants and provide them with login credentials to access their dashboard. Through the

Accountant Portal, accountants can manage teachers, parents, and students (excluding student login access) and assign credentials accordingly.

The Teacher Portal enables teachers to mark student attendance, add daily diaries, and track each student's tasks to help in overall performance aggregation. The Parent Portal provides parents with access to their child's attendance records, diary updates, notes, and performance reports in the form of simple, visual charts on a weekly or monthly basis.

Furthermore, the system ensures security through role-based access using JWT tokens, restricting access based on user roles such as admin, accountant, teacher, and parent.

1.4.2 Excluded Scope

This project specifically focuses on the academic and administrative management needs of madrasas; therefore, several features have been intentionally excluded. SMS alert services are not included due to the high cost of integrating with network providers. Similarly, proper fee management is not part of the current scope, as the system is primarily designed to manage attendance, basic fee module and student progress; however, financial features may be added in future updates.

Additionally, social media integration is excluded to maintain data privacy and avoid unnecessary sharing of student progress on platforms like Facebook or WhatsApp. The system also does not support live or recorded video classes, as it is not designed to function as an e-learning platform. Finally, no Learning Management System (LMS) features are included, since the main aim is to streamline administrative workflows rather than deliver online courses.

1.4.3 Technical Scope

The technical scope of this project includes developing a modern and efficient system using widely adopted web technologies. The frontend is built with React.js, providing simple, fast, and responsive dashboards tailored for different user roles. On the backend, Node.js with Express.js is used to handle API requests, ensuring smooth communication between the client and the server. Data storage is managed through MongoDB Atlas, a cloud-based NoSQL database, which securely stores user information and attendance records. For handling images, Cloudinary is integrated to store and manage media files in the cloud efficiently.

1.4.4 Limitations

This system is designed primarily for small to medium-sized madrasas, supporting user capacities of approximately 10 to 15 students. It requires a stable internet connection to function properly, as there is no offline mode available at this time. Additionally, the platform currently supports only the English language, which may limit accessibility for users who prefer other languages.

1.4.5 Constraints

The development of this project is constrained by a time frame of approximately 7 to 8 months, requiring efficient planning and execution. The budget is limited, so the system relies exclusively on free and open-source tools such as the MERN stack and NodeMailer for email functionalities. Additionally, the platform is designed with non-technical users in mind,

featuring simple buttons, minimal menus, and a clean user interface to ensure ease of use for all stakeholders.

1.5 Methodology Summary

To develop this eMadrasa Management System, A step-by-step process followed to make sure everything is done in the right way. First, tried to understand the needs of a real madrasa. Noted what kind of features would be helpful for teachers, parents, accountants, and admins. After understanding all the requirements, start working on the design of the system.

By using the **MERN stack** for building this project. This means **MongoDB** to store data like users, attendance, and daily progress. **Express.js** and **Node.js** were used to build the backend part, Then, used **React.js** to make the frontend, which is the part users see and interact with.

To make sure only the right people can use their part of the system, app use rolebased login. It helps give different features to admin, accountant, teacher, and parent based on their role.

Each step test the features to make sure they work properly. After building all parts, test the complete system and made improvements where needed. In the end, the system became a working solution for madrasa management.

1.6 Tools and Technologies Used

1.6.1 Frontend Development

The frontend of the system is developed using React.js to create user-friendly and role-specific dashboards for accountants, teachers, and parents. To efficiently manage API calls and global state, RTK Query is integrated, providing seamless data fetching and caching. Various React libraries such as React Redux, React Router DOM, React Toastify, and React Bootstrap are utilized to enhance functionality, routing, notifications, and styling. For responsive design, Bootstrap is employed to ensure that the application works smoothly across different devices, including both mobile phones and desktop computers.

1.6.2 Backend Development

The backend of the system is built using Node.js with Express.js to create robust and scalable RESTful APIs that handle all client-server communication. Security is a key focus, with JSON Web Tokens (JWT) implemented to provide secure, role-based authentication, ensuring that users can only access data and features appropriate to their roles. To maintain data integrity and prevent errors, Express Validator is used for validating input data, ensuring it meets the required formats before processing.

1.6.3 Database

The project uses MongoDB Atlas, a cloud-based NoSQL database, to securely store all user information, attendance records, and student progress logs. To interact efficiently with

MongoDB and maintain well-structured data, Mongoose is employed. Mongoose provides schema-based modeling, making it easier to define and enforce data structures within the application.

1.6.4 APIs & Third-Party Services

This project integrates several third-party services and APIs to enhance its functionality. Cloudinary is used for efficient and secure image uploads and storage in the cloud. NodeMailer is incorporated to send email notifications, such as delivering login credentials to users upon account creation. Additionally, the Quran API is integrated to provide Quran reading and listening features, enriching the educational experience within the system.

1.6.5 Testing & Debugging

During development, APIs were thoroughly tested using Postman to ensure proper functionality and response accuracy. Thunder Client, an extension within Visual Studio Code, was also used for quick and convenient API testing directly within the code editor. Additionally, browser consoles played a key role in logging errors and debugging, helping identify and resolve conflicts in both frontend and backend code effectively.

1.6.8 Other Tools

Version control for the project was managed using Git and GitHub, The primary code editor used throughout the development process was Visual Studio Code (VS Code).

CHAPTER 2
LITERATURE REVIEW

2.1 Summary of Similar Existing Systems or Research

There are many school management systems available to help schools with their daily work. One example is Fedena, which is used for taking attendance, making reports, and helping teachers communicate with parents. But Fedena is made for regular schools and does not have features that madrasas need. For example, it cannot track Quran lessons, Hifz progress, or daily Sabak, which are very important for madrasas. Also, Fedena can be hard to use for people who are not very familiar with computers or technology.

Because of this, the eMadrasa Management System was created to fit the special needs of madrasas. It includes important features like daily attendance, tracking of Quranic lessons, progress reports, and easy communication between teachers and parents. It also has different dashboards for admins, teachers, accountants, and parents, so everyone can use it easily.

This system focuses on the unique needs of madrasas, making it simple and useful, unlike Fedena which is made for general schools and can be complicated for madrasa users.

2.1.1 Existing Management Systems

There are some existing systems that schools and madrasas use, but they have limits for madrasa needs. For example, Google Classroom offers features like attendance tracking, assignment submission, and grading. However, it is too complex for madrasas and does not track Quranic progress. Also, some advanced features require a paid subscription.

Fedena is another popular system with features such as fee management, timetable scheduling, and report generation. But it costs more than 10,000 rupees per year, which is too expensive for small madrasas. It also does not support tracking of Sabqi or Manzil, which are important for Quran learning.

There are some Madrasa apps available on the Play Store that provide basic attendance or student listing features. However, these apps lack alert systems, role-based access for different users, automatic report generation, and do not support Quran reading or listening features.

2.2 Analysis of Gaps or Limitations in Current Solutions

Most current madrasa management systems don't work well because they are either too expensive or don't have the right features. While they can handle basic tasks like attendance, they don't track important Quran lesson progress (Sabqi, Manzil, Atkan) that madrasas need every day. These systems usually only give access to admins, leaving teachers and parents without proper ways to check or update student records. Some have smart features but make them too complicated to use. They also rely on paid SMS services to contact parents, which adds extra costs. Many have old-fashioned designs that are hard for non-technical users. This shows why created EMMS a simple, affordable system designed just for madrasas. It includes all the necessary features with an easy-to-use interface for admins, teachers and parents.

2.2.1 How EMMS Fills the Gaps

Feature	Existing Systems	EMMS
Cost	Mostly Paid (e.g., Fedena)	Onetime Fee / Free access for trail purposes
Sabaq Tracking	Not Available or Very Basic	Sabqi, Manzil, Atkan
Alerts and Updates	Paid SMS / Limited Email Alerts	Free Email Alerts (Login Credentials, Forget password OTP)
User Roles	Mostly 2 Roles (Admin, Student)	4 Roles (admin, accountant, teacher, parent)
Technology Stack	Mixed or Outdated Tech	Modern MERN Stack with RTK Query & Cloudinary
Data Security	Basic or Not Focused	RoleBased Access with JWT authentication
User Interface	Not User-Friendly in Most Cases	Clean, RoleBased UI with React Components

Table 1 Analysis Gap

2.3 Justification for your Approach

The eMadrasa system was made to solve the special problems that madrasas face. Many school management systems are expensive and hard to use. This system, however, provides a simple web-based solution that meets the real needs of madrasas. It tracks Quran lesson progress (Sabqi, Manzil, Atkan) and gives separate access for admins, teachers, parents, and accountants, which is something many other systems do not have.

The system uses modern and easy technologies like the MERN Stack to ensure it works quickly and is easy to use. Instead of using costly SMS services, it offers free email alerts. Most importantly, the system is affordable, with just a onetime payment, so even small madrasas can use it without financial stress. This approach effectively meets the real needs of madrasas in a simple and practical way.

Here is the breakdown of the approach for this project:

2.3.1 Focus on Madrasa-Specific Needs

Most school management tools are made for general schools and do not include features to track important Quranic lessons like Sabqi and Manzil. This makes it hard for madrasas to use them properly. The eMadrasa Management System (EMMS) solves this problem by adding

special fields and tools that focus on Islamic education. Unlike Google Classroom or Fedena, EMMS is designed to meet the unique needs of madrasas.

2.3.2 Cost-Effective Technology

Many paid tools like Fedena are too expensive for small madrasas to afford. To solve this, the eMadrasa Management System is built using free and open-source technologies like the MERN stack (React, Node.js, MongoDB) and Netlify for hosting. This keeps the system affordable and easy to use for madrasas with limited budgets.

2.3.3 Simplicity for Non-Tech Users

Many teachers find complex systems like Moodle hard to use because of confusing menus and technical words. To fix this, the eMadrasa Management System is made very simple. Marking attendance takes just a few seconds, and buttons have easy names like “Register” or “Delete” instead of difficult terms. This helps teachers who are not familiar with technology use the system easily.

2.3.4 Security Without Complexity

Many existing systems do not protect user data well and sometimes use weak passwords or no encryption at all. The eMadrasa Management System keeps things safe without making it complicated. It uses JWT tokens for secure login and Cloudinary to store images safely on the cloud. This way, the system stays secure while remaining easy to use.

2.3.5 Scalability for Growth

Many older systems use outdated tools that make it hard to grow or add new features. To avoid this, the eMadrasa Management System uses modern technology like MongoDB Atlas, a cloud database that can easily handle more data as the madrasa grows. It also uses RTK Query to manage API calls efficiently, which helps the system work faster and smoother even with more users.

CHAPTER 3
SYSTEM ANALYSIS

3.1 Requirements Gathering

Before starting the development of eMadrasa Management System (EMMS), it was important to understand what users actually need from this system. So, began the process by talking to different people who are directly connected with madrasa management like teachers, parents, admin staff, and accountants.

Survey Story for requirements gathering:

The idea for this system started when the developer went to pick up a younger brother from madrasa. While waiting, the developer sat in the manager's office and talked with him. The manager asked what the developer did, and the developer said they were a BSCS student and made websites.

The developer then asked how the madrasa managed attendance, results, and student progress. The manager explained that they wrote everything by hand in a register, like daily attendance and how much sabaq each student had memorized. The developer suggested that they should use a software system for this. When the manager learned that the developer could create websites, he asked if the developer could make a system for them. The developer agreed and decided to make this project for their Final Year Project.

After that, the developer talked more with the manager to understand what was needed. The manager said they tracked each student's sabaq, manzil, and atkan every day, but the manual way was not helpful. Parents could not see their children's progress. The manager wanted a system where they could easily enter data, and the system would show the student's overall progress clearly.

The manager also said the system should have different roles, like admin, accountant, teacher, and parent, each with their own logins and functions. He even gave the developer a sample mobile app to look at. This helped the developer understand how to design the user interface and what forms were needed.

This real-life talk helped the developer see the actual needs of the madrasa and gather all the requirements directly from the madrasa.

Key Findings:

- **Accountant Needs:** Provide login to teacher and parent, manage students.
- **Teacher Needs:** Mark attendance, update student sabaq, add diary or notes.
- **Parent Needs:** View child progress, receive diaries, see attendance records.

3.2 Functional Requirements

The eMadrasa Management System has different functional features for each user role like admin, accountant, teacher, and parent. Each user can login in to their own portal using the login system. If anyone forgets their password, they can recover it through the forget password feature.

Below will list all the functional requirements of this project:

3.2.1 FR01: Sign Up

Req. No	FR_01
FR01-01	The system will enable the user to register with their phone number and password.

Table 2 FR01 Sign Up

3.2.2 FR02: Reset Password

Req. No	FR_02
FR02-01	The system will enable the user to reset their password using the "forgot password" option.

Table 3 FR02 Reset Password

3.2.3 FR03: Reset Password OTP

Req. No	FR_03
FR03-01	The system will enable the user to enter their email address for receiving password reset OTP.

Table 4 FR03 Reset Password OTP

3.2.4 FR04: Change Password

Req. No	FR_04
FR04-01	The system will verify OTP and enable the user to enter new password and confirm password to change its password.

Table 5 FR04 Change Password

3.2.5 FR05: Login Account

Req. No	FR_05
FR05-01	The system will allow the admin/ accountant/ teacher/ parent to enter their phone number and password.
FR05-02	The system will direct the admin/ accountant/ teacher/ parent to the main dashboard screen upon successful login.

Table 6 FR05 Login Account

3.2.6 FR06: Dashboard

Req. No	FR_06
FR06-01	The system will enable the user to access their portal according to their role (admin/ accountant/ teacher/ parent)
FR06-02	The system will allow the user to manage madrasa from the dashboard based on their role. For example, Teacher portal has facility to mark attendance of students.

Table 7 FR06 Dashboard

3.2.7 FR07: Create/View/ Update/Delete Details

Req. No	FR_07
FR07-01	The system will allow the admin/ accountant to create/ view/ update/ delete information of teacher, parent, student. The admin has facility to create accountant and provide credentials.

Table 8 FR07 CRUD Operations

3.2.8 FR08: Read and Listen Quran

Req. No	FR_08
FR08-01	The system will allow the user to read and listen Quran through his/her portal. I host this module on server so that user redirect separately and access site.

Table 9 FR08 Read and Listen Quran

3.2.9 FR09: Admin Portal

Req. No	FR_09
FR09-01	The system allows the admin to access its portal after login.
FR09-02	Admin have access to create accountant profile.
FR09-03	Admin have access to update accountant profile.
FR09-04	Admin have access to delete accountant profile.
FR09-05	Admin have access to view all the list of registered accountants.
FR09-06	Admin have access to change the password of accountant profile.

Table 10 FR09 Admin Portal

3.2.10 FR 10: Accountant Portal

Req. No	FR_10
FR10-01	The system allows the accountant to access its portal after login.
FR10-02	Accountant have access to add student profile.
FR10-03	Admin have access to update student profile.
FR10-04	Admin have access to delete student profile.
FR10-05	Admin have access to view all the list of students.
FR10-06	Accountant have access to create teacher profile.
FR10-07	Accountant have access to update teacher profile.
FR10-08	Accountant have access to delete teacher profile.
FR10-09	Accountant have access to view all the list of registered teachers.
FR10-10	Accountant have access to create parent profile.
FR10-11	Accountant have access to update parent profile.
FR10-12	Accountant have access to delete parent profile.
FR10-13	Accountant have access to view all the list of registered parents.
FR10-15	Accountant have access to change the password of teacher profile.
FR10-16	Accountant have access to change the password of parent profile.
FR11-17	Accountant have access to view the aggregated student performance.

Table 11 FR10 Accountant Portal

3.2.11 FR 11: Teacher Portal

Req. No	FR_11
FR11-01	The system allows the teacher to access its portal after login.
FR11-02	Teacher has access to mark student attendance.
FR11-03	Teacher has access to update student lesson (sabqi, atkan, manzil).
FR11-04	Teacher has access to read and listen Quran.

Table 12 FR11 Teacher Portal

3.2.12 FR 12: Parent Portal

Req. No	FR_12
FR12-01	The system allows the parent to access its portal after login.
FR12-02	Parent has access to view the aggregated progress of his/her child.
FR12-03	Parent has access to view attendance record of his/her child.
FR12-04	Parent has access to view daily diaries, updates or notes.
FR12-05	Parent has access to read and listen Quran.

Table 13 FR12 Parent Portal

3.2.13 FR 13: Logout

Req. No	FR_13
FR13-01	The system allows the user to logout of their account.
FR13-02	The system will direct the user to the login screen upon successful logout.

Table 14 FR13 Logout

More functional modules are added as per need of madrasa this makes the system more flexible and scalable.

3.3 Non-Functional Requirements

The eMadrasa Management System is not only focused on what the system does but also on how well it does those things. These are called non-functional requirements. They make sure the system works smoothly, is safe to use, and gives a good experience to the users.

3.3.1 Performance

The system is designed to be fast and responsive so that all users can work without delays. It should handle many users at the same time without slowing down. Also, when processing data or running searches, the system should give quick and correct results, even if there is a lot of data to manage.

3.3.2 Security

The system makes sure that all users log in securely using a strong authentication process. It uses role-based access control so that each user can only see and use the features meant for

their role. Also, sensitive information like passwords is kept safe by encrypting the data both when it is sent over the internet and when it is stored.

3.3.3 Usability

The system's interface is designed to be simple and easy to use, with consistent navigation on all pages so users don't get confused. It also follows accessibility rules to make sure people with disabilities can use the system comfortably.

3.3.4 Reliability

The system is built to stay online and available almost all the time, aiming for 99.9% uptime. If any errors happen, the system handles them smoothly by showing clear and helpful messages to users without confusing technical details.

3.3.5 Maintainability

The system's code is written carefully following clean coding standards, which means it is easy to read and understand. Good documentation is also provided, so anyone working on the project in the future can quickly learn how it works and make improvements. The system is designed in a modular way, meaning it is divided into smaller parts or components. This makes it possible to update or replace one part without causing problems in other areas. To keep the system reliable, automated tests are set up to regularly check if everything is working correctly. These tests help catch errors early, especially when new changes or updates are made.

3.4 Gantt Chart



Figure 1 Gantt Chart

3.5 Use Case Diagrams

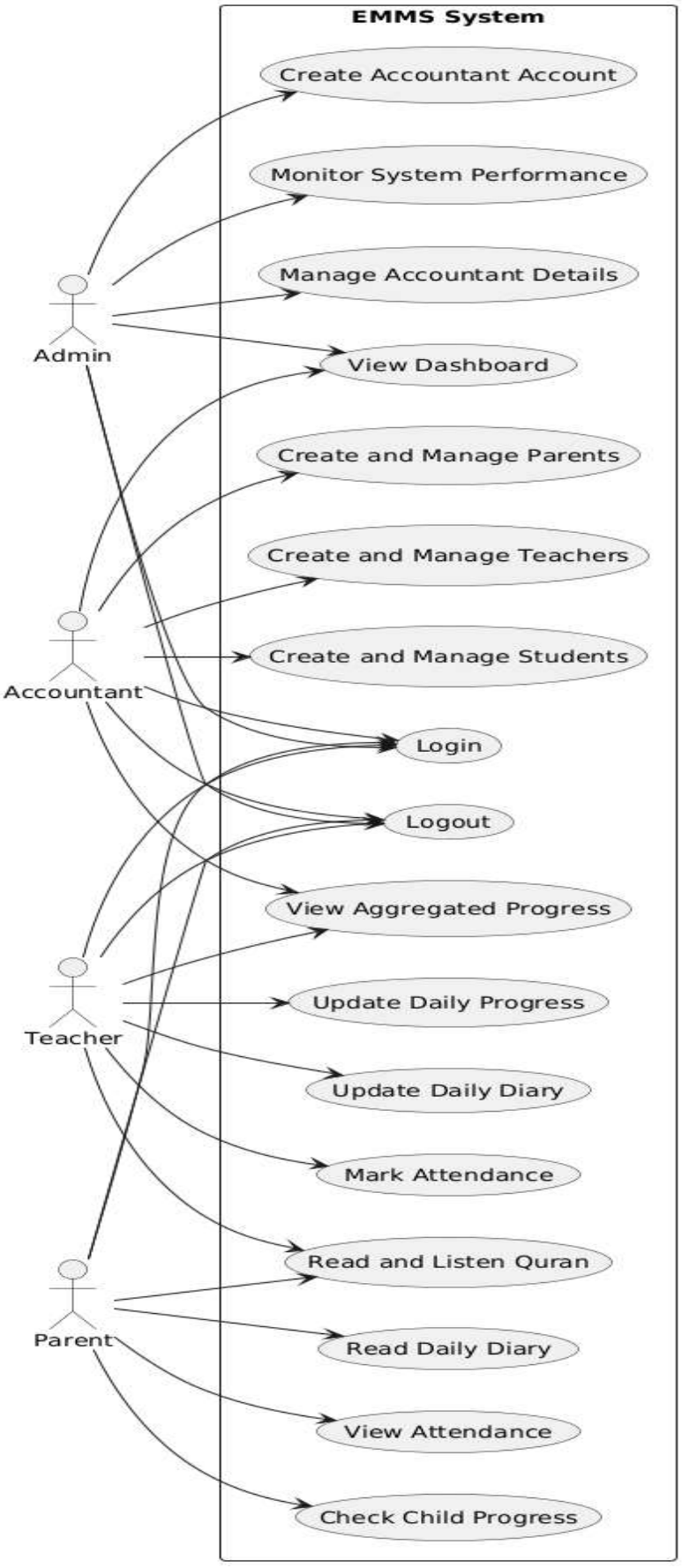


Figure 2 Complete System Use Case Diagram

3.5.1 Extended Admin Use Case

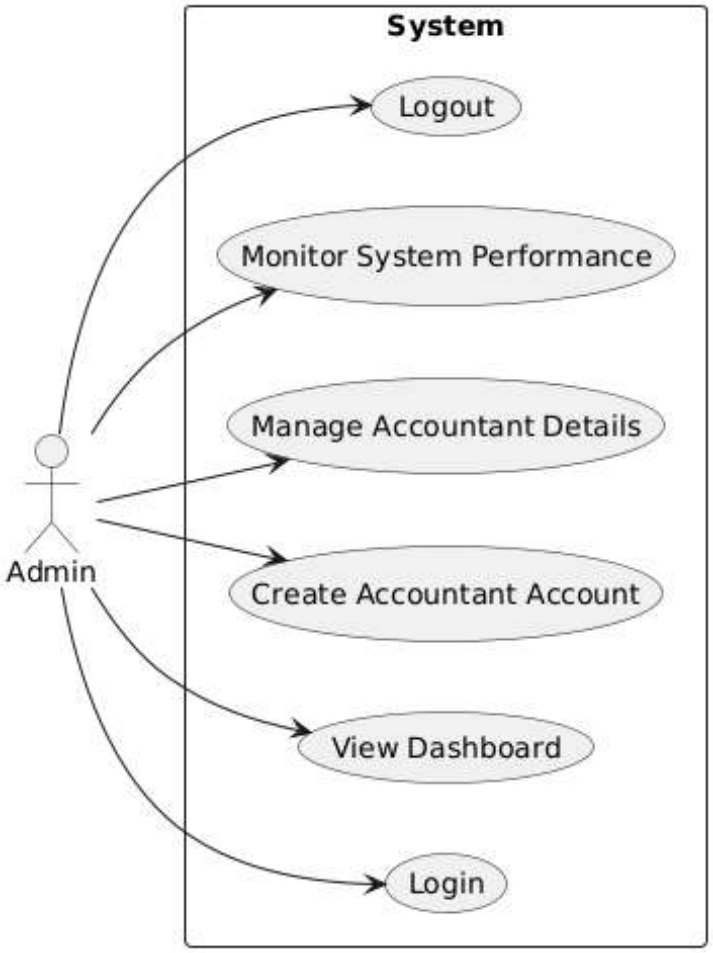


Figure 3 Admin Use Case Diagram

3.5.1.1 Admin Login

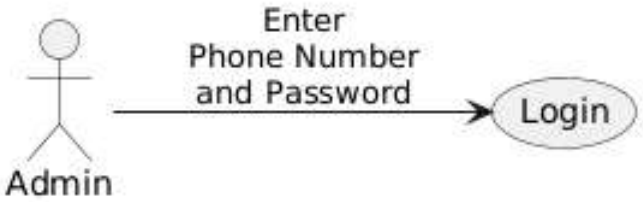


Figure 4 UC1-01

Use Case Description:

Use Case ID	UC1-01	
Use Case Name	Admin Login	
Description	Allows the admin to securely log in to the EMMS system to access their dashboard and manage accountants.	
Primary Actor	Admin	
PreCondition	The admin must be a registered with valid credentials (phone number & password).	
Post-Condition	The admin is authenticated and redirected to the admin dashboard.	
Basic Flow	**Actor Action**	**System Action**
	1. The use case starts when the admin opens the EMMS login page. 2. The admin enters a valid email and password. 3. The system verifies the credentials. 4. If valid, the system logs in the admin and redirects to the dashboard. 5. The session is securely created and maintained.	• The system authenticates the admin and creates a session, granting access to the admin dashboard.
Alternate Flow	**-** If the credentials are invalid, the system shows an error message. **-** Admin can choose "Forgot Password" to recover access.	

Table 15 UC1-01

3.5.1.2 Admin View Dashboard

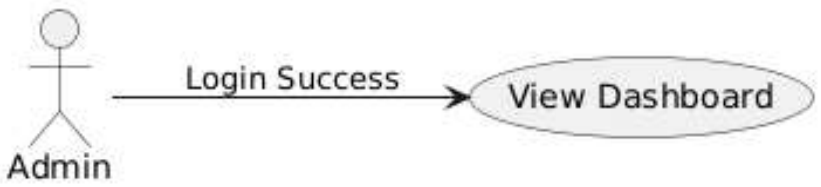


Figure 5 UC1-02

Use Case Description:

Use Case ID	UC1-02	
Use Case Name	Admin View Dashboard	
Description	Allows the admin to view an overview of the system, including managing accountant profiles and provide credentials.	
Primary Actor	Admin	
PreCondition	Admin must be logged into the EMMS system.	
Post-Condition	The admin can view real-time accountant profiles on the dashboard.	
Basic Flow	**Actor Action**	**System Action**
	1. The admin enters a valid email and password to access their dashboard. 2. The system fetches and displays key metrics like accountant profiles and provide facility to update, delete, view and create them. 3. System allow the user to logout from their portal.	• System queries the database and generates the dashboard data in real time.
Alternate Flow	**-** If the system fails to fetch data, an error message is displayed: “Unable to load dashboard. Please try again later.”	

Table 16 UC1-02

3.5.1.3 Admin Create Accountant

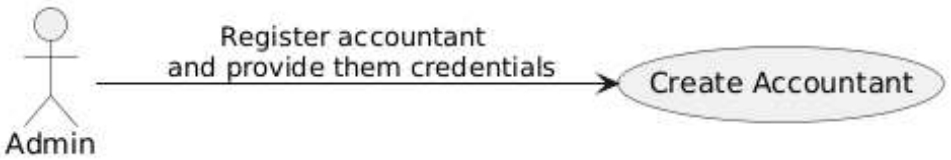


Figure 6 UC1-03

Use Case Description:

Use Case ID	UC1-03	
Use Case Name	Admin create Accountant	
Description	Allows the admin to create a new accountant account with relevant information like name, email, phone number, and role assignment.	
Primary Actor	Admin	
PreCondition	Admin must be logged into the EMMS system.	
Post-Condition	The accountant's information is successfully stored, and their account is created in the system.	
Basic Flow	**Actor Action**	**System Action**
	1. Admin navigates to the "Create User" or "Add Accountant" section. 2. Admin fills in the details: Full Name, Email, Phone Number, and Password. 3. Admin selects the "Role" as Accountant and submits the form.	• System validates the input fields. • System creates an accountant user with the specified credentials. • System stores the data in the database and sends an email with login credentials to the provided email.
Alternate Flow	**-** If the email already exists in the system, an error is shown: “User with this email already exists.” **-** If any required field is empty or invalid, validation messages are displayed.	

Table 17 UC1-03

3.5.1.4 Admin Update Accountant

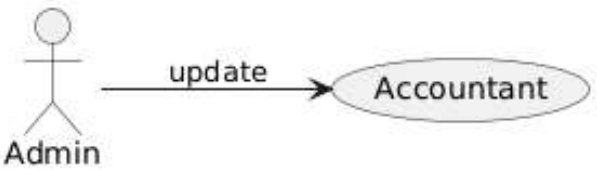


Figure 7 UC1-04

Use Case Description:

Use Case ID	UC1-04	
Use Case Name	Admin Update Accountant	
Description	Enables the admin to update details of accountant from the system.	
Primary Actor	Admin	
PreCondition	Admin must be logged into the EMMS system.	
Post-Condition	Selected accountant information is updated and the system reflects the changes in real time.	
Basic Flow	**Actor Action**	**System Action**
	1. Admin navigates to the "Update Accountants" section. 2. Admin selects a specific accountant to update. 3. Admin modifies fields like name, email, or phone and submits the form.	• System validates the input fields. • System stores the updated data into the database.
Alternate Flow	**-** If the email already exists in the system, an error is shown: “User with this email already exists.” **-** If any required field is empty or invalid, validation messages are displayed.	

Table 18 UC1-04

3.5.1.5 Admin Delete Accountant

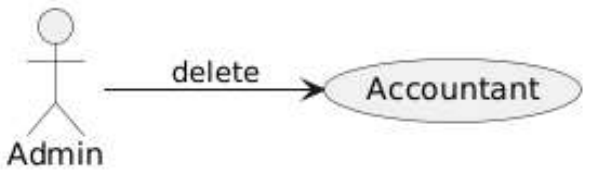


Figure 8 UC1-05

Use Case Description:

Use Case ID	UC1-05	
Use Case Name	Admin Delete Accountant	
Description	Enables the admin to delete details of accountant from the system.	
Primary Actor	Admin	
PreCondition	Admin must be logged into the EMMS system.	
Post-Condition	Selected accountant is deleted and the system reflects the changes in real time.	
Basic Flow	**Actor Action**	**System Action**
	1. Admin navigates to the "Delete Accountants" section. 2. Admin selects a specific accountant to delete.	• System validates the existence of profile. • System delete profile from the database.
Alternate Flow	**-** None	

Table 19 UC1-05

3.5.1.6 Admin Monitor System Performance

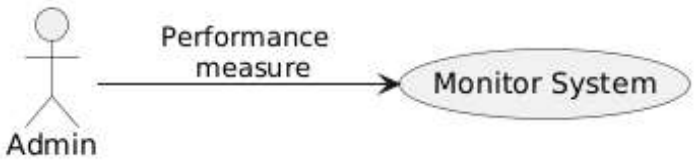


Figure 9 UC1-06

Use Case Description:

Use Case ID	UC1-06	
Use Case Name	Admin Monitor System Performance	
Description	This use case allows the admin to keep an eye on key metrics like user activities, error logs, API usage, and data flow within the EMMS platform to ensure everything runs smoothly.	
Primary Actor	Admin	
PreCondition	Admin must be logged into the EMMS system.	
Post-Condition	The admin successfully views the system's load and performance metrics.	
Basic Flow	**Actor Action**	**System Action**
	1. Admin navigates to the "All users" section from the dashboard. 2. Admin see the number of accountants or admins enrolled. 3. Admin remove extra users for better performance.	• Fetches and displays real-time users from the backend. • Provides visual charts and summaries of current system performance. • Allows exporting of data if needed.
Alternate Flow	**-** If the monitoring tool is temporarily unavailable, system shows a message and logs the failure. **-** If metrics cannot be fetched due to a backend issue, an alert is generated for the admin.	

Table 20 UC1-06

3.5.1.7 Admin Logout



Figure 10 UC1-07

Use Case Description:

Use Case ID	UC1-07	
Use Case Name	Admin Logout	
Description	This use case allows the admin to securely end their session from the EMMS system to protect the platform from unauthorized access.	
Primary Actor	Admin	
PreCondition	Admin must be logged into the EMMS system.	
Post-Condition	The admin is successfully logged out, and the session is terminated securely.	
Basic Flow	**Actor Action**	**System Action**
	1. Admin clicks on a dropdown menu from the navigation bar. 2. Admin clicks the "Logout" button from drop down menu.	• Ends the admin session. • Redirects them to the login screen. • Clears all temporary session-related data.
Alternate Flow	**-** If the session has expired, the system automatically logs out the admin.	

Table 21 UC1-07

3.5.2 Extended Accountant Use Case

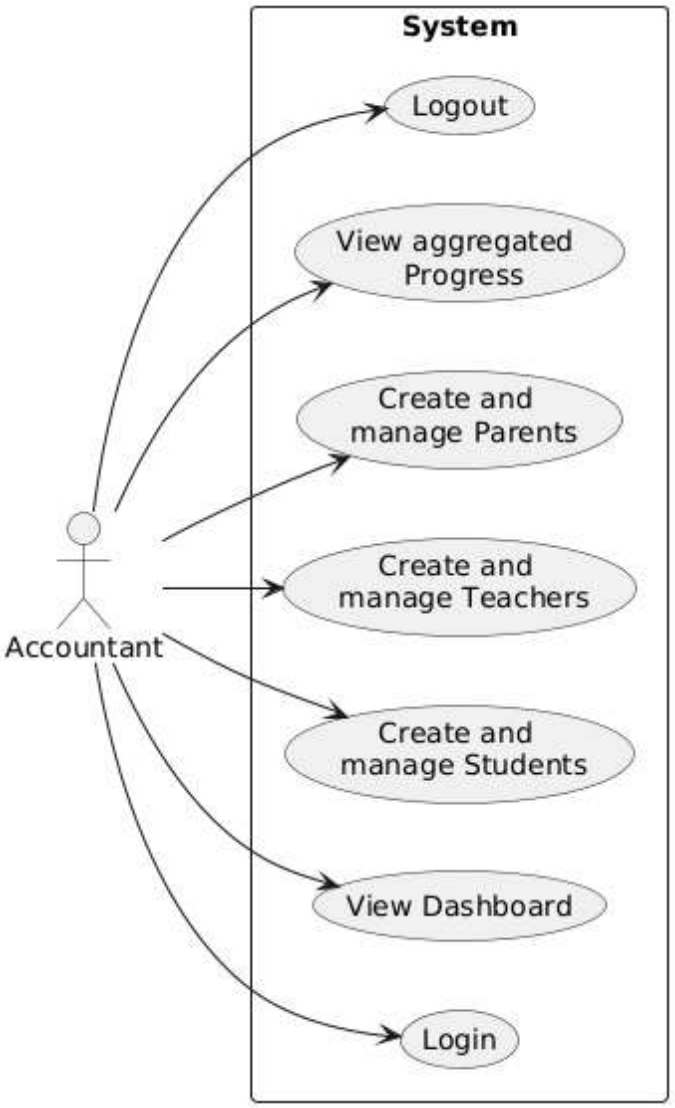


Figure 3.5.2 Accountant Use Case Diagram

3.5.2.1 Accountant Login

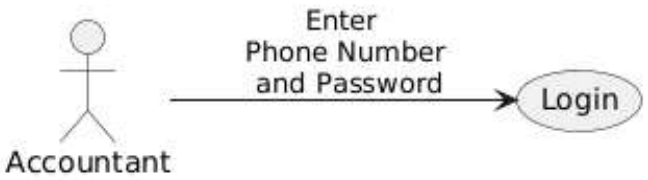


Figure 11 UC2-01

Use Case Description:

Use Case ID	UC2-01	
Use Case Name	Accountant Login	
Description	This use case allows the accountant to securely access the EMMS system using valid login credentials.	
Primary Actor	Accountant	
PreCondition	The accountant must be registered in the system and be on the login screen.	
Post-Condition	The accountant is authenticated and redirected to the accountant dashboard.	
Basic Flow	**Actor Action**	**System Action**
	1. Accountant opens the EMMS login page. 2. Enters **Phone Number** and **Password**. 3. Clicks the **Login** button.	• Verifies the entered credentials. • If valid, logs in the accountant and redirects them to the dashboard. • If invalid, shows an error message.
Alternate Flow	**-** If the credentials are invalid, the system shows an error message.	

Table 22 UC2-01

3.5.2.2 Accountant View Dashboard



Figure 12 UC2-02

Use Case Description:

Use Case ID	UC2-02	
Use Case Name	Accountant View Dashboard	
Description	This use case allows the accountant to access and view the EMMS dashboard, which presents an overview of student, teacher, and parent data and summary stats.	
Primary Actor	Accountant	
PreCondition	The accountant must be registered in the system and be on the login screen.	
Post-Condition	The accountant access and manage different users.	
Basic Flow	**Actor Action**	**System Action**
	1. Accountant logs into the system. 2. Navigates to the dashboard section after login success.	• Fetches all users data (e.g., student, teacher, parents, progress summaries). • Displays the data in a user-friendly layout.
Alternate Flow	**-** If the system fails to fetch data, an error message is displayed: “Unable to load dashboard. Please try again later.”	

Table 23 UC2-02

3.5.2.3 Accountant Create Student

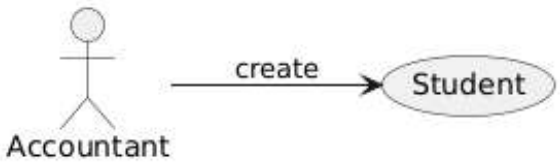


Figure 13 UC2-03

Use Case Description:

Use Case ID	UC2-03	
Use Case Name	Accountant create Student	
Description	This use case allows the accountant to add a new student to the system by filling out a registration form with required student details.	
Primary Actor	Accountant	
PreCondition	The accountant must be registered in the system and be on the login screen.	
Post-Condition	The student is added successfully and saved in the database.	
Basic Flow	**Actor Action**	**System Action**
	1. Accountant goes to "Student" section from dashboard. 2. Clicks on "Register Student." 3. Fills in student details such as name, gender, class, admission date etc. 4. Submits the form.	• Validates all input fields. • Saves new student record to the database.
Alternate Flow	**-** If validation fails, The system prompts the accountant to correct invalid or missing fields.	

Table 24 UC2-03

3.5.2.4 Accountant Update Student

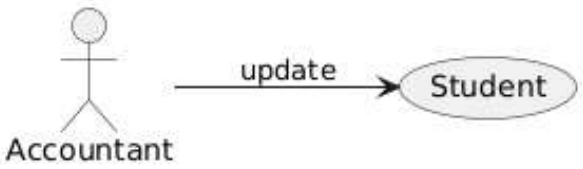


Figure 14 UC2-04

Use Case Description:

Use Case ID	UC2-04	
Use Case Name	Accountant update Student	
Description	This use case allows the accountant to update or change details of an already registered student.	
Primary Actor	Accountant	
PreCondition	The accountant must be registered in the system and be on the login screen.	
Post-Condition	The student is updated successfully and saved in the database.	
Basic Flow	**Actor Action**	**System Action**
	1. Accountant goes to "Student" section from dashboard. 2. Clicks on "Update Student." 3. Changes the required fields like name, class, or admission date. 4. Submits the form.	• Validates all input fields. • Saves updated student record to the database.
Alternate Flow	**-** If validation fails, The system prompts the accountant to correct invalid or missing fields.	

Table 25 UC2-04

3.5.2.5 Accountant Delete Student

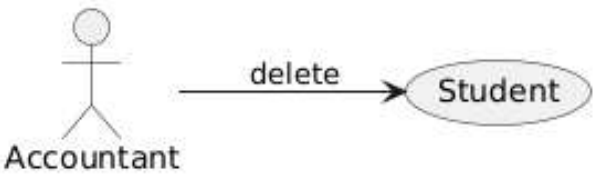


Figure 15 UC2-05

Use Case Description:

Use Case ID	UC2-05	
Use Case Name	Accountant delete Student	
Description	This use case allows the accountant to delete record of student.	
Primary Actor	Accountant	
PreCondition	The accountant must be registered in the system and be on the login screen.	
Post-Condition	The student is deleted successfully and removed from the database.	
Basic Flow	**Actor Action**	**System Action**
	1. Accountant goes to "Student" section from dashboard. 2. Clicks on "Delete Student."	• System displays a list of all registered students. • System shows a delete option. • System removes the student's data from the database and updates the list.
Alternate Flow	**-** None	

Table 26 UC2-05

3.5.2.6 Accountant Create Teacher

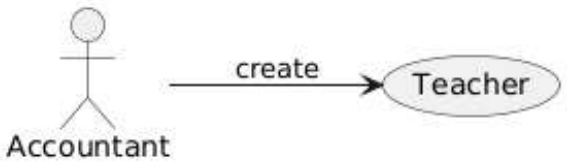


Figure 16 UC2-06

Use Case Description:

Use Case ID	UC2-06	
Use Case Name	Accountant create Teacher	
Description	This use case allows the accountant to add a new teacher to the system by filling out a registration form with required teacher details.	
Primary Actor	Accountant	
PreCondition	The accountant must be registered in the system and be on the login screen.	
Post-Condition	The teacher is added successfully and saved in the database.	
Basic Flow	**Actor Action**	**System Action**
	1. Accountant goes to "Student" section from dashboard. 2. Clicks on "Register Teacher." 3. Fills in teacher details such as name, gender, class, salary etc. 4. Submits the form.	• Validates all input fields. • Saves new teacher record to the database.
Alternate Flow	**-** If validation fails, The system prompts the accountant to correct invalid or missing fields.	

Table 27 UC2-06

3.5.2.7 Accountant Update Teacher

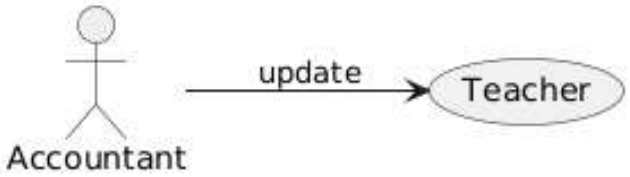


Figure 17 UC2-07

Use Case Description:

Use Case ID	UC2-07	
Use Case Name	Accountant update Teacher	
Description	This use case allows the accountant to update or change details of an already registered teacher.	
Primary Actor	Accountant	
PreCondition	The accountant must be registered in the system and be on the login screen.	
Post-Condition	The teacher is updated successfully and saved in the database.	
Basic Flow	**Actor Action**	**System Action**
	4.1 Accountant goes to "Teacher" section from dashboard. 4.2 Clicks on "Update Teacher." 4.3 Changes the required fields like name, class, password. 4.4 Submits the form.	• Validates all input fields. • Saves updated teacher record to the database.
Alternate Flow	**-** If validation fails, The system prompts the accountant to correct invalid or missing fields.	

Table 28 UC2-07

3.5.2.8 Accountant Delete Student

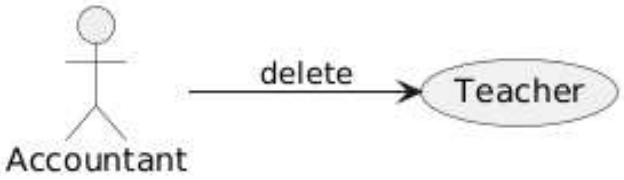


Figure 18 UC2-08

Use Case Description:

Use Case ID	UC2-08	
Use Case Name	Accountant delete Teacher	
Description	This use case allows the accountant to delete record of teacher.	
Primary Actor	Accountant	
PreCondition	The accountant must be registered in the system and be on the login screen.	
Post-Condition	The teacher is deleted successfully and removed from the database.	
Basic Flow	**Actor Action**	**System Action**
	1. Accountant goes to "Teacher" section from dashboard. 2. Clicks on "Delete" button.	• System displays a list of all registered teachers. • System shows a delete option. • System removes teacher data from the database and updates the list.
Alternate Flow	**-** None	

Table 29 UC2-08

3.5.2.9 Accountant Create Parent

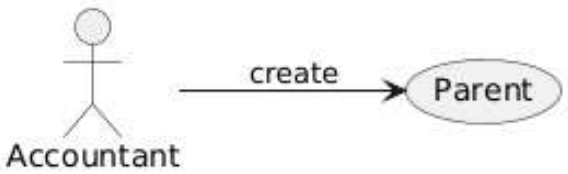


Figure 19 UC2-09

Use Case Description:

Use Case ID	UC2-09	
Use Case Name	Accountant create Parent	
Description	This use case allows the accountant to add a new parent to the system by filling out a registration form with required parent details.	
Primary Actor	Accountant	
PreCondition	The accountant must be registered in the system and be on the login screen.	
Post-Condition	The parent is added successfully and saved in the database.	
Basic Flow	**Actor Action**	**System Action**
	1. Accountant goes to "Parent" section from dashboard. 2. Clicks on "Register Parent." 3. Fills in parent details such as name, password, phone number, child_ID etc. 4. Submits the form.	• Validates all input fields. • Saves new parent record to the database.
Alternate Flow	**-** If validation fails, The system prompts the accountant to correct invalid or missing fields.	

Table 30 UC2-09

3.5.2.10 Accountant Update Parent



Figure 20 UC2-10

Use Case Description:

Use Case ID	UC2-10	
Use Case Name	Accountant update Parent	
Description	This use case allows the accountant to update or change details of an already registered parent.	
Primary Actor	Accountant	
PreCondition	The accountant must be registered in the system and be on the login screen.	
Post-Condition	The parent is updated successfully and saved in the database.	
Basic Flow	**Actor Action**	**System Action**
	1. Accountant goes to "Parent" section from dashboard. 2. Clicks on "Update Parent." 3. Changes the required fields like name, password, avatar etc. 4. Submits the form.	• Validates all input fields. • Saves updated parent record to the database.
Alternate Flow	**-** If validation fails, The system prompts the accountant to correct invalid or missing fields.	

Table 31 UC2-10

3.5.2.11 Accountant Delete Parent

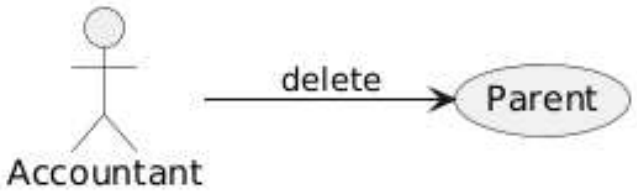


Figure 21 UC2-11

Use Case Description:

Use Case ID	UC2-08	
Use Case Name	Accountant delete Parent	
Description	This use case allows the accountant to delete record of parent.	
Primary Actor	Accountant	
PreCondition	The accountant must be registered in the system and be on the login screen.	
Post-Condition	The parent is deleted successfully and removed from the database.	
Basic Flow	**Actor Action**	**System Action**
	1. Accountant goes to "Parent" section from dashboard. 2. Clicks on "Delete" button.	• System displays a list of all registered parents. • System shows a delete option. • System removes parent data from the database and updates the list.
Alternate Flow	**-** None	

Table 32 UC2-11

3.5.2.12 Accountant view Progress

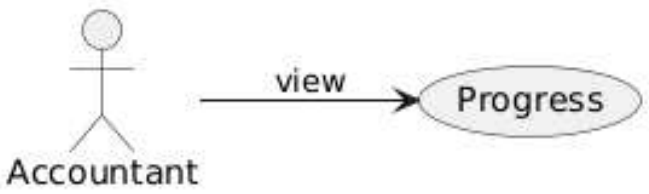


Figure 22 UC2-12

Use Case Description:

Use Case ID	UC2-12	
Use Case Name	Accountant view Progress	
Description	This use case allows the accountant to view the aggregated progress of all students, including their attendance, and sabqi/manzil/atkan records in one place for performance monitoring and reporting.	
Primary Actor	Accountant	
PreCondition	The accountant must be registered in the system and be on the login screen.	
Post-Condition	The accountant successfully view detailed progress summaries for individual students.	
Basic Flow	**Actor Action**	**System Action**
	1. Accountant clicks on the "Progress Report" or "Performance Summary" section. 2. Selects filters (e.g., class, section, individual student. 3. Clicks "View Report" to see the aggregated progress.	• Retrieves and processes progress data from the database. • Displays progress in a structured, readable format (i.e charts).
Alternate Flow	**-** If no data is found the system displays a "No Data Found" message.	

Table 33 UC2-12

3.5.2.13 Accountant Logout



Figure 23 UC2-13

Use Case Description:

Use Case ID	UC2-13	
Use Case Name	Accountant Logout	
Description	This use case allows the accountant to securely end their session from the EMMS system to protect the platform from unauthorized access.	
Primary Actor	Accountant	
PreCondition	Accountant must be logged into the EMMS system.	
Post-Condition	The accountant is successfully logged out, and the session is terminated securely.	
Basic Flow	**Actor Action**	**System Action**
	1. Accountant clicks on a dropdown menu from the navigation bar. 2. Accountant clicks the "Logout" button from drop down menu.	• Ends the accountant session. • Redirects them to the login screen. • Clears all temporary session-related data.
Alternate Flow	**-** If the session has expired, the system automatically logs out the accountant.	

Table 34 UC2-13

3.5.3 Extended Teacher Use Case

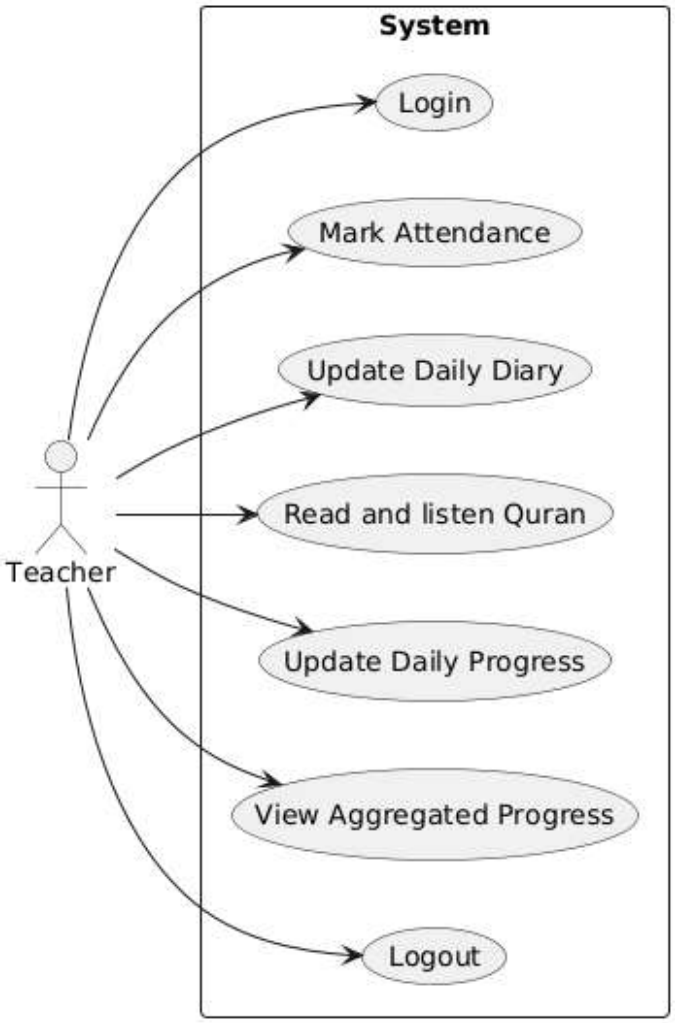


Figure 24 Teacher Use Case Diagram

3.5.3.1 Teacher Login

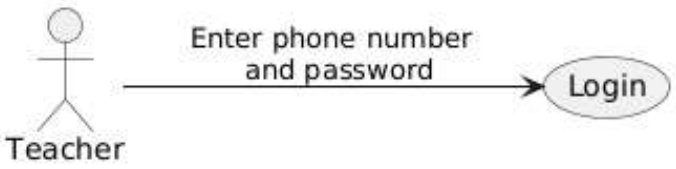


Figure 25 UC3-01

Use Case Description:

Use Case ID	UC3-01	
Use Case Name	Teacher Login	
Description	Allows the teacher to securely log in to the EMMS system to access their dashboard and manage tasks.	
Primary Actor	Teacher	
PreCondition	The teacher must be a registered with valid credentials (phone number & password).	
Post-Condition	The teacher is authenticated and redirected to the teacher dashboard.	
Basic Flow	**Actor Action**	**System Action**
	1. The use case starts when the teacher opens the EMMS login page. 2. The teacher enters a valid email and password. 3. The system verifies the credentials. 4. If valid, the system logs in the teacher and redirects to the dashboard. 5. The session is securely created and maintained.	• The system authenticates the teacher and creates a session, granting access to the teachers dashboard.
Alternate Flow	**-** If the credentials are invalid, the system shows an error message.	

Table 35 UC3-01

3.5.3.2 Teacher Mark Attendance

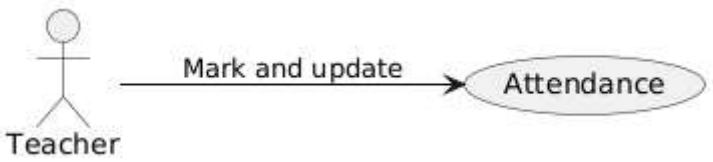


Figure 26 UC3-02

Use Case Description:

Use Case ID	UC3-02	
Use Case Name	Teacher Mark Attendance	
Description	This use case allows a teacher to mark attendance for their assigned students.	
Primary Actor	Teacher	
PreCondition	The teacher must be logged in to the system.	
Post-Condition	Attendance record is successfully saved into the database.	
Basic Flow	**Actor Action**	**System Action**
	1. Teacher logs into the system. 2. Navigates to the **Attendance** section from the dashboard. 3. Selects the student for marking attendance. 4. Marks each student as present or absent. 5. Submits the attendance form.	• Displays the list of students based on the selected class. • Validates the form input before submission. • Saves attendance records into the database.
Alternate Flow	**-** If no class is assigned to the teacher, show a message: "No class assigned.".	

Table 36 UC3-02

3.5.3.3 Update Daily Dairy

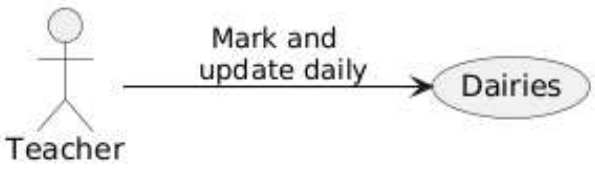


Figure 27 UC3-03

Use Case Description:

Use Case ID	UC3-03	
Use Case Name	Update Daily Dairy	
Description	This use case allows a teacher to update the daily diary with classwork, homework, and announcements for students.	
Primary Actor	Teacher	
PreCondition	The teacher must be logged in to the system.	
Post-Condition	Diary updates are successfully saved and viewable by parents.	
Basic Flow	**Actor Action**	**System Action**
	1. Teacher logs into the system. 2. Navigates to the **Diary** section from the dashboard. 3. Selects the relevant student. 4. Enters classwork, homework, and any announcements. 5. Submits the daily diary update.	• Displays class and student details. • Validates diary content input. • Stores the diary entry in the database.
Alternate Flow	**-** If teacher forgets to fill any required field, system shows: "Please complete all required fields."	

Table 37 UC3-03

3.5.3.4 Update Student Progress

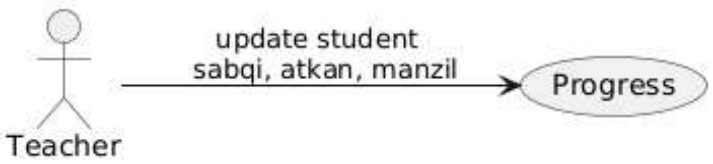


Figure 28 UC3-04

Use Case Description:

Use Case ID	UC3-04	
Use Case Name	Update Student Progress	
Description	This use case allow the teacher to update the student's daily Hifz progress, including Sabqi, Atkan and Manzil.	
Primary Actor	Teacher	
PreCondition	The teacher must be logged in to the system.	
Post-Condition	The updated progress is saved and visible to parents and the accountant in reports.	
Basic Flow	**Actor Action**	**System Action**
	1. Teacher logs into the system. 2. Navigates to the **Student Progress** section. 3. Selects the student. 4. Enters the details for **Sabqi**, **Atkan** and **Manzil**. 5. Submits the progress entry.	• Retrieves list of students. • Validates the entered progress details. • Saves the progress data in the database. • Automatically updates aggregated progress for the accountant and parent views.
Alternate Flow	**-** If the input is invalid (e.g., empty fields), system prompts: "Please enter valid progress data."	

Table 38 UC3-04

3.5.3.6 Teacher reads and listen Quran

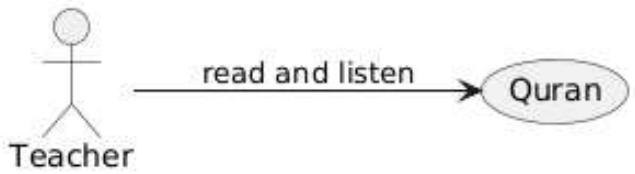


Figure 29 UC3-05

Use Case Description:

Use Case ID	UC3-05	
Use Case Name	Reads and listen Quran	
Description	This use case allows the teacher to open and read any Surah and listen to the recitation of Quran.	
Primary Actor	Teacher	
PreCondition	The teacher must be logged in to the system.	
Post-Condition	The selected Surah is opened and audio is played if requested.	
Basic Flow	**Actor Action**	**System Action**
	1. Navigates to the **Quran Section**. 2. Selects a Surah from the list. 3. Views the verses of the Surah. 4. Clicks on a verse to listen to its audio.	• Fetches Surah list from the Quran API. • Displays the complete Surah in Arabic. • Streams audio for the selected verse using an external API. • Ensures smooth playback with a play/pause button.
Alternate Flow	None	

Table 39 UC3-05

3.5.3.6 Teacher Logout



Figure 30 UC3-06

Use Case Description:

Use Case ID	UC3-06	
Use Case Name	Teacher Logout	
Description	This use case allows the teacher to securely end their session from the EMMS system to protect the platform from unauthorized access.	
Primary Actor	Teacher	
PreCondition	Teacher must be logged into the EMMS system.	
Post-Condition	The teacher is successfully logged out, and the session is terminated securely.	
Basic Flow	**Actor Action**	**System Action**
	1. Teacher clicks on a dropdown menu from the navigation bar. 2. Teacher clicks the "Logout" button from drop down menu.	• Ends the teacher session. • Redirects them to the login screen. • Clears all temporary session-related data.
Alternate Flow	**-** If the session has expired, the system automatically logs out the teacher.	

Table 40 UC3-06

3.5.4 Extended Parent Use Case

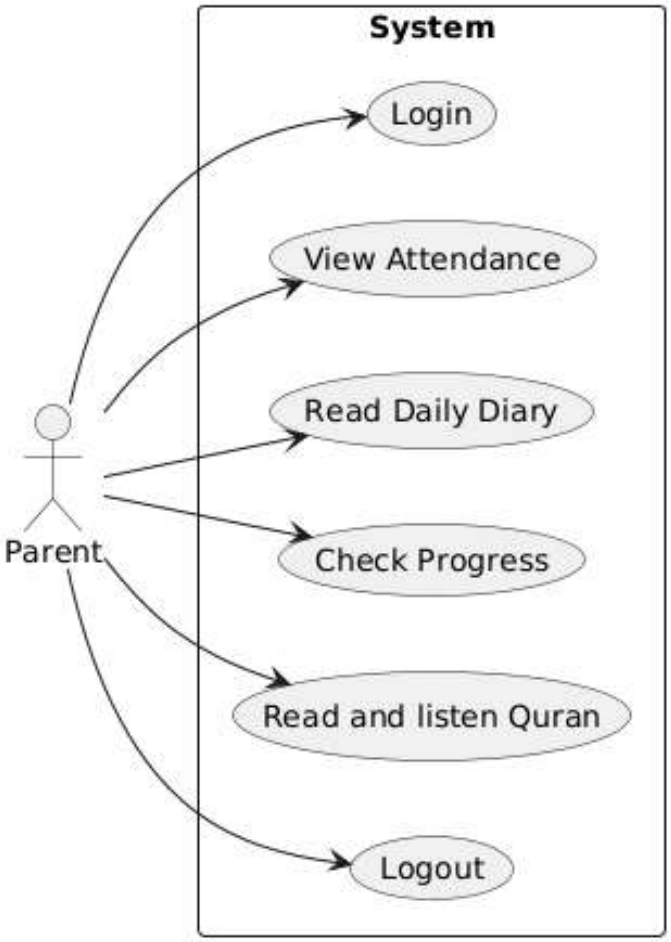


Figure 31 Parent Use Case Diagram

3.5.4.1 Parent Login

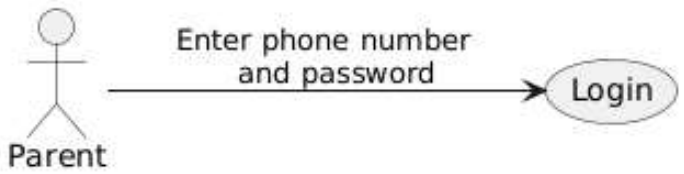


Figure 32 UC4-01

Use Case Description:

Use Case ID	UC4-01	
Use Case Name	Parent Login	
Description	Allows the parent to securely log in to the EMMS system to access their dashboard and manage tasks.	
Primary Actor	Parent	
PreCondition	The parent must be a registered with valid credentials (phone number & password).	
Post-Condition	The parent is authenticated and redirected to the parent dashboard.	
Basic Flow	**Actor Action**	**System Action**
	1. The use case starts when the parent opens the EMMS login page. 2. The parent enters a valid email and password. 3. The system verifies the credentials. 4. If valid, the system logs in the parent and redirects to the dashboard. 5. The session is securely created and maintained.	• The system authenticates the parent and creates a session, granting access to the parent dashboard.
Alternate Flow	**-** If the credentials are invalid, the system shows an error message.	

Table 41 UC4-01

3.5.4.2 View Attendance

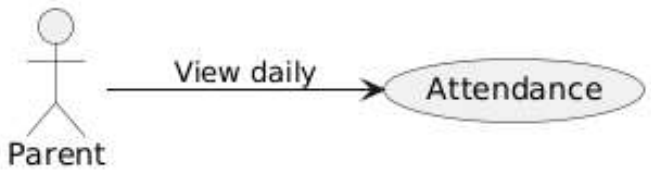


Figure 33 UC4-02

Use Case Description:

Use Case ID	UC4-01	
Use Case Name	View Attendance	
Description	Allows the parent to view the attendance record of their child.	
Primary Actor	Parent	
PreCondition	The parent must be a registered with valid credentials (phone number & password).	
Post-Condition	The parent is shown the attendance record of the respective student.	
Basic Flow	**Actor Action**	**System Action**
	1. Parent navigates to the "Attendance" section. 2. Parent views the attendance data in a readable format.	• The system fetches and displays the attendance data for the parent's child from the database.
Alternate Flow	None	

Table 42 UC4-02

3.5.4.3 Read Daily Diary

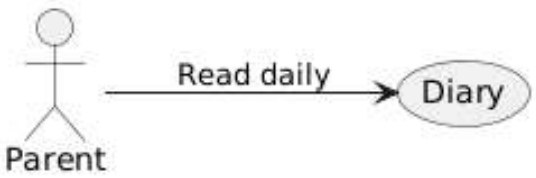


Figure 34 UC4-03

Use Case Description:

Use Case ID	UC4-03	
Use Case Name	View Daily Diary	
Description	Allows the parent to read the daily diary entries updated by the teacher for their child.	
Primary Actor	Parent	
PreCondition	The parent must be a registered with valid credentials (phone number & password).	
Post-Condition	The parent views the child's daily diary updates.	
Basic Flow	**Actor Action**	**System Action**
	1. Parent clicks on the "Daily Diary" section. 3. Parent views the diary updated by teacher.	• The system fetches and displays the daily diary entries related to the parent's child.
Alternate Flow	None	

Table 43 UC4-03

3.5.4.4 Check Progress

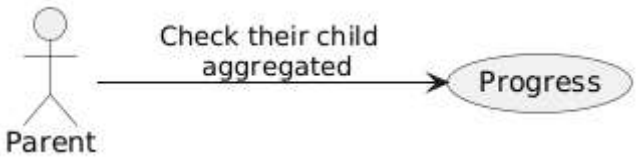


Figure 35 UC4-04

Use Case Description:

Use Case ID	UC4-04	
Use Case Name	Check Progress	
Description	Allows the parent to check the academic and Hifz progress of their child, including sabqi, manzil, and atkan performance.	
Primary Actor	Parent	
PreCondition	The parent must be a registered with valid credentials (phone number & password).	
Post-Condition	The parent views detailed performance statistics of the child.	
Basic Flow	**Actor Action**	**System Action**
	1. Parent navigates to the "Progress" section. 2. Parent views progress details.	• The system displays the latest progress of the child with charts.
Alternate Flow	None	

Table 44 UC4-04

3.5.4.5 Read and listen Quran

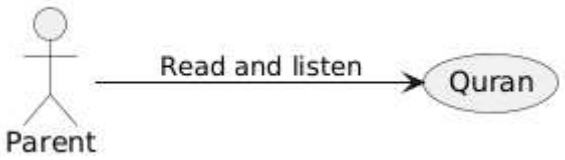


Figure 36 UC4-05

Use Case Description:

Use Case ID	UC4-05	
Use Case Name	Reads and listen Quran	
Description	This use case allows the parent to open and read any Surah and listen to the recitation of Quran.	
Primary Actor	Parent	
PreCondition	The parent must be logged in to the system.	
Post-Condition	The selected Surah is opened and audio is played if requested.	
Basic Flow	**Actor Action**	**System Action**
	1. Navigates to the **Quran Section**. 2. Selects a Surah from the list. 3. Views the verses of the Surah. 4. Clicks on a verse to listen to its audio.	• Fetches Surah list from the Quran API. • Displays the complete Surah in Arabic. • Streams audio for the selected verse using an external API. • Ensures smooth playback with a play/pause button.
Alternate Flow	None	

Table 45 UC4-05

3.5.4.6 Parent Logout



Figure 37 UC4-06

Use Case Description:

Use Case ID	UC4-06	
Use Case Name	Parent Logout	
Description	This use case allows the parent to securely end their session from the EMMS system to protect the platform from unauthorized access.	
Primary Actor	Parent	
PreCondition	Parent must be logged into the EMMS system.	
Post-Condition	The parent is successfully logged out, and the session is terminated securely.	
Basic Flow	**Actor Action**	**System Action**
	1. Parent clicks on a dropdown menu from the navigation bar. 2. Parent clicks the "Logout" button from drop down menu.	• Ends the parent session. • Redirects them to the login screen. • Clears all temporary session-related data.
Alternate Flow	**-** If the session has expired, the system automatically logs out the parent.	

Table 46 UC4-06

3.6 Domain Model Diagram

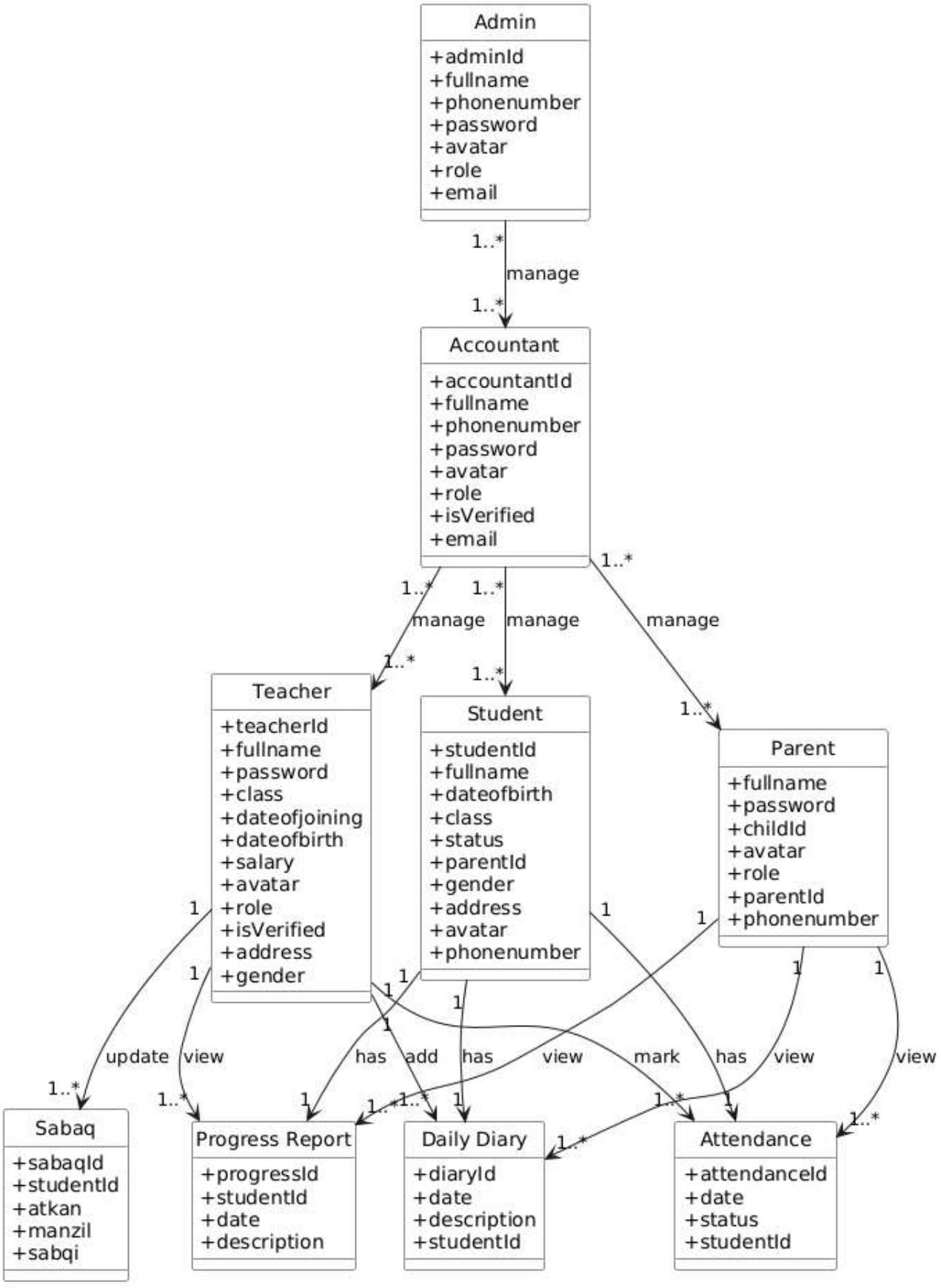


Figure 38 Domain Model Diagram

3.7 Data Flow Diagram

3.7.1 Level 0 Data Flow Diagram

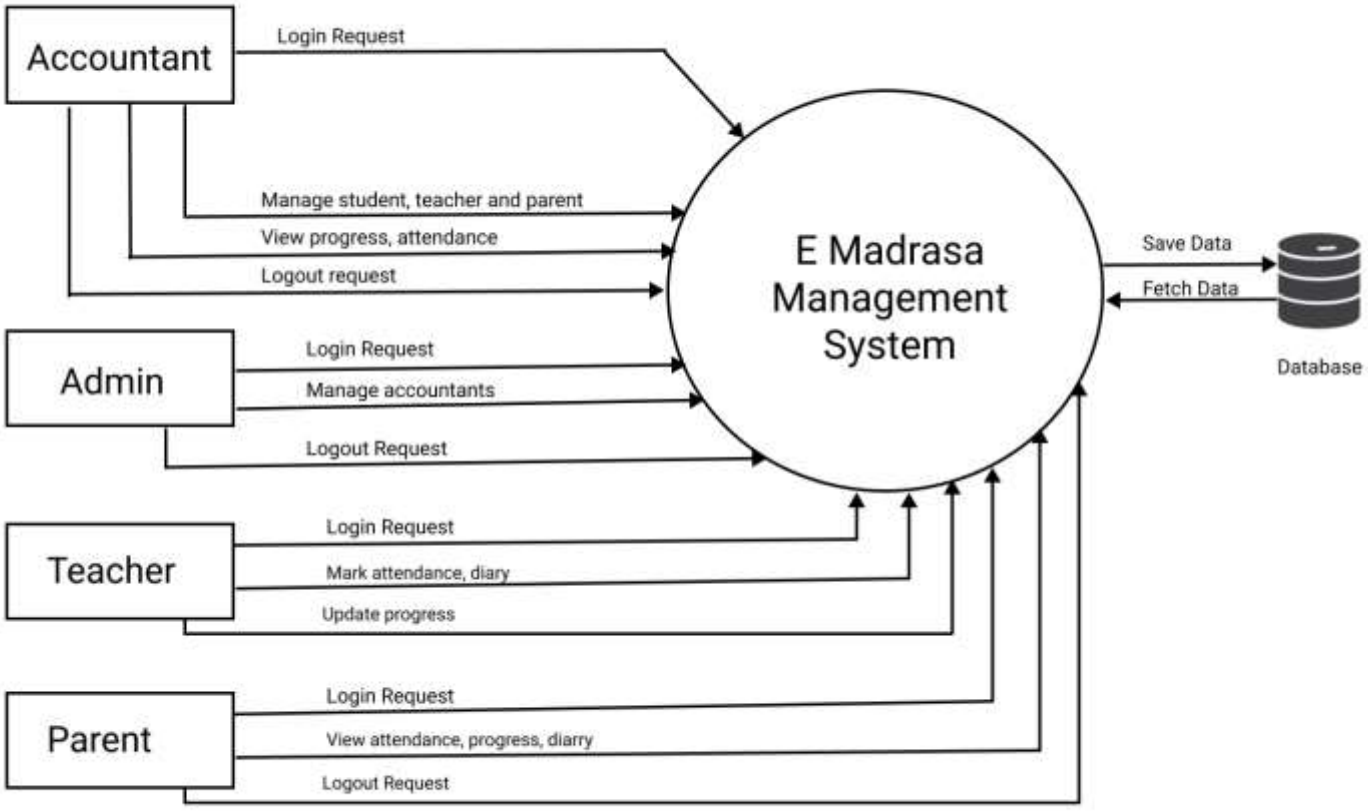


Figure 39 Level 0 Data Flow Diagram

3.7.2 Level 1 Data Flow Diagram

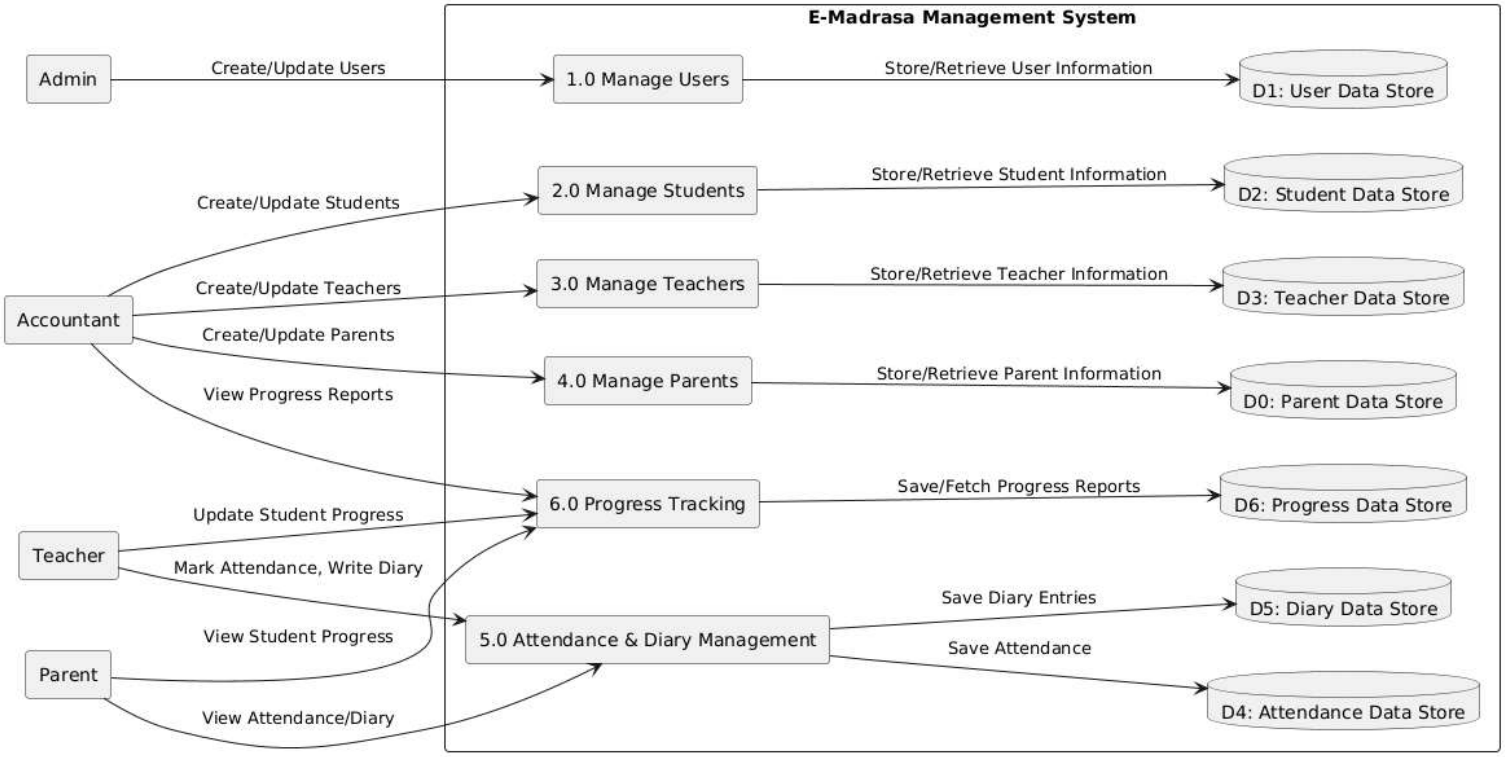


Figure 40 Level 1 Data Flow Diagram

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

An architecture diagram is a simple picture that shows how different parts of a system are connected and work together.

This project uses a web based three layer architecture system.

- The first layer is the **Frontend**, where users will see and use the website.
- The second layer is the **Backend**, where all logic happens.
- The third layer is the **Database**, where all the is safely stored.

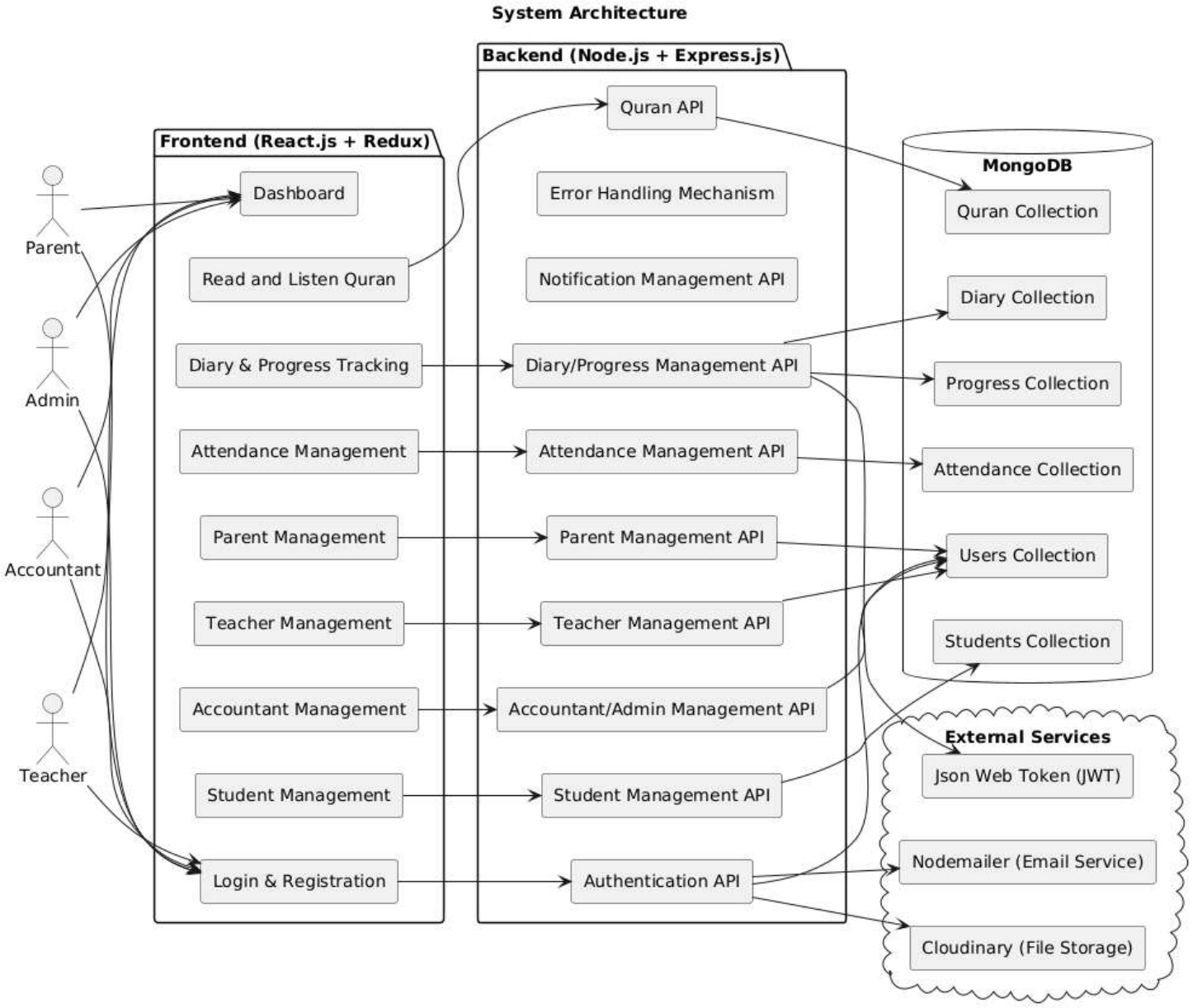


Figure 41 System Architecture Diagram

4.2 Database Design

In mongo db there is only one collection used named as “user” in which all fields are defined, here is the break down of every collection separately.

4.2.1 Admin Collection

Description: This collection stores admin details like their name, contact info, and role in the system.

Field Name	Data Type	Description	Constraints
admin_id	string	Unique ID for admin	primary key (PK)
fullname	string	Full name of admin	required
phonenumber	number	Phone number	unique
password	string	Password for login	required
email	varchar	Email address	unique
avatar	string	Profile picture base 64 encoded	required
roles	enum	Role of admin	required

Table 47 Admin Collection

4.2.2 Accountant Collection

Description: This collection keeps details about accountants, including their profile and login information.

Field Name	Data Type	Description	Constraints
accountant_id	string	Unique ID for accountant	primary key (PK)
fullname	string	Accountant's full name	required
phonenumber	number	Contact number	unique
password	string	Password for login	required
email	string	Email address	unique
avatar	string	Profile picture base 64 encoded	required
isVerified	bool	Account verification status	default false
roles	enum	Role of accountant	required

Table 48 Accountant Collection

4.2.3 Teacher Collection

Description: This collection saves teacher details like personal info, salary, and assigned class.

Field Name	Data Type	Description	Constraints
teacher_id	string	Unique ID for teacher	primary key (PK)
fullname	string	Full name of teacher	required
phonenumber	number	Phone number	unique
password	string	Password for login	required
email	varchar	Email address	unique
avatar	string	Profile picture base 64 encoded	required
isVerified	bool	Account verification status	default false

address	string	Address of teacher	required
salary	number	Monthly salary	required
class	enum	Assigned class	required
dateofjoining	date	Joining date	required
roles	enum	Role of admin	enum

Table 49 Teacher Collection

4.2.4 Parent Collection

Description: This collection manages parent profiles and links them to their child.

Field Name	Data Type	Description	Constraints
parent_id	string	Unique ID for parent	primary key (PK)
fullname	string	Full name of parent	required
phonenumber	number	Phone number	unique
password	string	Password for login	required
rollnumber	string	Children roll number	required
avatar	string	Profile picture base 64 encoded	required

Table 50 Parent Collection

4.2.5 Student Collection

Description: This collection stores all student-related information including registration, class, and parent link.

Field Name	Data Type	Description	Constraints
student_id	string	Unique ID for student	primary key (PK)
fullname	string	Full name of student	required
phonenumber	number	Phone number	unique
password	string	Password for login	required
rollnumber	string	Roll number for parent	required
registrationnumber	number	Registration number	unique
dateofadmission	date	Admission date	required
dateofbirth	date	Date of birth	required
gender	enum	Gender of student	required
status	enum	completed, fired, active	required
class	enum	Awal, Sani, Demo	required
avatar	string	base 64 encoded picture	required

Table 51 Student Collection

4.2.6 Diary Collection

Description: This table is used for writing daily diary notes for students.

Field Name	Data Type	Description	Constraints
diary_id	string	Unique ID for diary entry	primary key (PK)
date	date	Entry Date	required
description	string	Description or notes	required
student_id	string	Linked student ID	foreign key (FK)

Table 52 Diary Collection

4.2.7 Attendance Collection

Description: This table records the daily attendance status of students.

Field Name	Data Type	Description	Constraints
attendance_id	string	Unique ID for attendance	primary key (PK)
date	date	Entry Date	required
status	enum	Present/Absent	required
student_id	string	Linked student ID	foreign key (FK)

Table 53 Attendance Collection

4.2.8 Progress Collection

Description: This table learning progress of each student.

Field Name	Data Type	Description	Constraints
progress_id	string	Unique ID for attendance	primary key (PK)
date	date	Entry Date	required
atkan	string	Atkan progress description	required
sabaq	string	Sabaq progress description	required
mistakes	string	Mistakes of student	required
student_id	string	Linked student ID	foreign key (FK)

Table 54 Progress Collection

4.3 ER Diagrams

4.3.1 Admin Entity

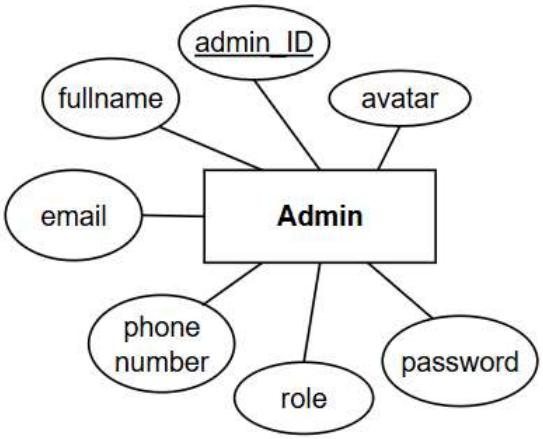


Figure 42 Admin Entity

4.3.2 Accountant Entity

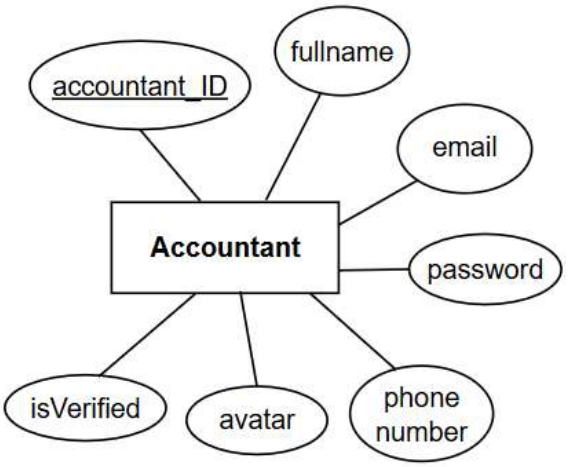


Figure 43 Accountant Entity

4.3.3 Diary Entity

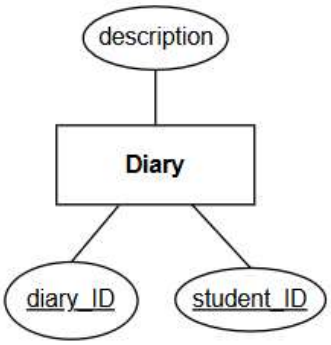


Figure 44 Diary Entity

4.3.4 Student Entity

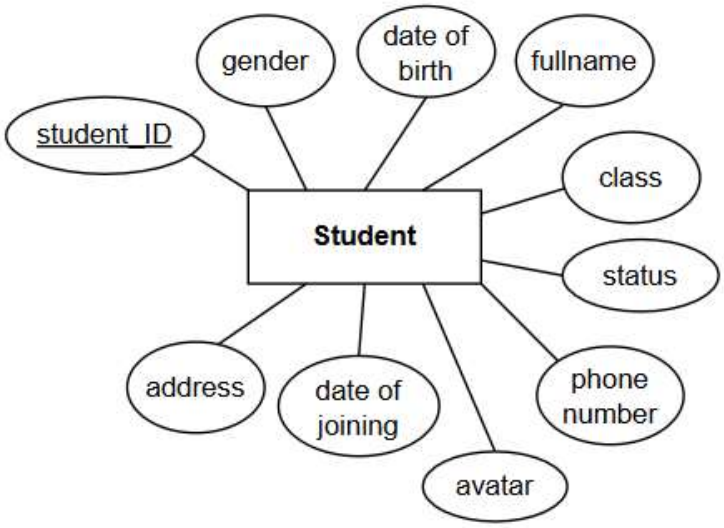


Figure 45 Student Entity

4.3.5 Teacher Entity

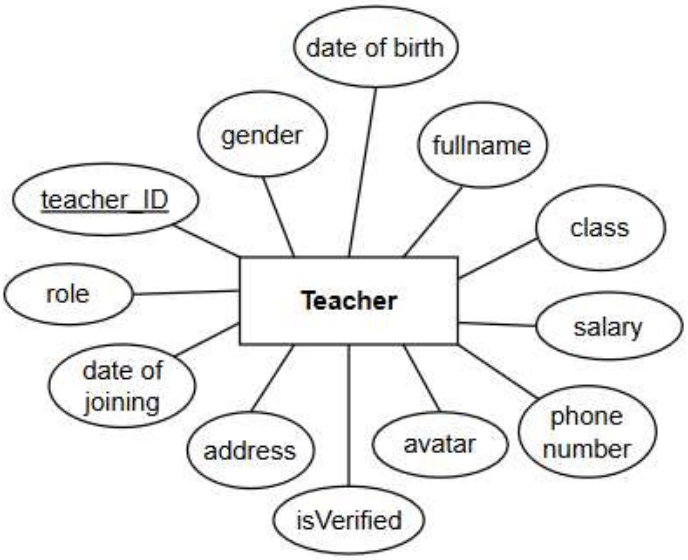


Figure 46 Teacher Entity

4.3.6 Attendance Entity

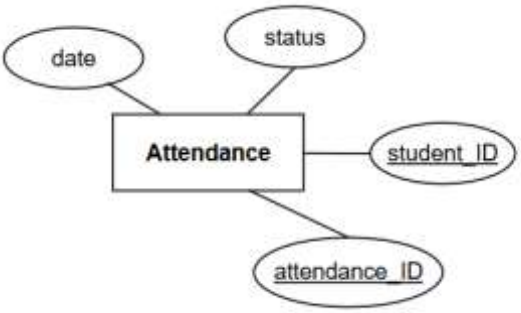


Figure 47 Attendance Entity

4.3.7 Parent Entity

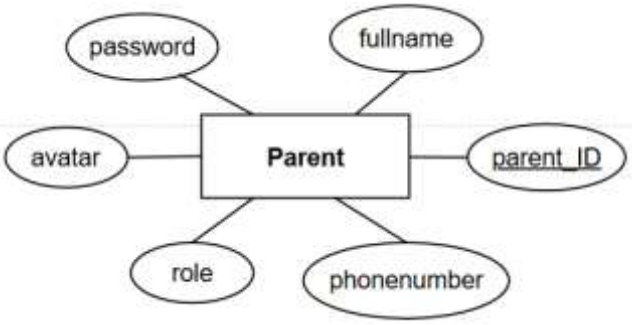


Figure 48 Parent Entity

4.3.8 Progress Entity

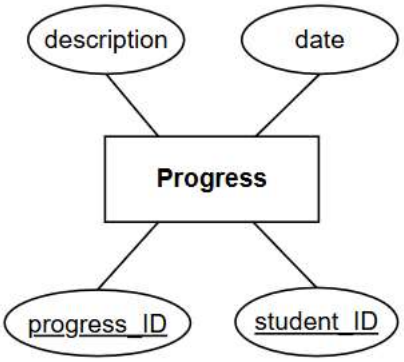


Figure 49 Progress Entity

4.3.9 Admin Accountant Relationship

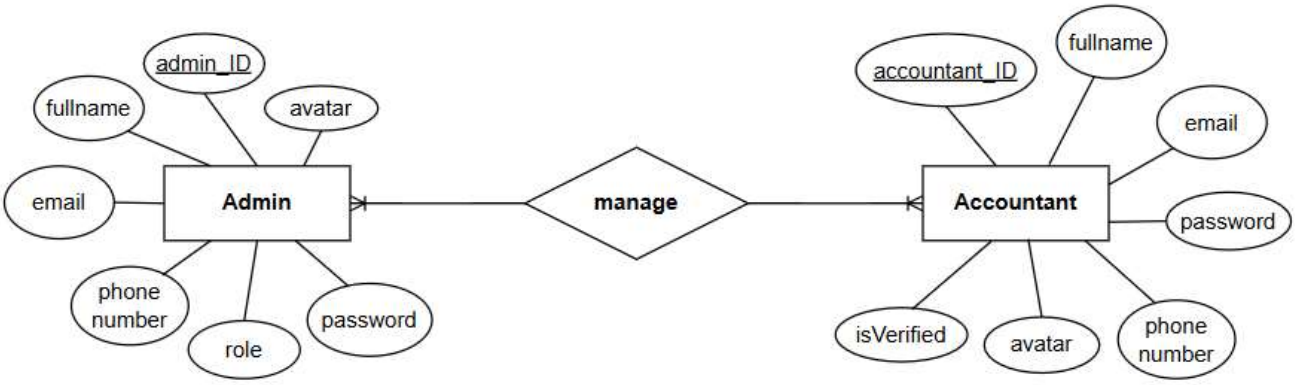


Figure 50 Admin Accountant Relationship

4.3.10 Accountant Student Relationship

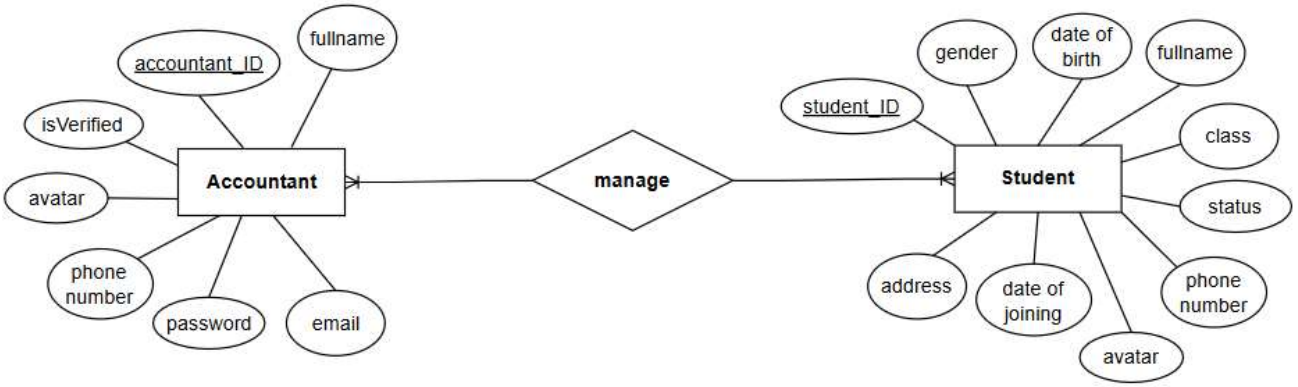


Figure 51 Accountant Student Relationship

4.3.11 Accountant Teacher Relationship

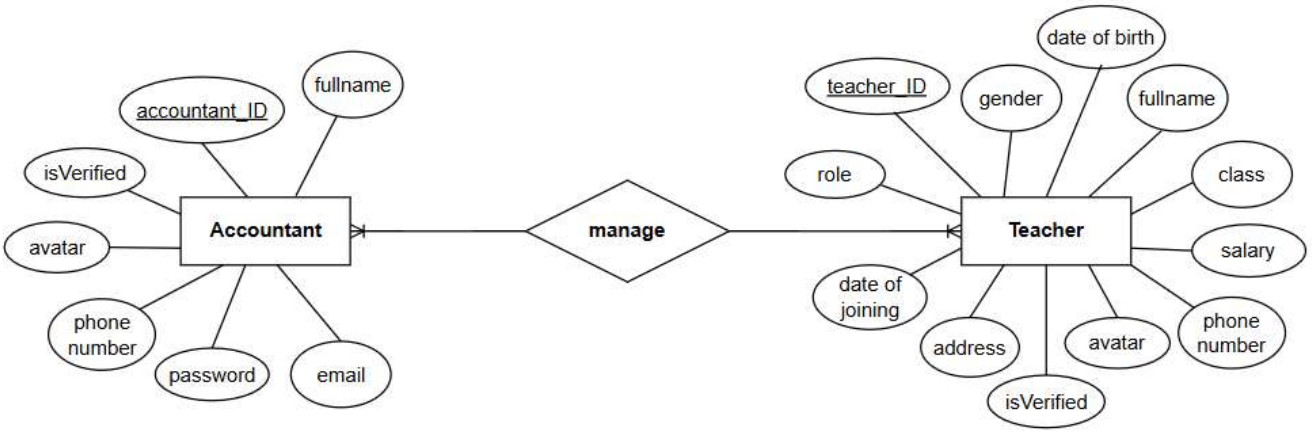


Figure 52 Accountant Teacher Relationship

4.3.12 Accountant Parent Relationship

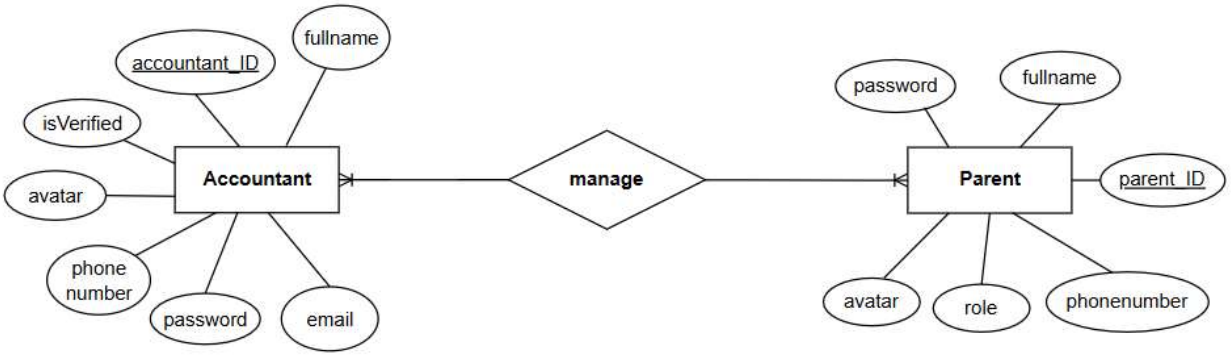


Figure 53 Accountant Parent Relationship

4.3.13 Accountant Progress Relationship

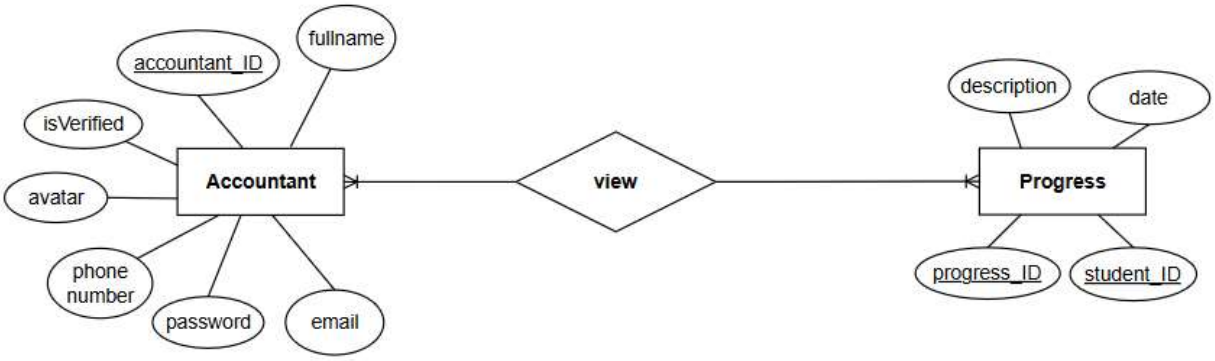


Figure 54 Accountant Progress Relationship

4.3.14 Teacher Diary Relationship

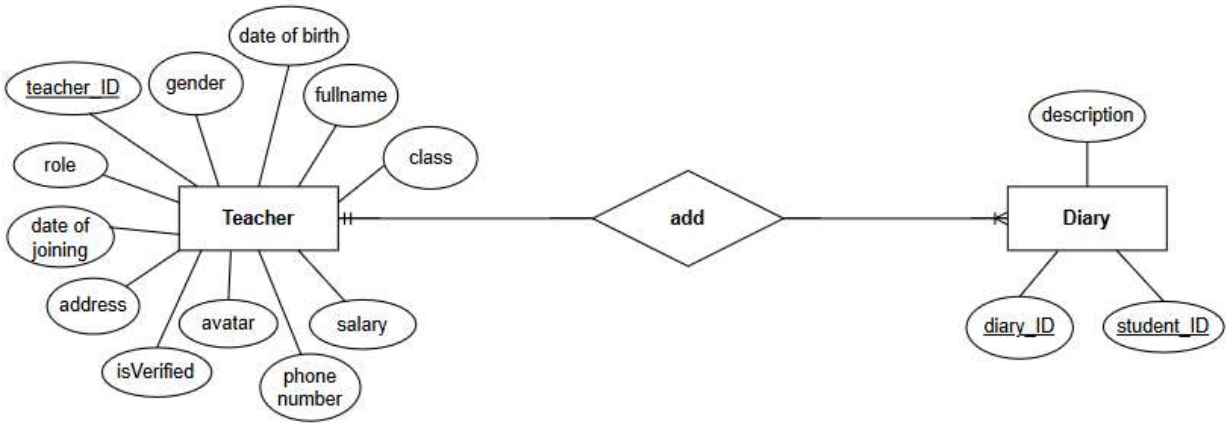


Figure 55 Teacher Diary Relationship

4.3.15 Teacher Attendance Relationship:

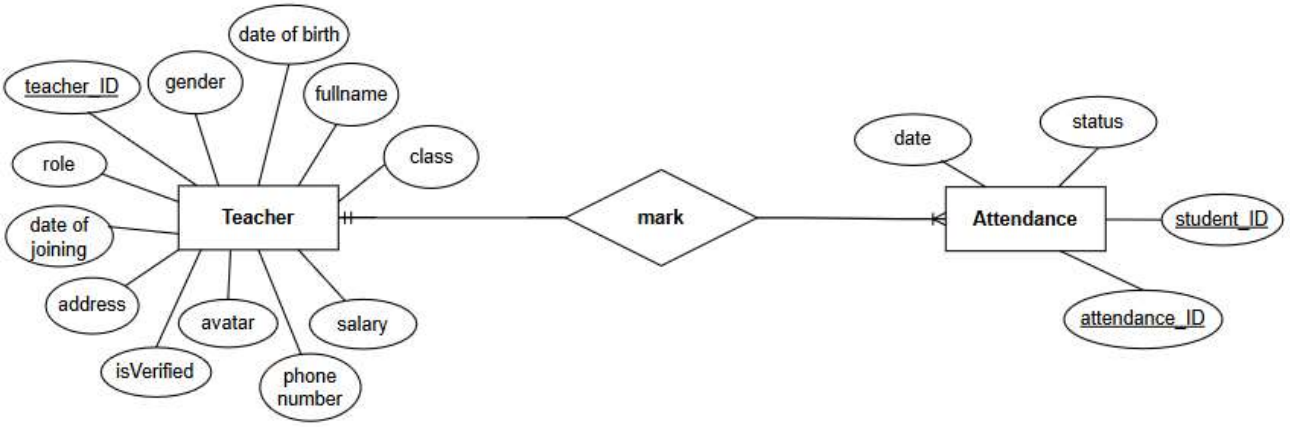


Figure 56 Teacher Attendance Relationship

4.3.16 Teacher Progress Relationship

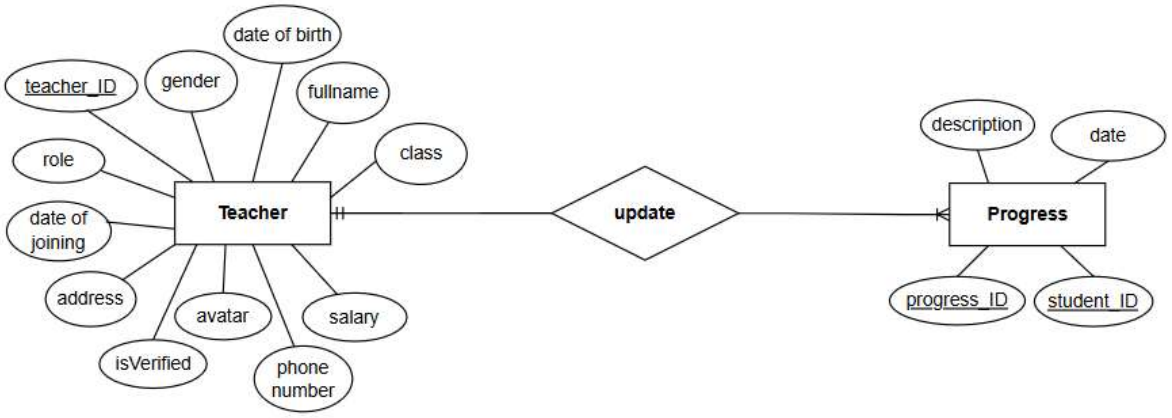


Figure 57 Teacher Progress Relationship

4.3.17 Teacher Progress Relationship

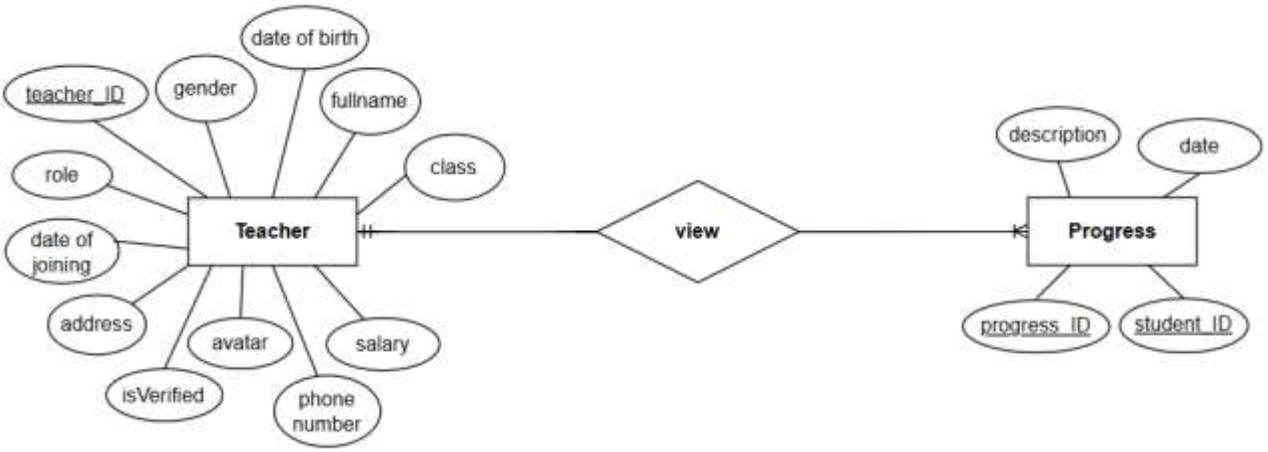


Figure 58 Teacher view Progress Relationship

4.3.18 Parent Diary Relationship

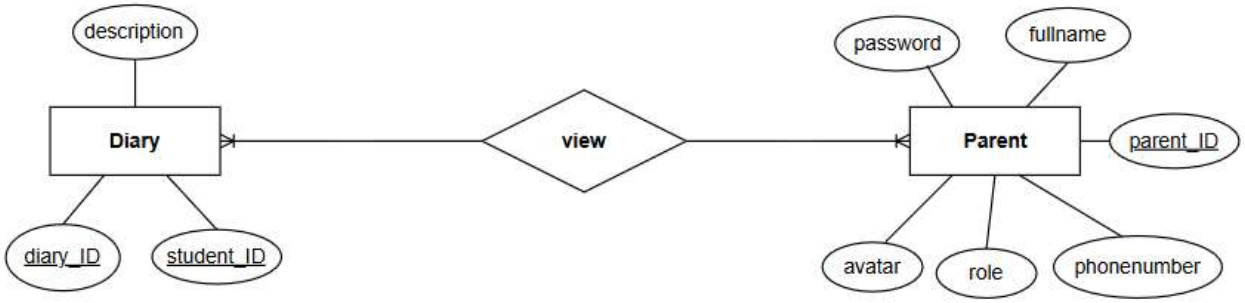


Figure 59 Parent Diary Relationship

4.3.19 Parent Attendance Relationship

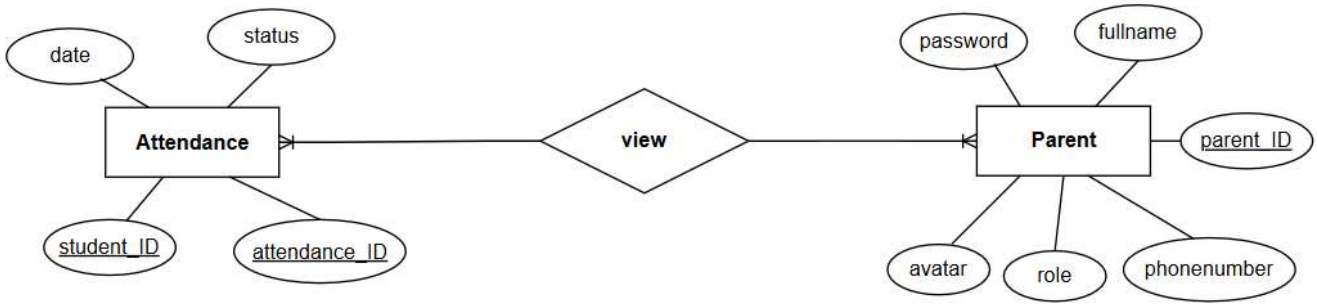


Figure 60 Parent Attendance Relationship

4.3.20 Parent Progress Relationship

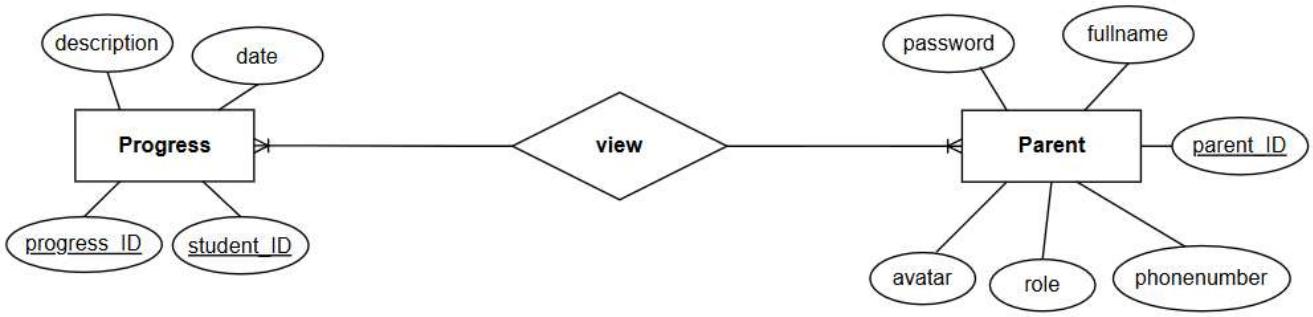


Figure 61 Parent Progress Relationship

4.3.21 Student Diary Relationship

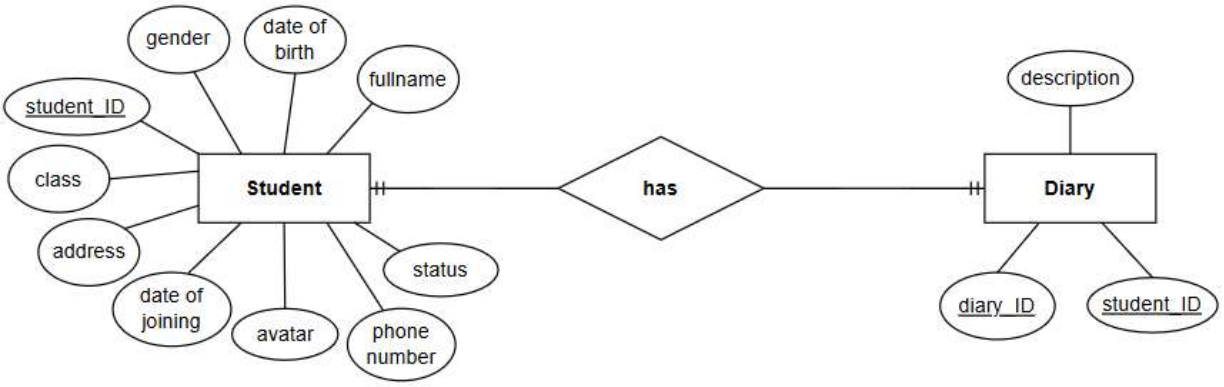


Figure 62 Student Diary Relationship

4.3.22 Student Attendance Relationship

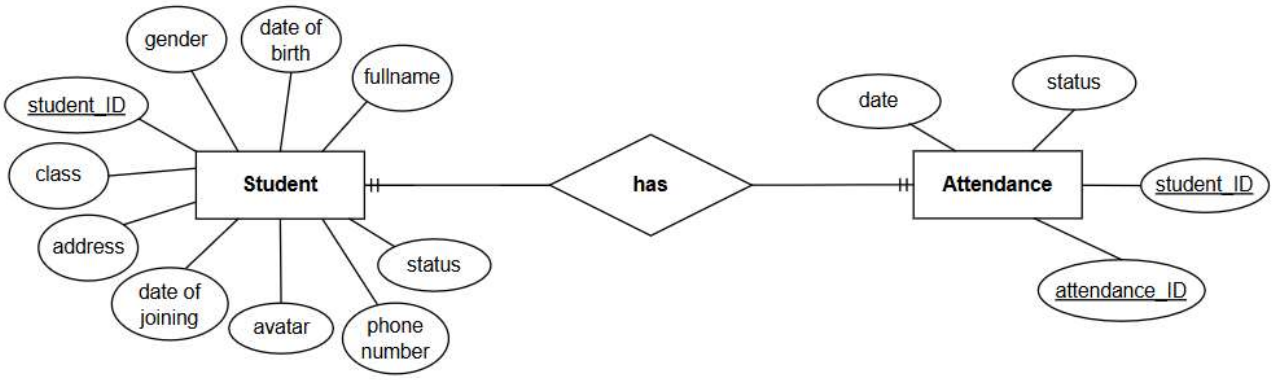


Figure 63 Student Attendance Relationship

4.3.23 Student Progress Relationship

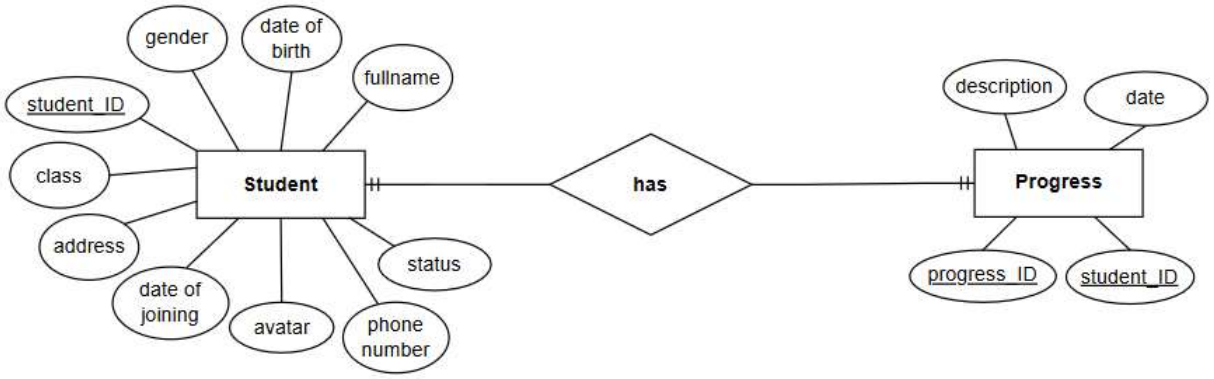


Figure 64 Student Progress Relationship

4.3.24 Complete System ER Diagram

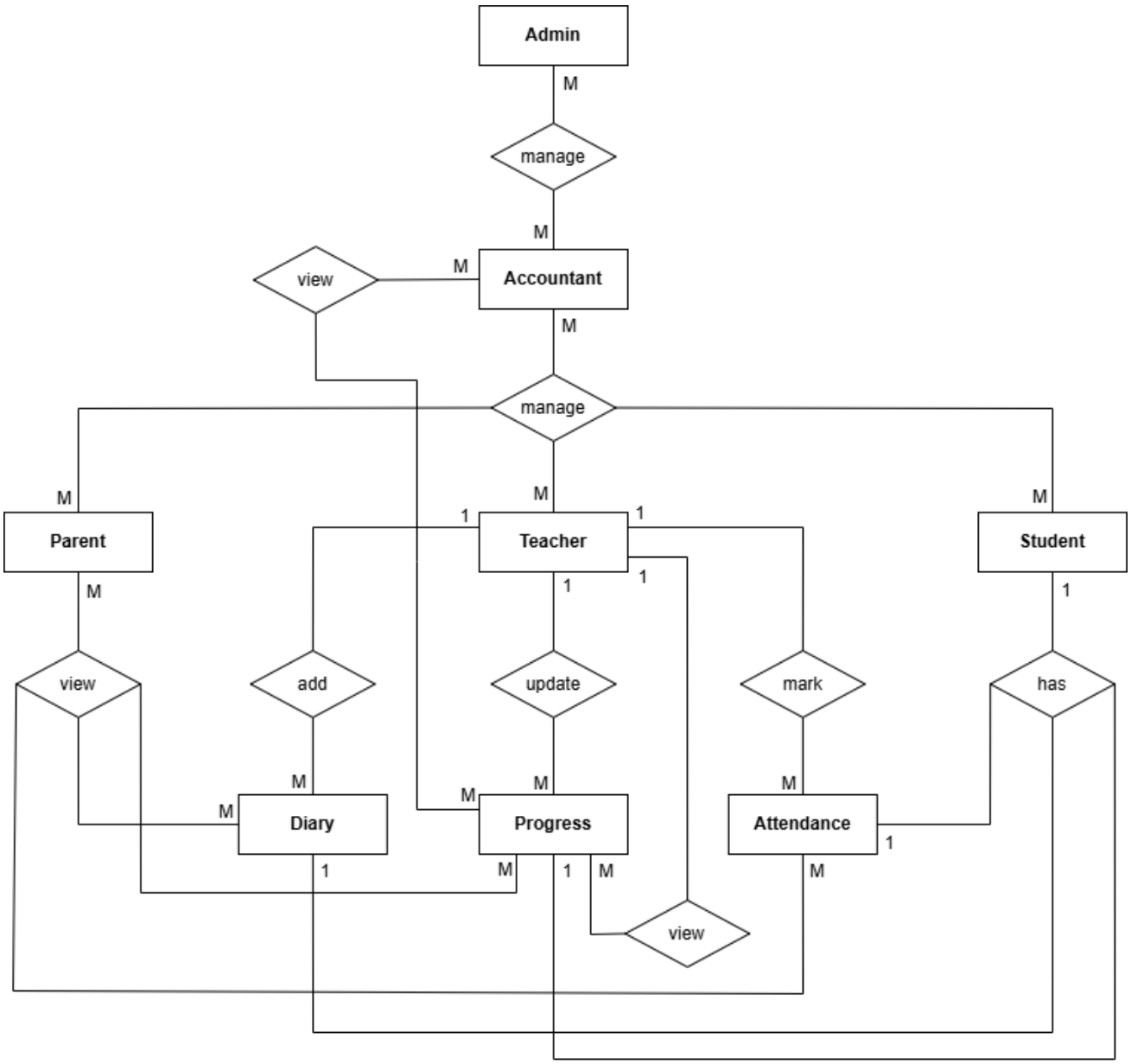


Figure 65 Complete System ER Diagram

4.4 Interface Design

4.4.1 Welcome Page Design

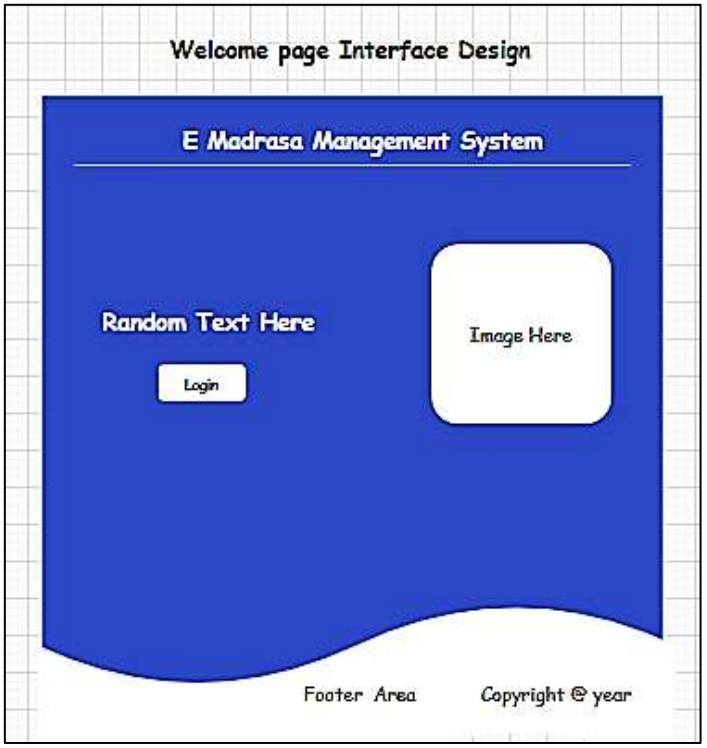


Figure 66 Welcome Page Interface Design

4.4.2 Admin Dashboard Interface Design

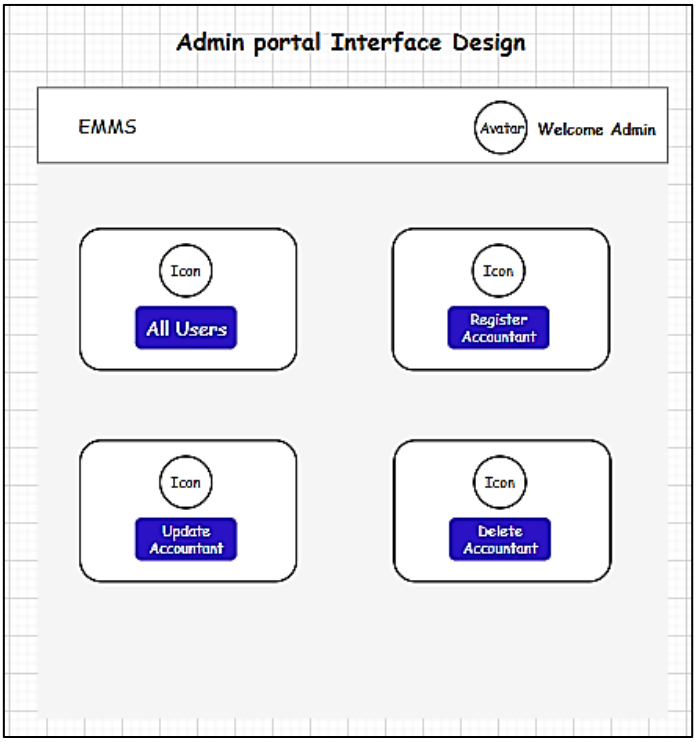


Figure 67 Admin Dashboard Interface Design

4.4.3 Accountant Dashboard Interface Design

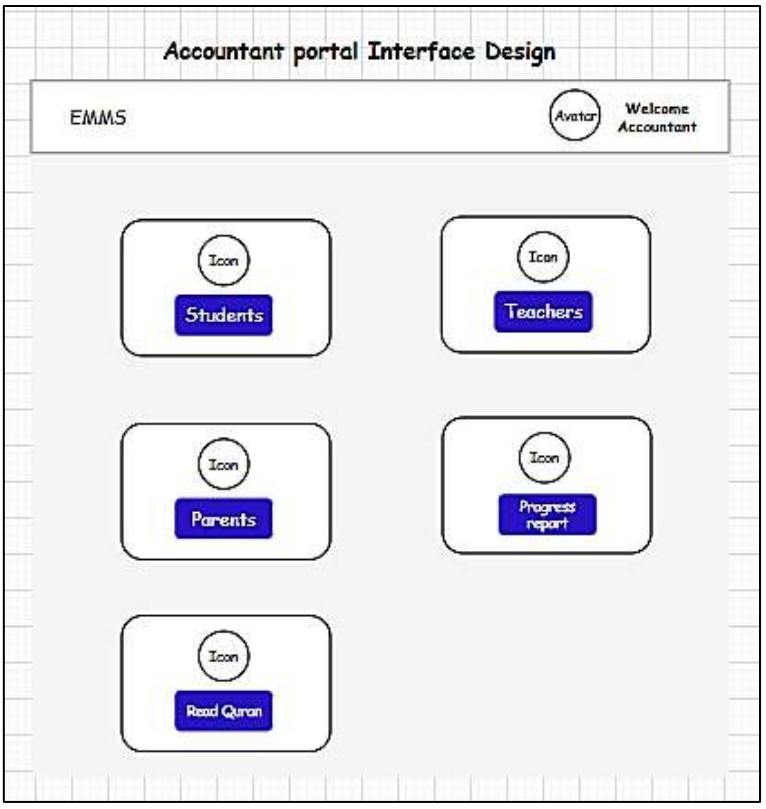


Figure 68 Accountant Dashboard Interface Design

4.4.4 Teacher Dashboard Interface Design

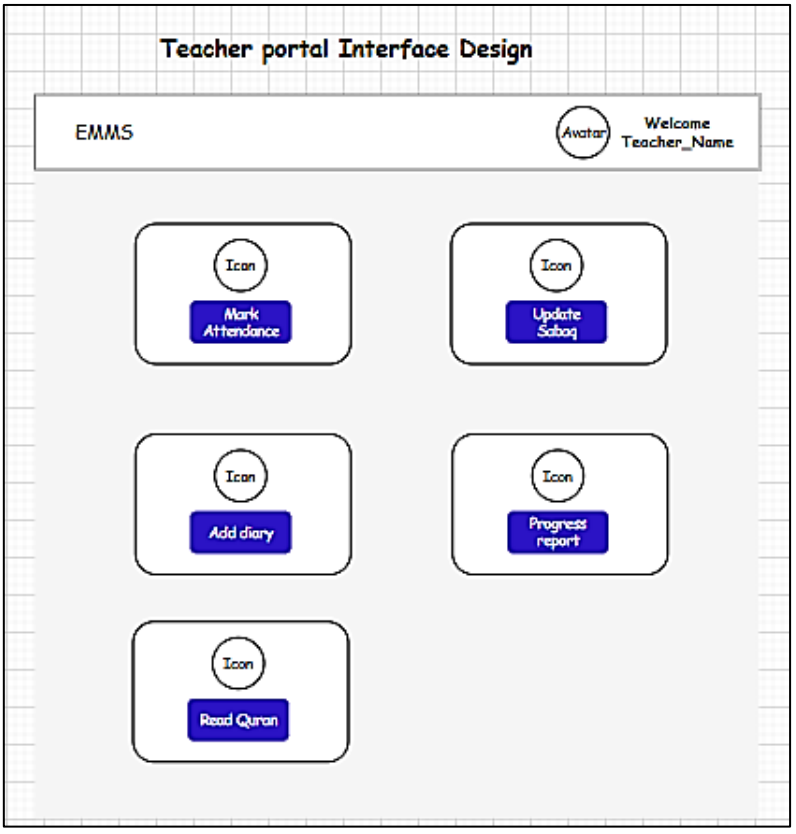


Figure 69 Teacher Dashboard Interface Design

4.4.5 Parent Interface Design

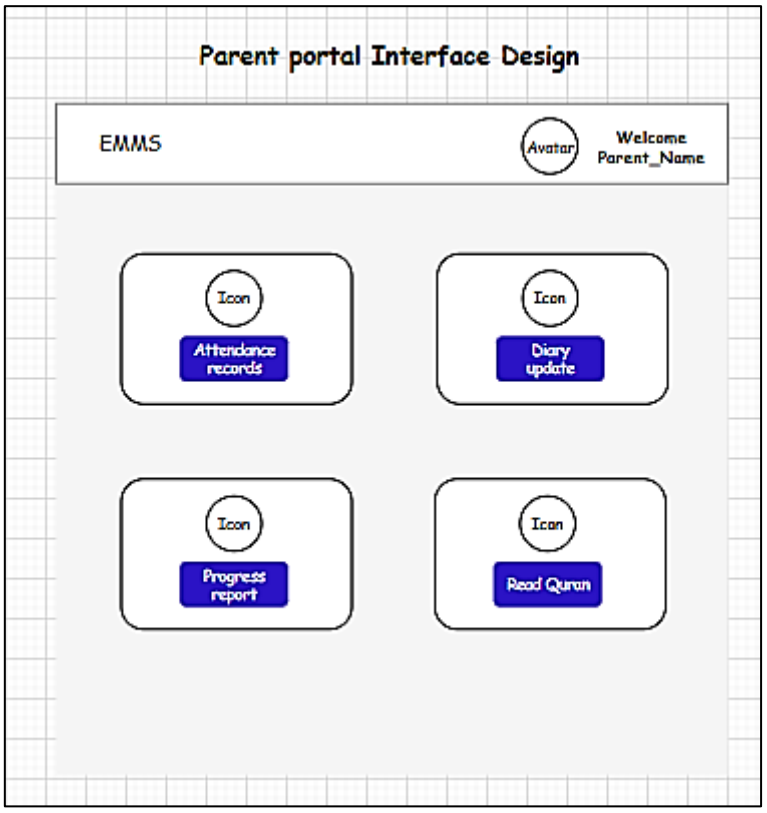


Figure 70 Parent Dashboard Interface Design

4.5 Component Design

4.5.1 Login Component

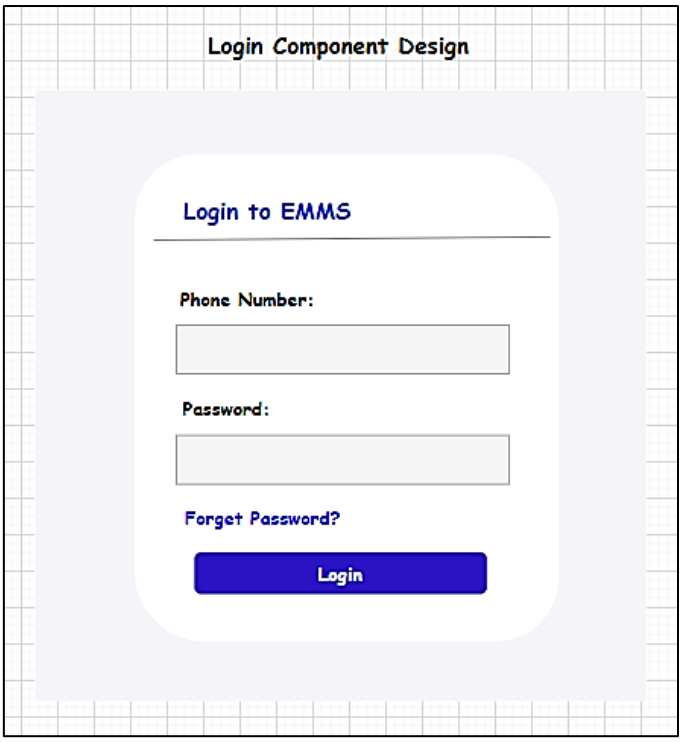


Figure 71 Login Component Design

4.5.2 Forget Password Component



Figure 72 Forget Password Component Design

4.5.3 Reset Password Component

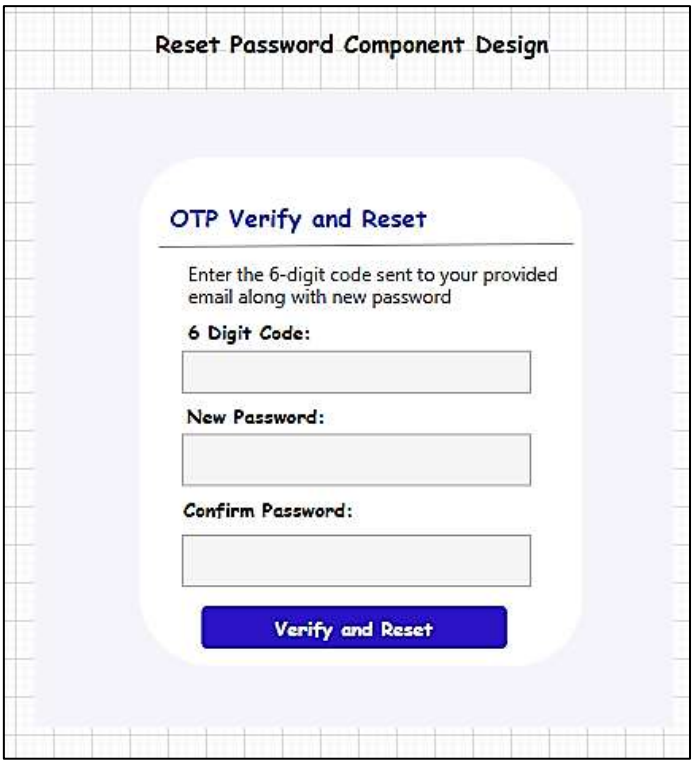


Figure 73 Reset Password Component Design

4.5.4 Register Accountant Component

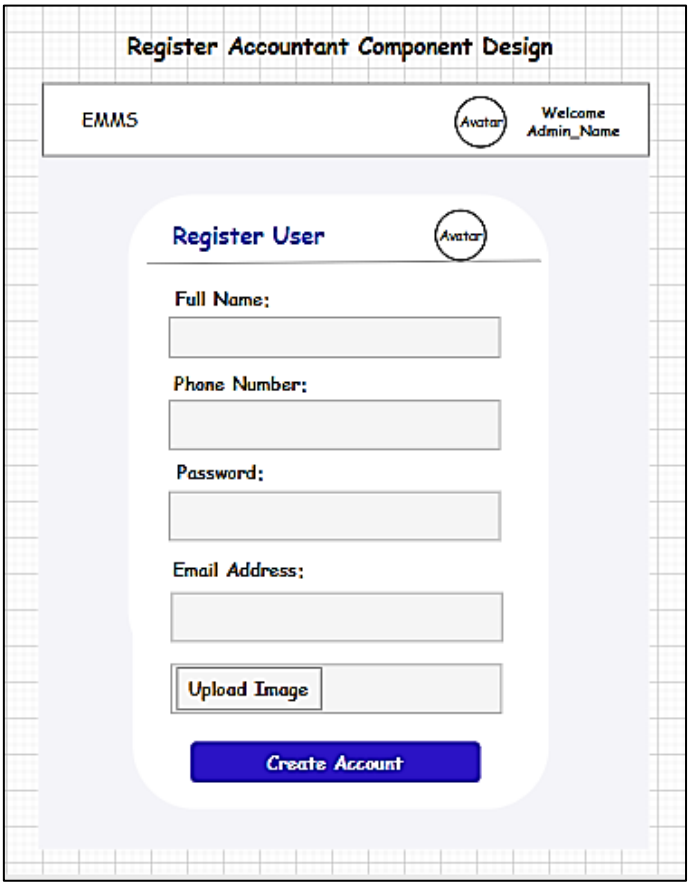


Figure 74 Register Accountant Component Design

4.5.5 Update Accountant Component

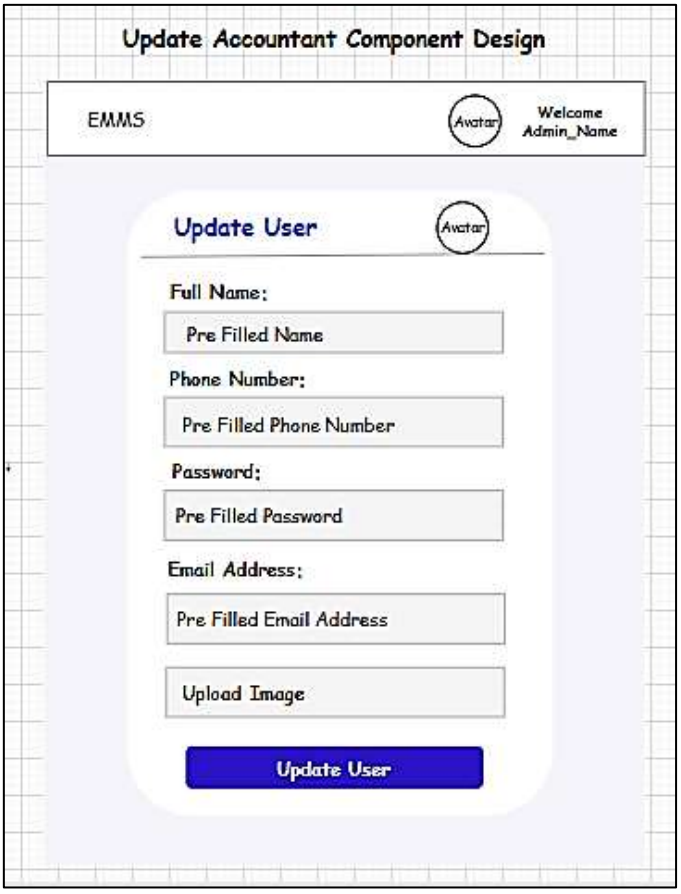


Figure 75 Update Accountant Component Design

4.5.6 Delete Accountant Component

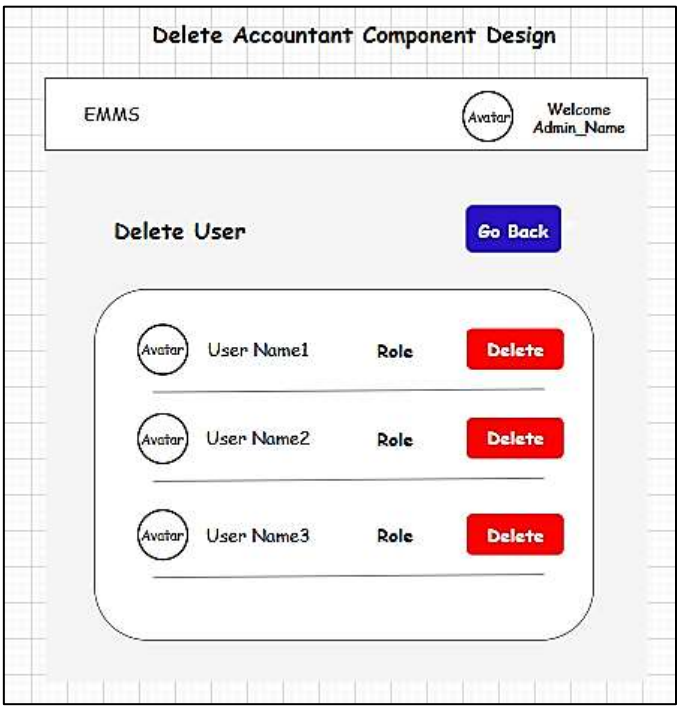


Figure 76 Delete Accountant Component Design

4.5.7 All Users Component

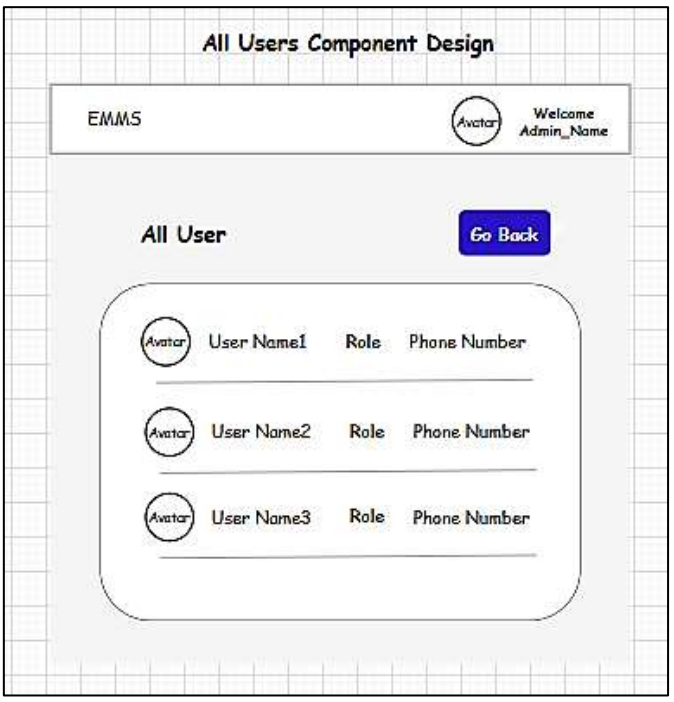


Figure 77 All Users Component Design

4.5.8 Add Student Component

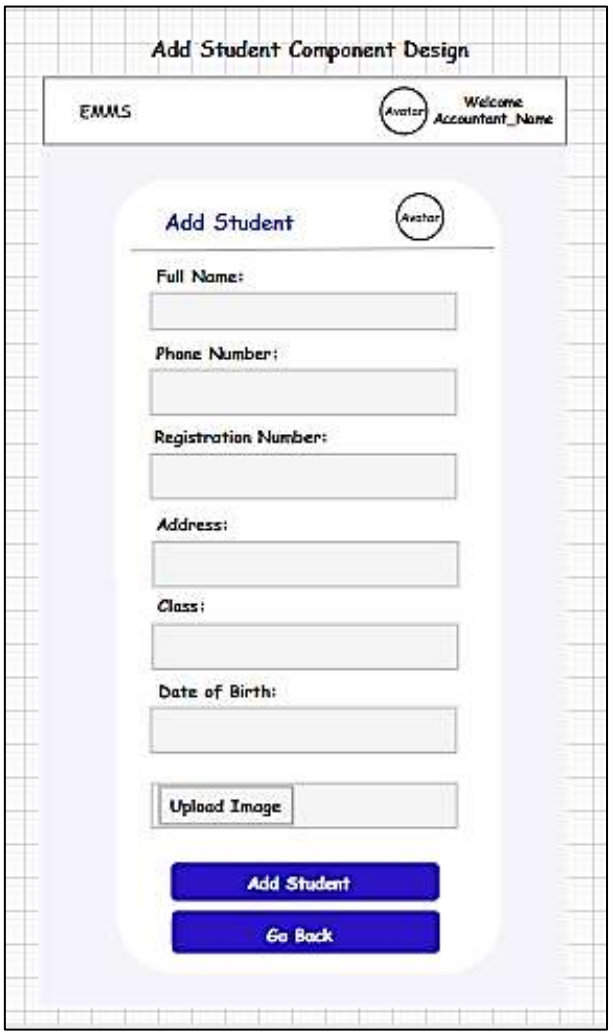


Figure 78 Add Student Component Design

4.5.9 Update Student Component

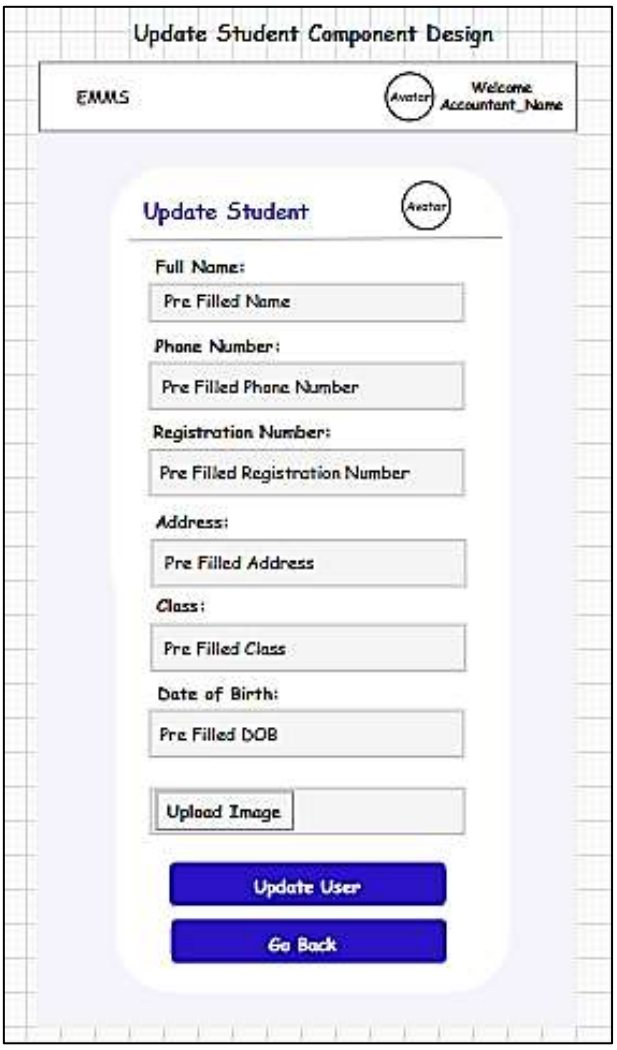


Figure 79 Update Student Component Design

4.5.10 Delete Student Component

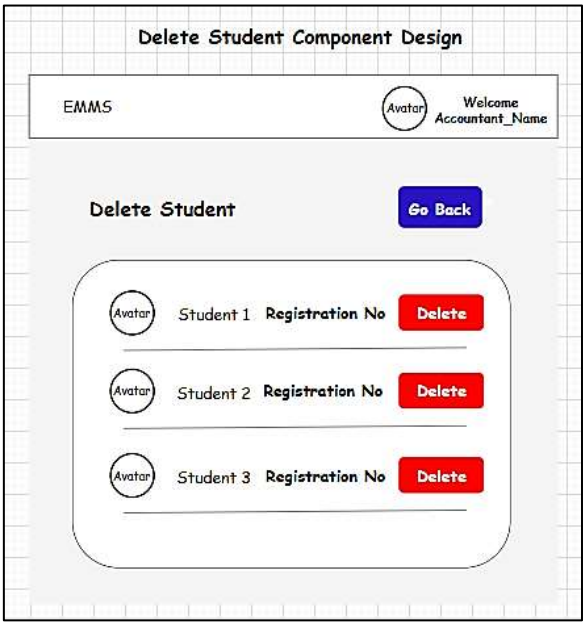


Figure 80 Delete Student Component Design

4.5.11 All Students Component

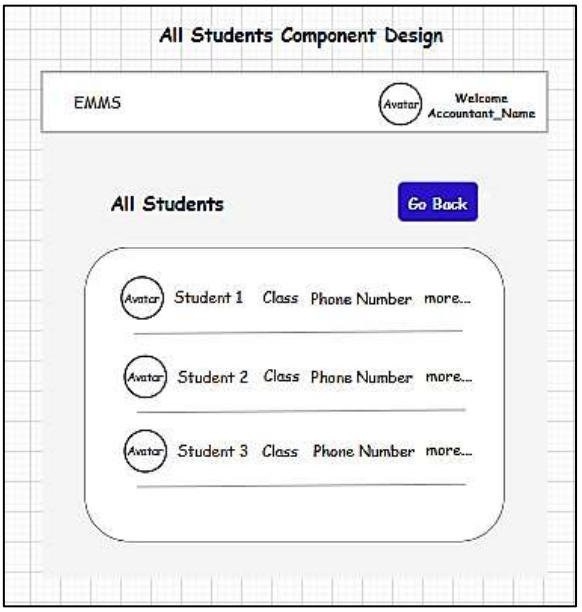


Figure 81 All Students Component Design

4.5.12 Create Teacher Component

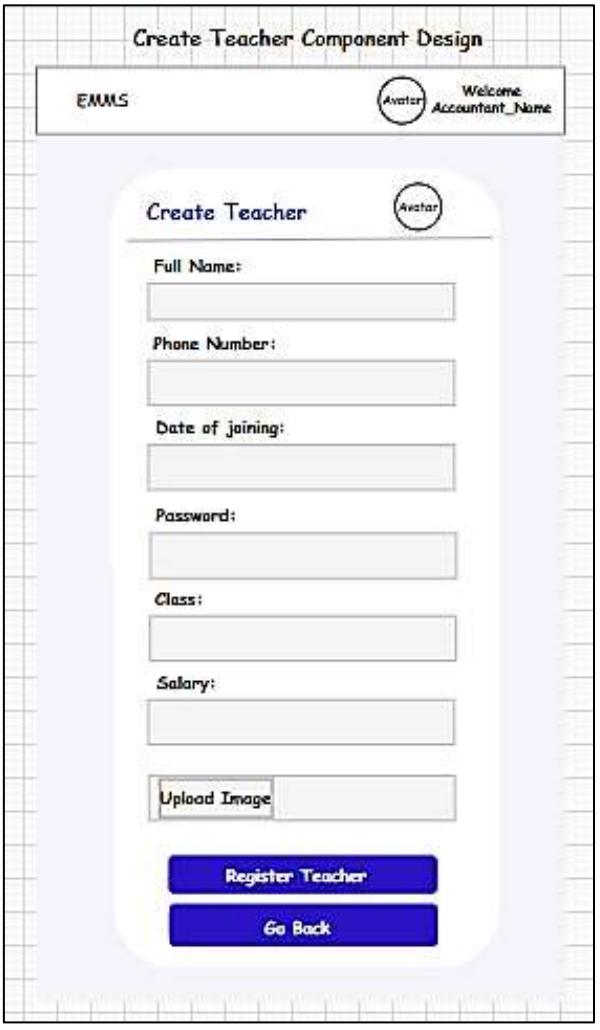


Figure 82 Create Teacher Component Design

4.5.13 Update Teacher Component



Figure 83 Update Teacher Component Design

4.5.14 Delete Teacher Component

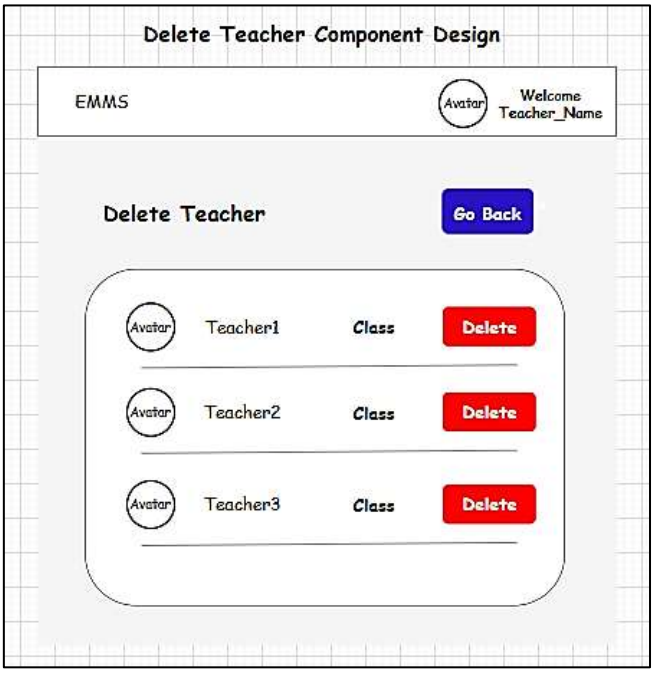


Figure 84 Delete Teacher Component Design

4.5.15 All Teacher Component

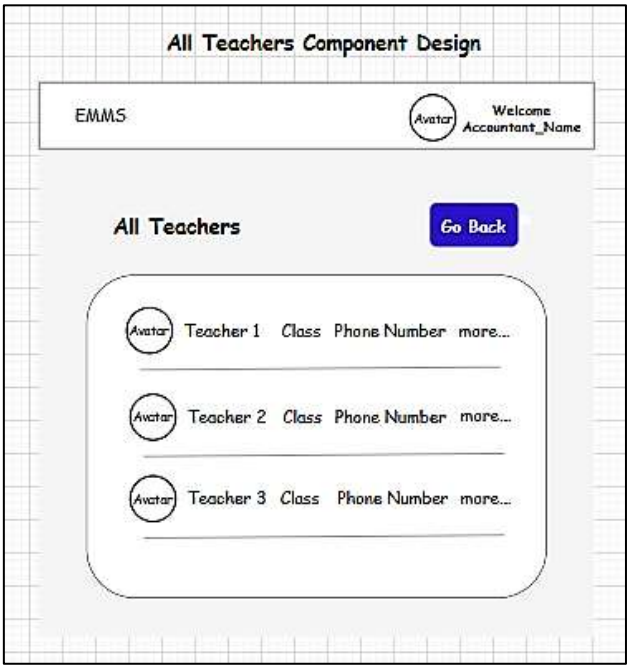


Figure 85 All Teacher Component Design

4.5.16 Create Parent Component

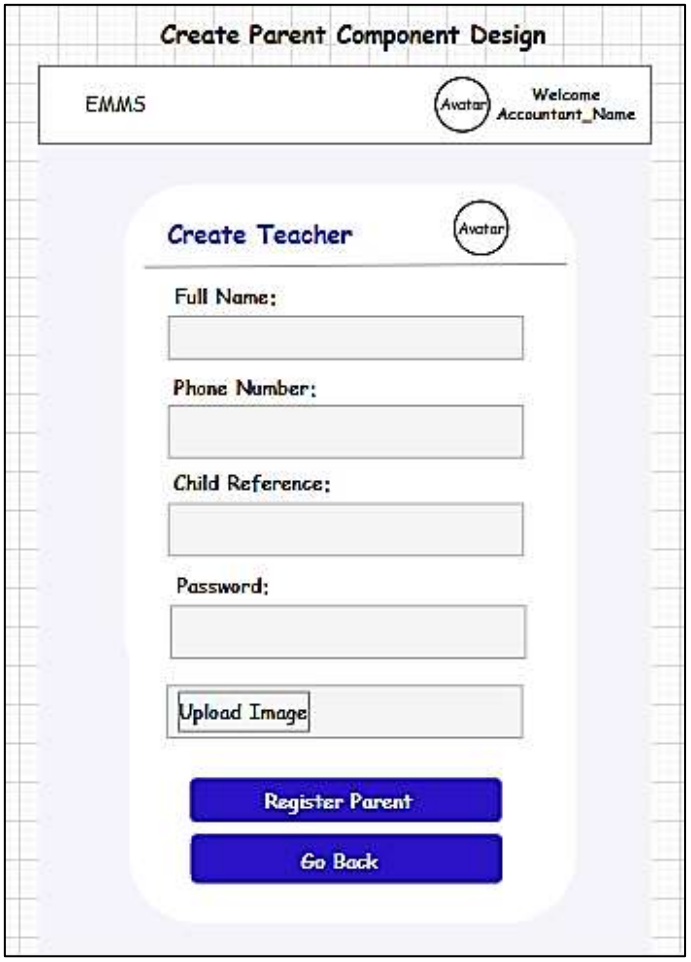


Figure 86 Create Parent Component Design

4.5.17 Update Parent Component



Figure 87 Update Parent Component Design

4.5.18 Delete Parent Component

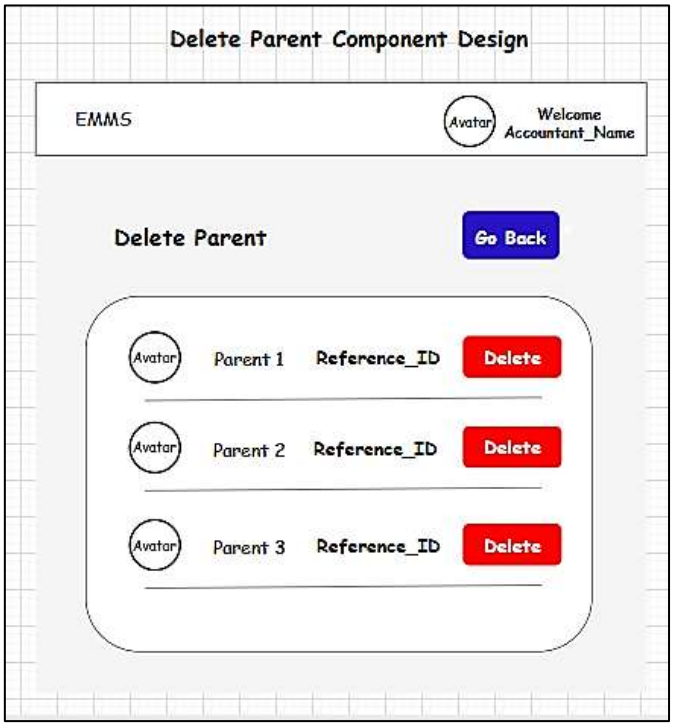


Figure 88 Delete Parent Component Design

4.5.19 All Parents Component

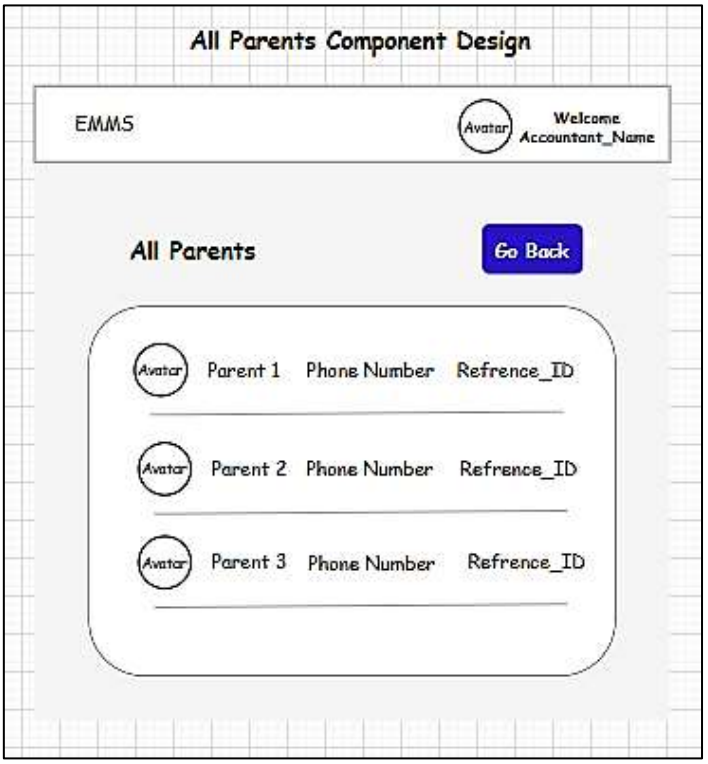


Figure 89 All Parents Component Design

4.5.20 Attendance Component

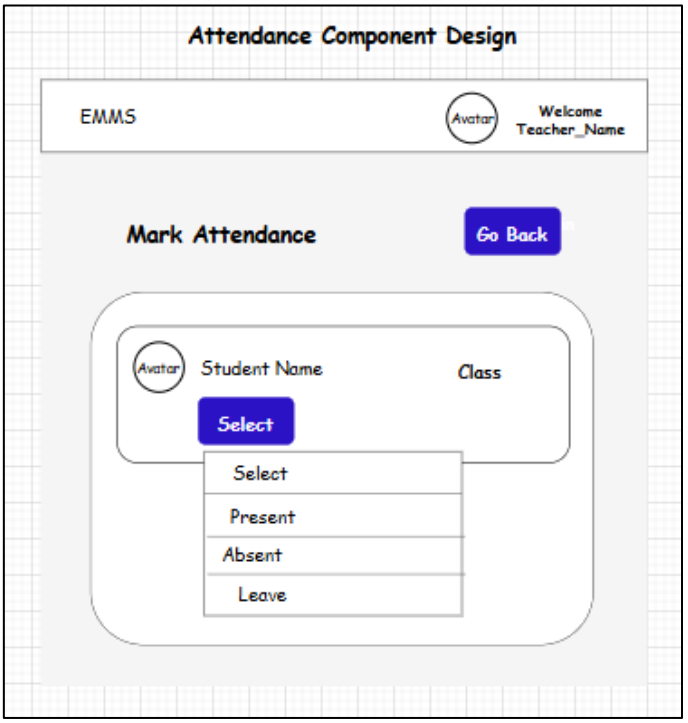


Figure 90 Attendance Component Design

4.5.21 Update Sabaq Component

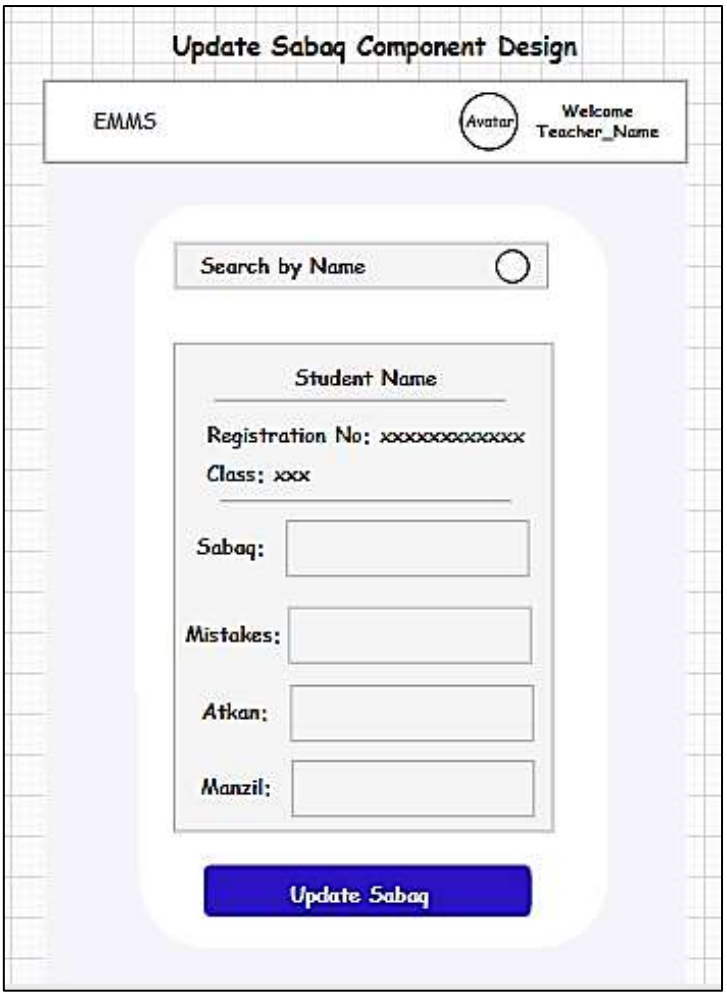


Figure 91 Update Sabaq Component Design

4.5.22 Add Diary Component

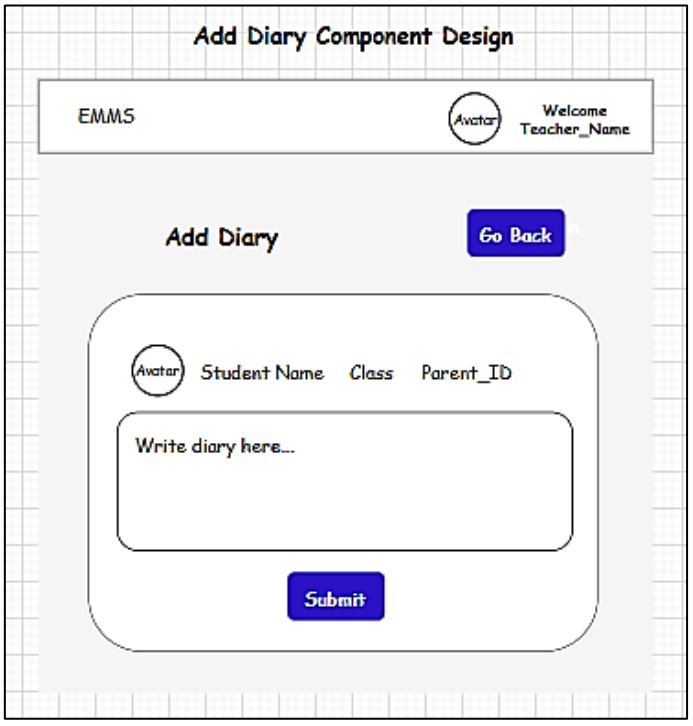


Figure 92 Add Diary Component Design

4.5.23 Progress Report Component

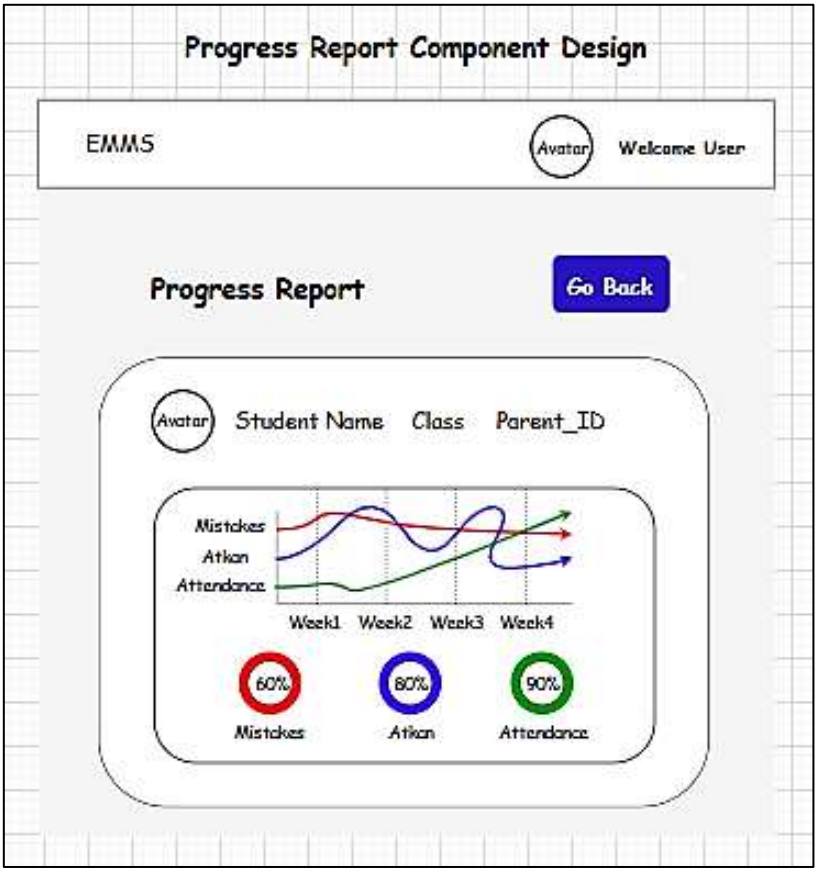


Figure 93 Progress Report Component Design

4.6 Class Diagram

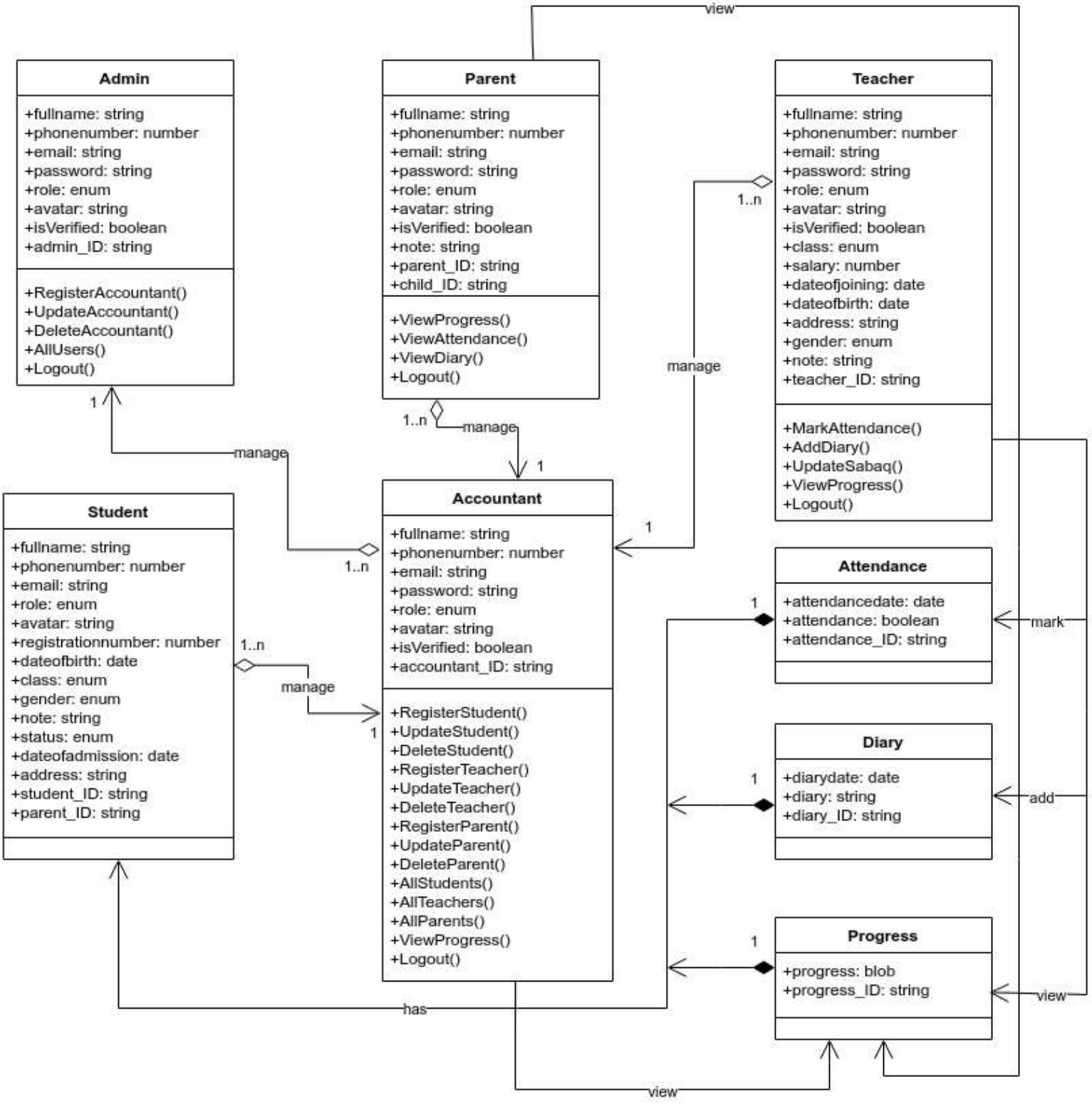


Figure 94 Class Diagram

4.7 Object Diagram

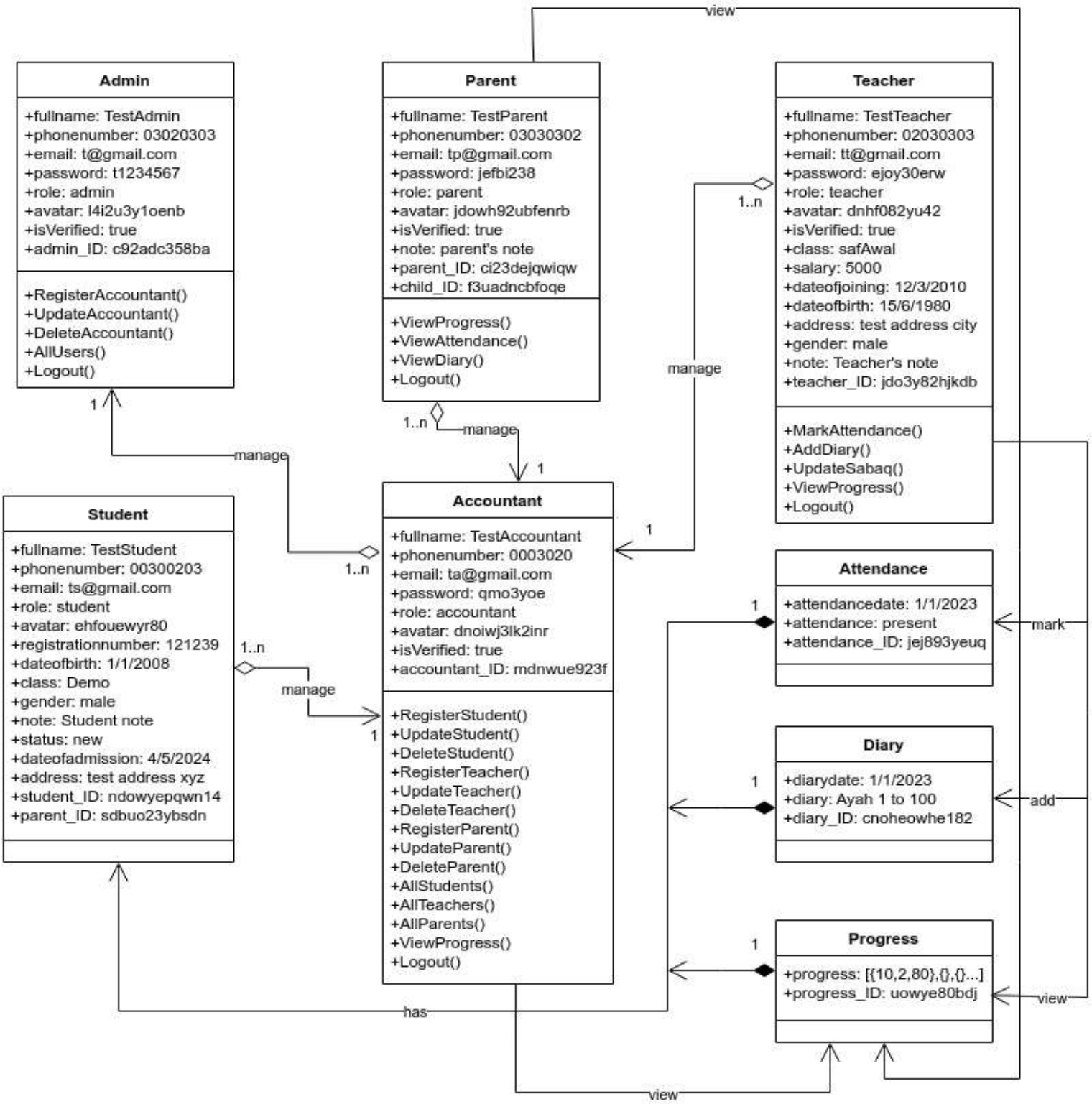


Figure 95 Object Diagram

4.8 Sequence Diagram

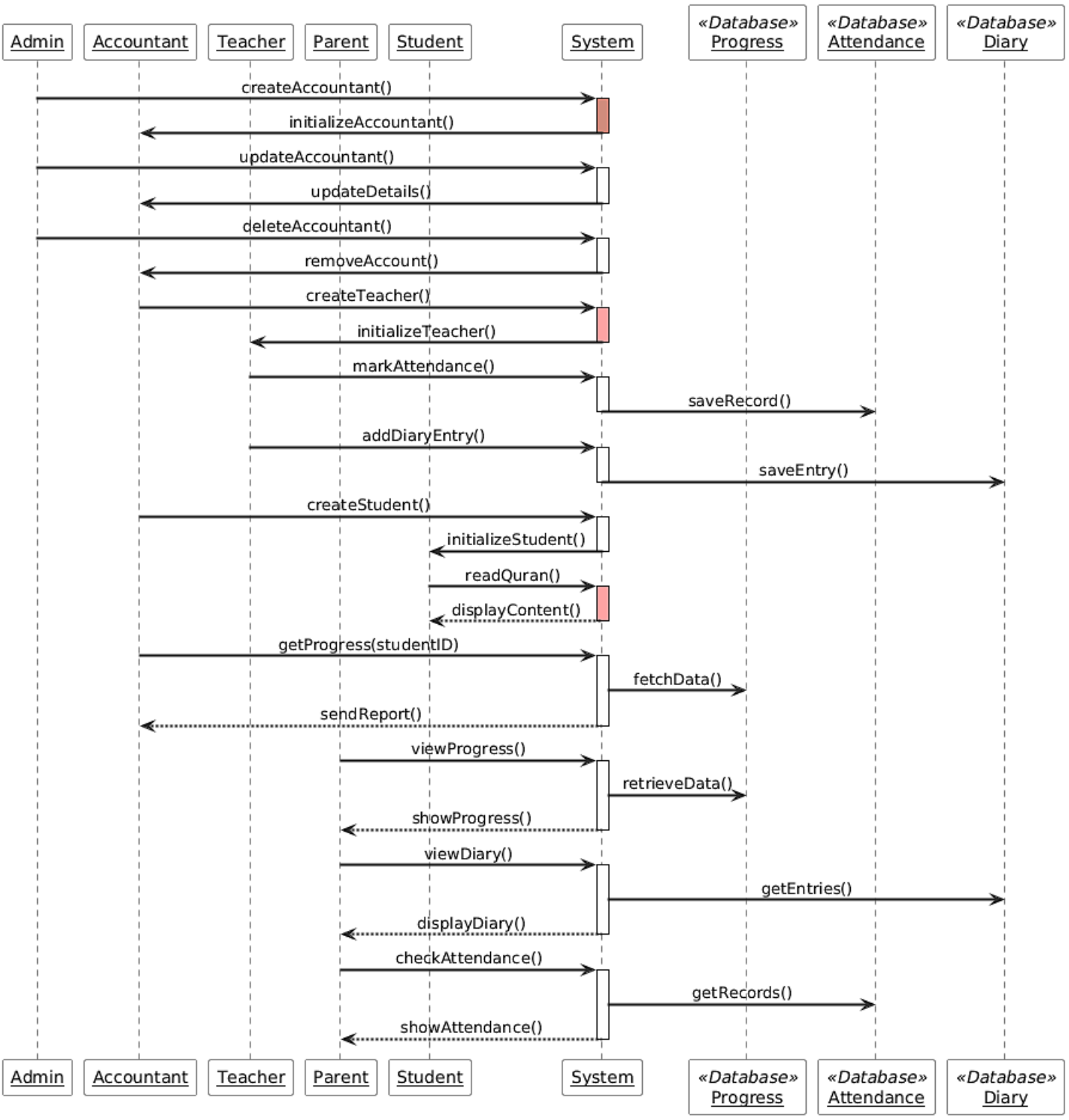


Figure 96 Sequence Diagram

4.8.1 Admin Register Accountant Sequence Diagram

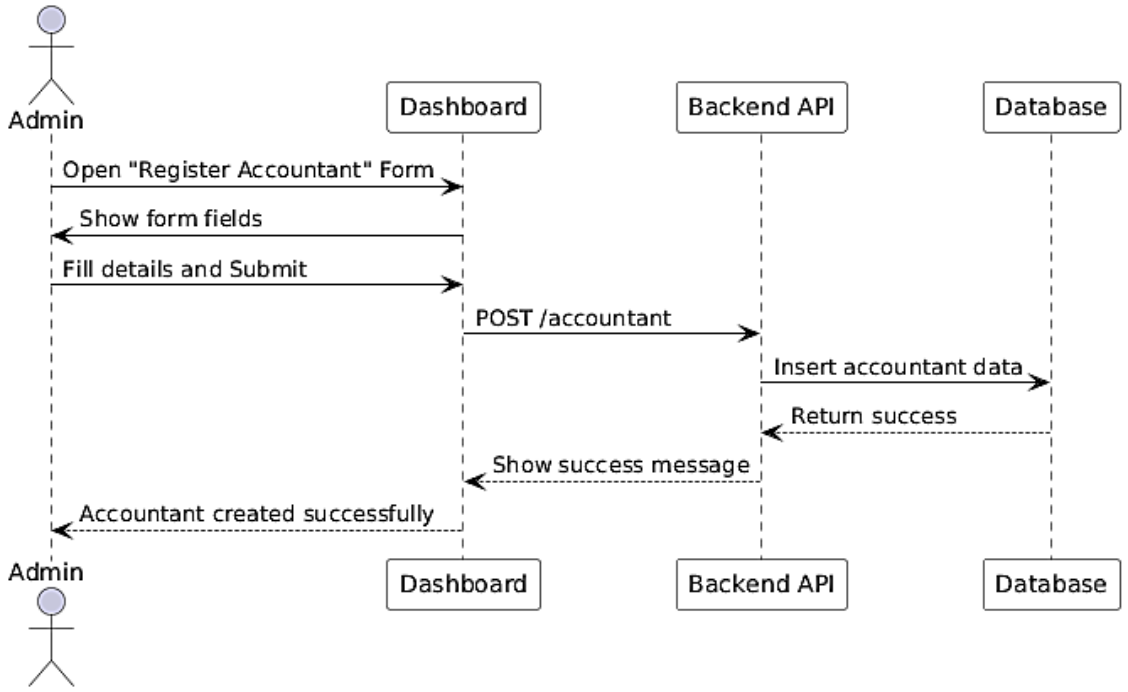


Figure 97 Admin Register Accountant Sequence Diagram

4.8.2 Admin Update Accountant Sequence Diagram

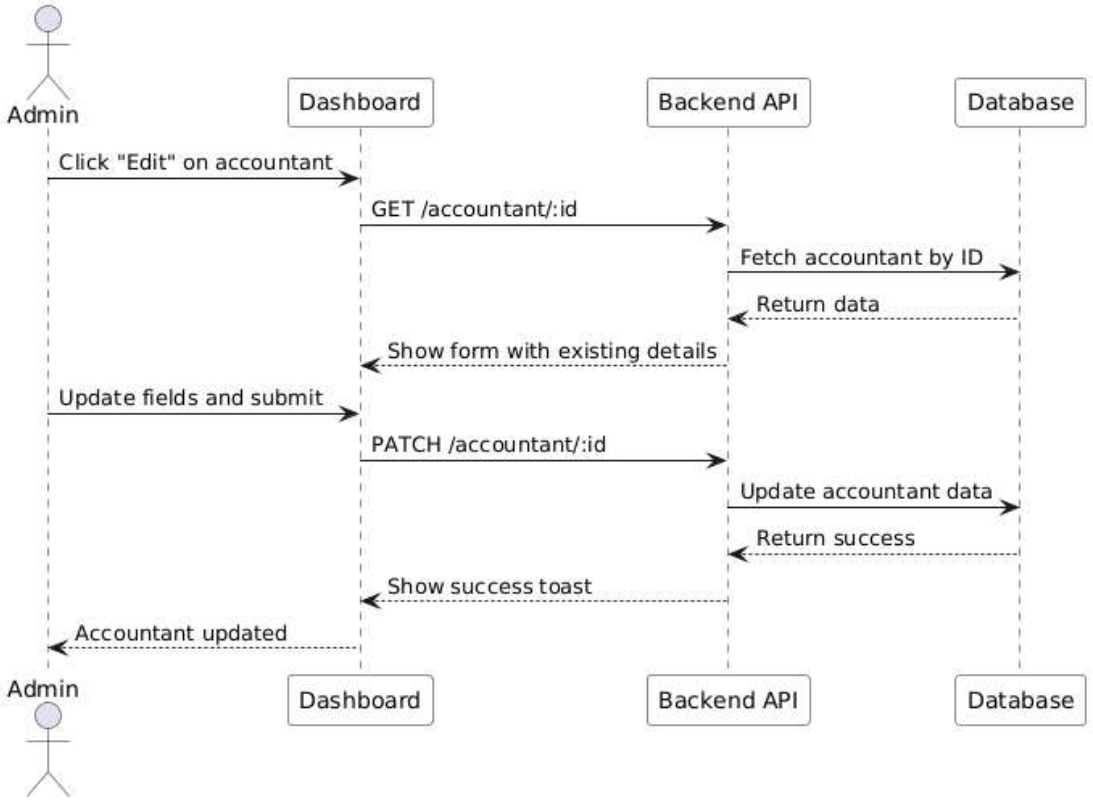


Figure 98 Admin Update Accountant Sequence Diagram

4.8.3 Admin Delete Accountant Sequence Diagram

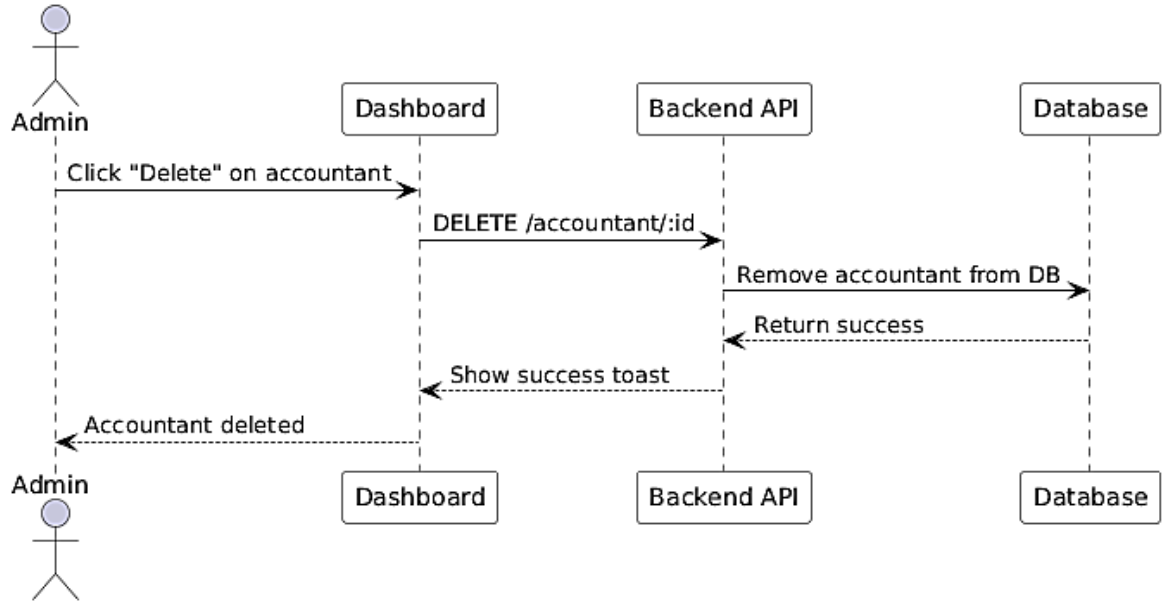


Figure 99 Admin Delete Accountant Sequence Diagram

4.8.4 Admin View Accountant List Sequence Diagram

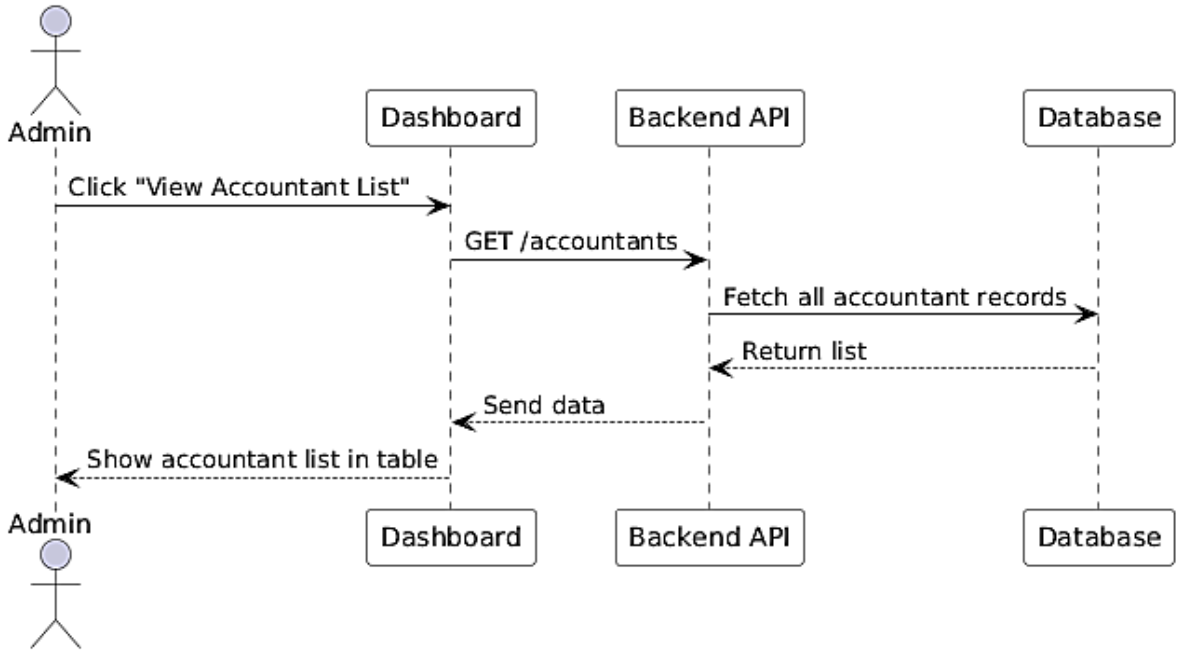


Figure 100 Admin View Accountant List Sequence Diagram

4.8.5 Accountant Create Student Sequence Diagram

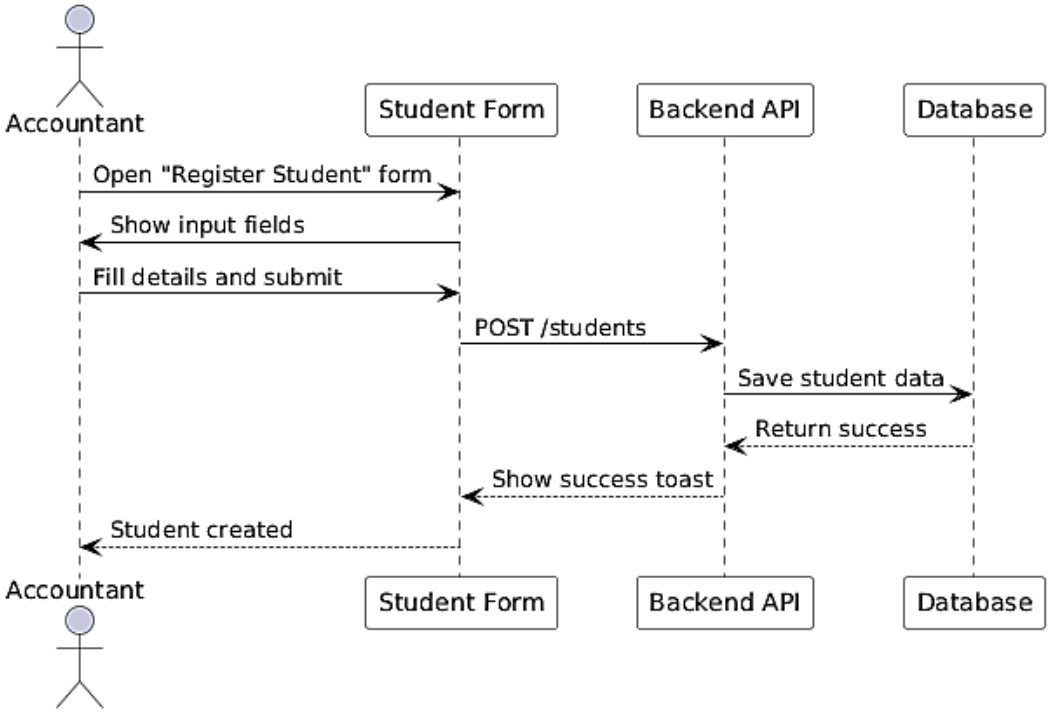


Figure 101 Accountant Create Student Sequence Diagram

4.8.6 Accountant Update Student Sequence Diagram

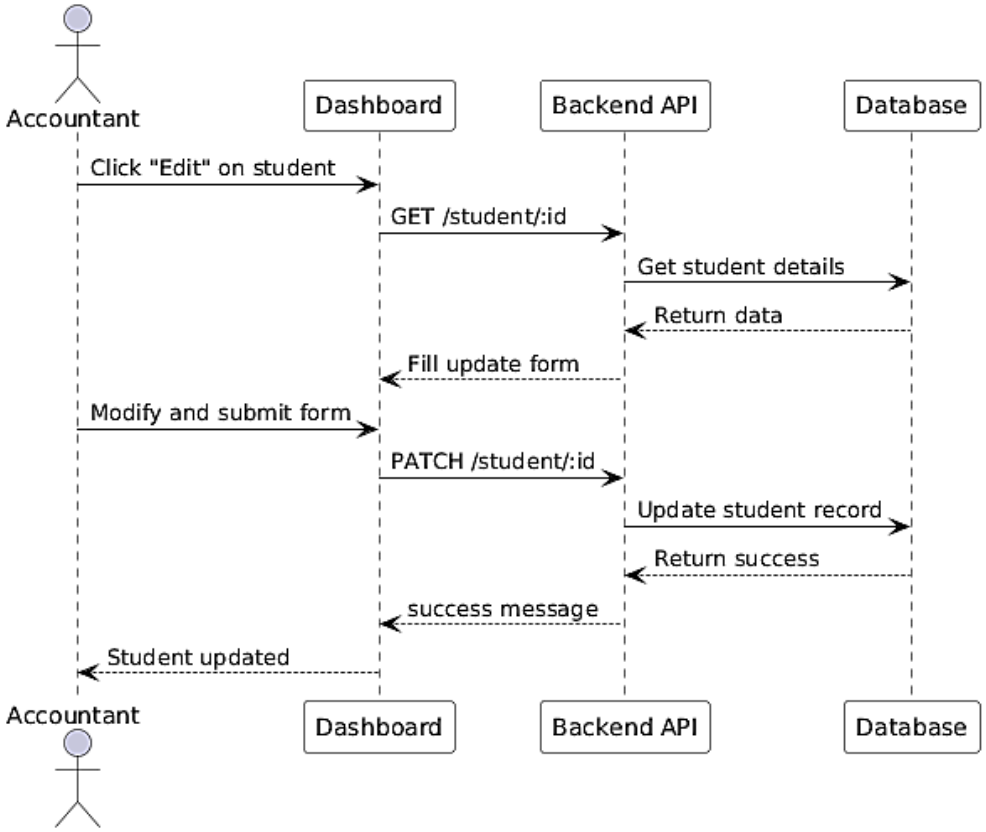


Figure 4.8.6 Accountant Update Student Sequence Diagram

4.8.7 Accountant Delete Student Sequence Diagram

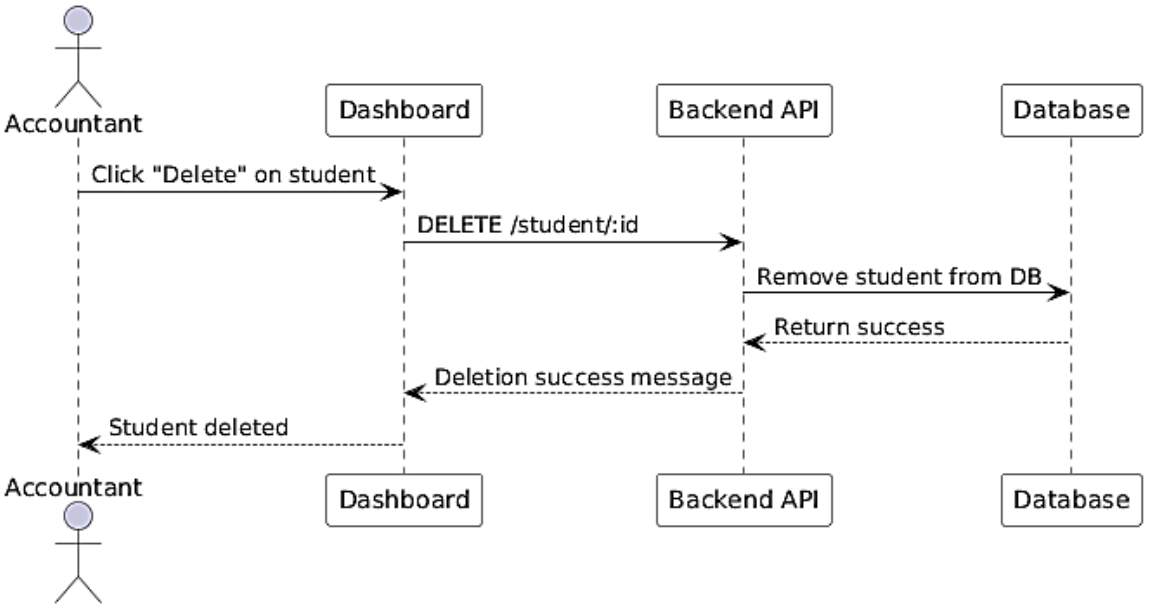


Figure 4.8.7 Accountant Delete Student Sequence Diagram

4.8.8 Accountant View Student List Sequence Diagram

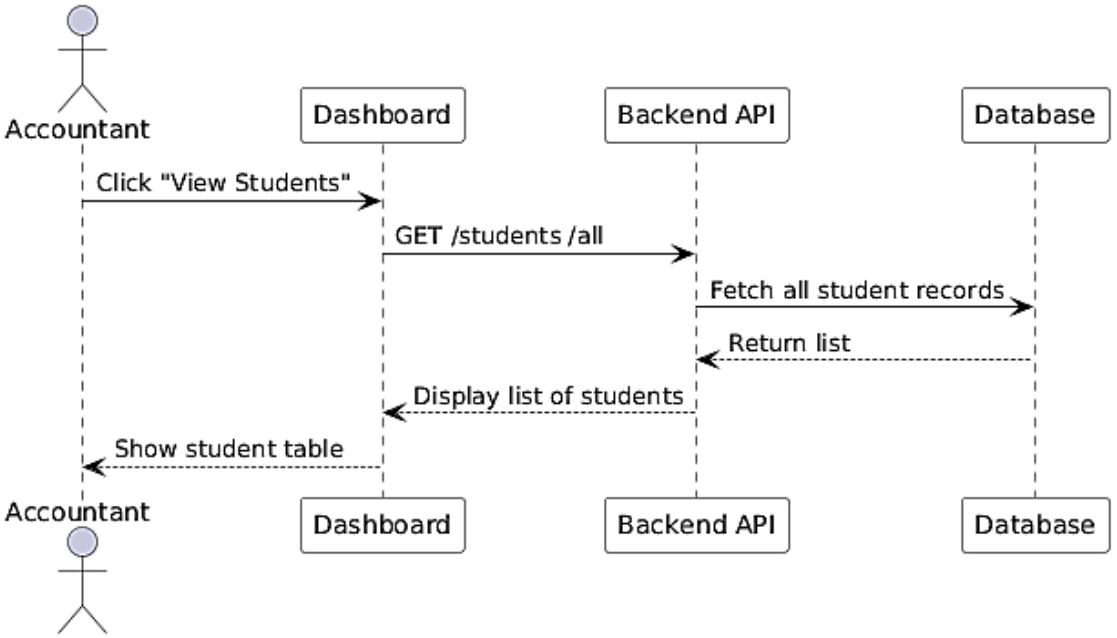


Figure 102 Accountant View Student List Sequence Diagram

4.8.9 Accountant Create Teacher Sequence Diagram

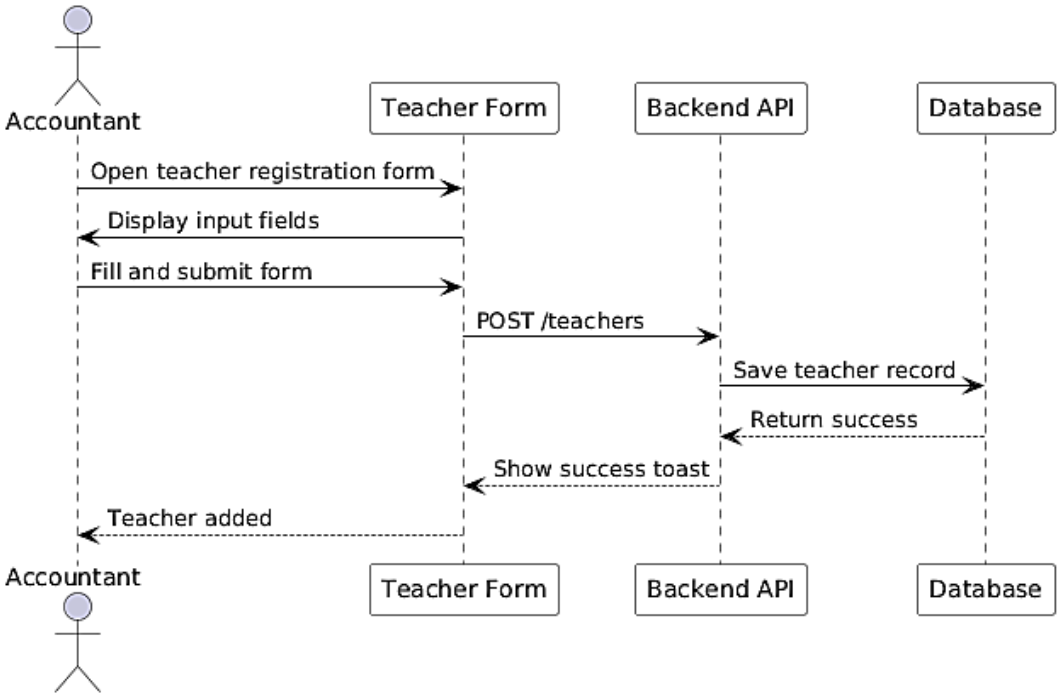


Figure 103 Accountant Create Teacher Sequence Diagram

4.8.10 Accountant View Teacher List Sequence Diagram

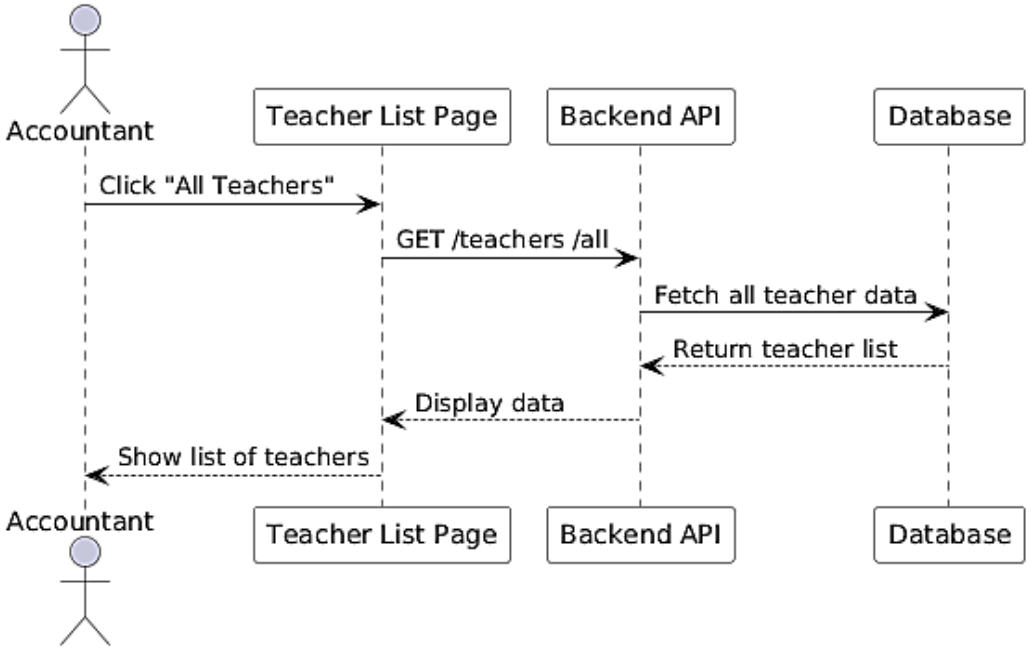


Figure 104 Accountant View Teacher List Sequence Diagram

4.8.11 Accountant Update Teacher Sequence Diagram

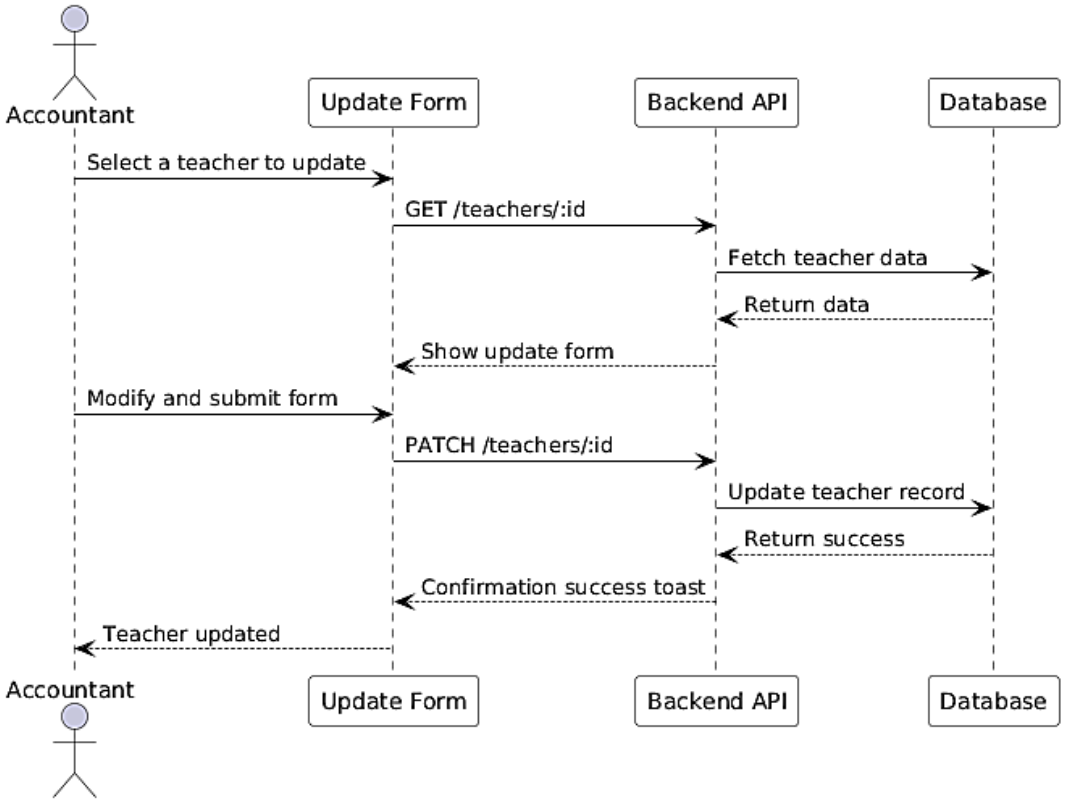


Figure 105 Accountant Update Teacher Sequence Diagram

4.8.12 Accountant Delete Teacher Sequence Diagram

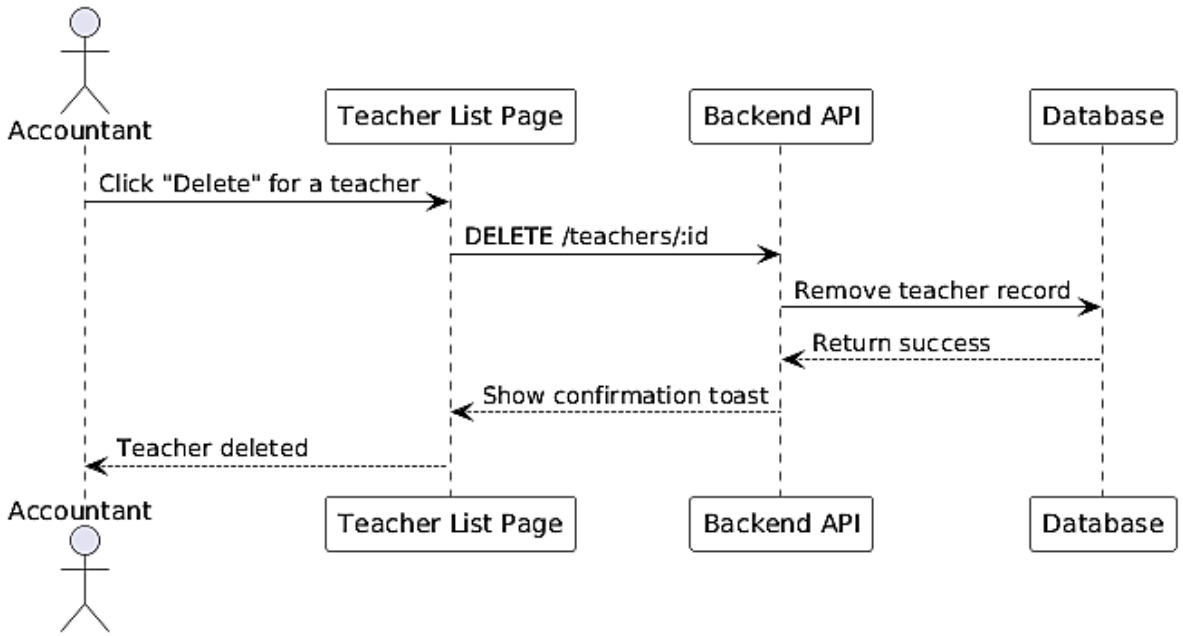


Figure 106 Accountant Delete Teacher Sequence Diagram

4.8.13 Accountant Create Parent Sequence Diagram

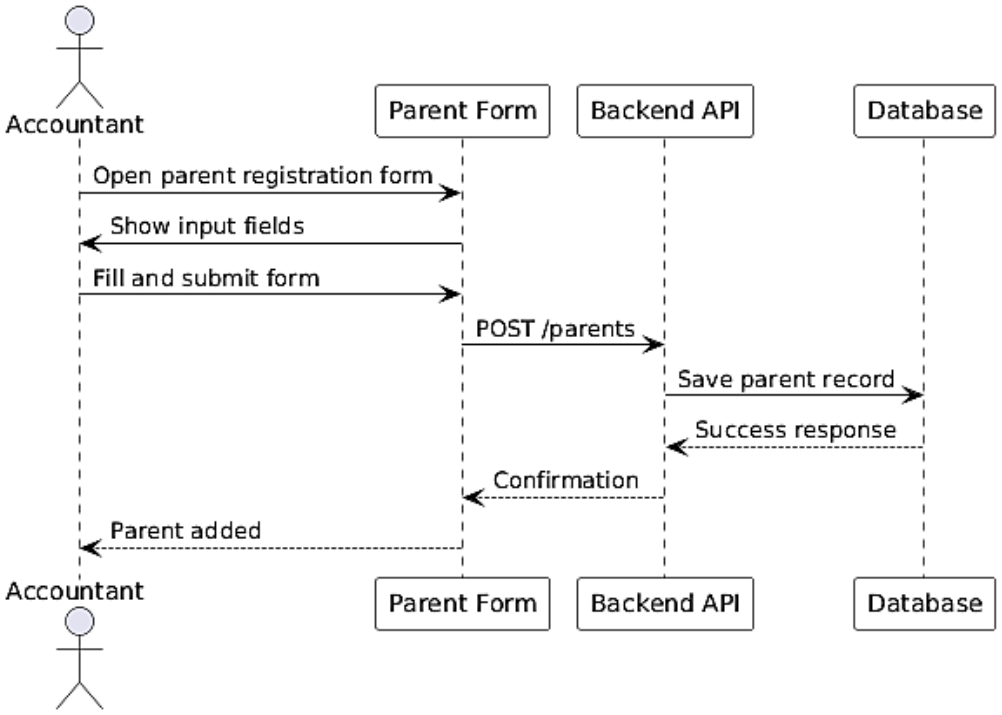


Figure 107 Accountant Create Parent Sequence Diagram

4.8.14 Accountant Update Parent Sequence Diagram

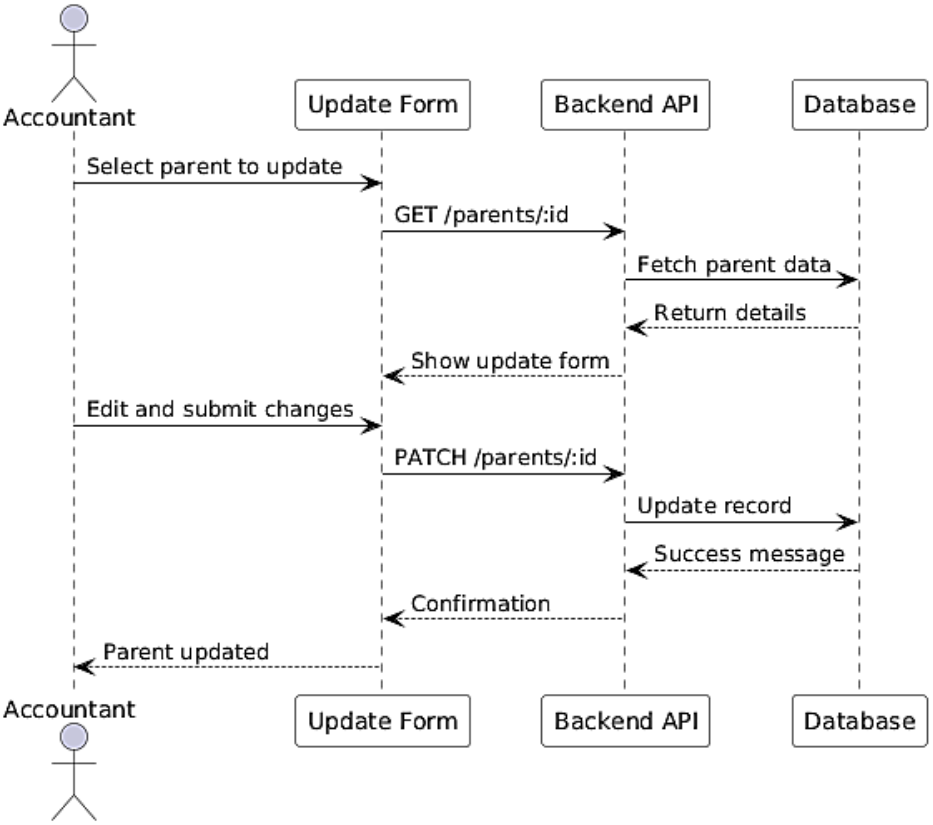


Figure 108 Accountant Update Parent Sequence Diagram

4.8.15 Accountant Delete Parent Sequence Diagram

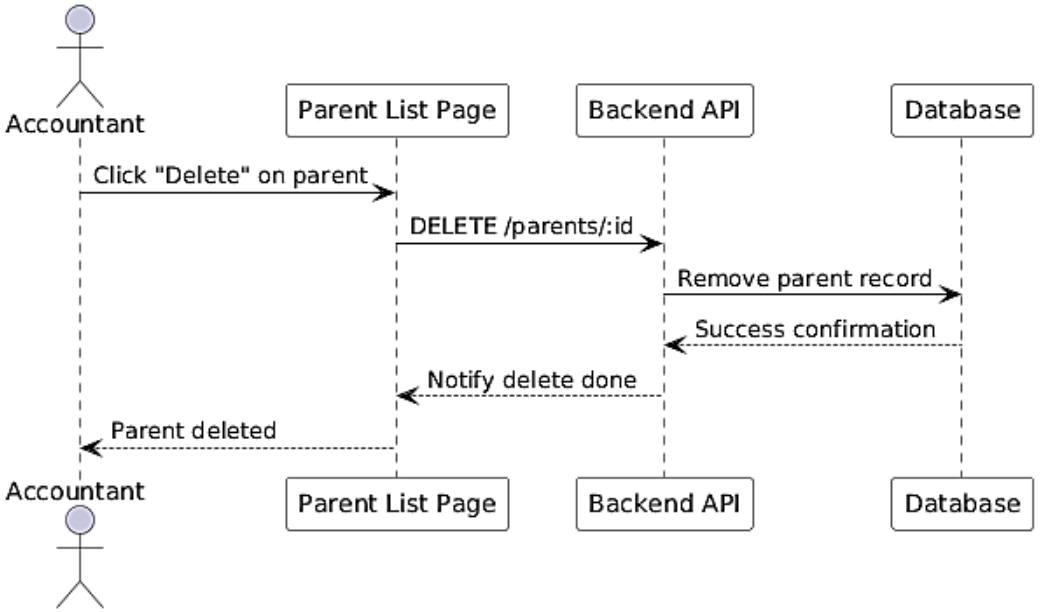


Figure 109 Accountant Delete Parent Sequence Diagram

4.8.16 Accountant View Parent List Sequence Diagram

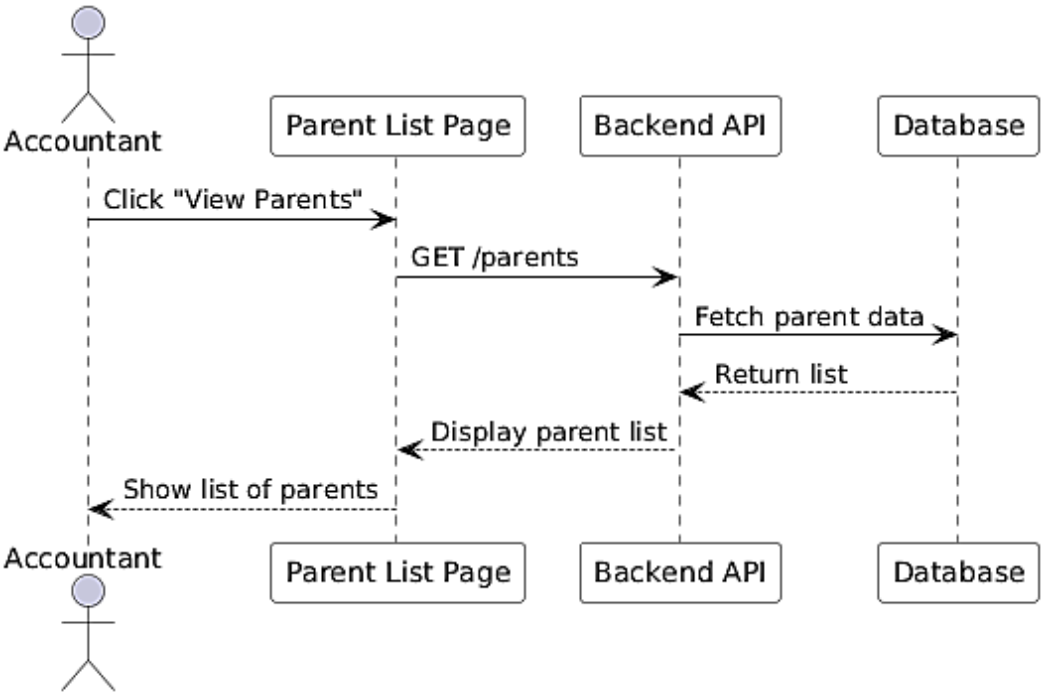


Figure 110 Accountant View Parent List Sequence Diagram

4.8.17 Accountant View Progress Sequence Diagram

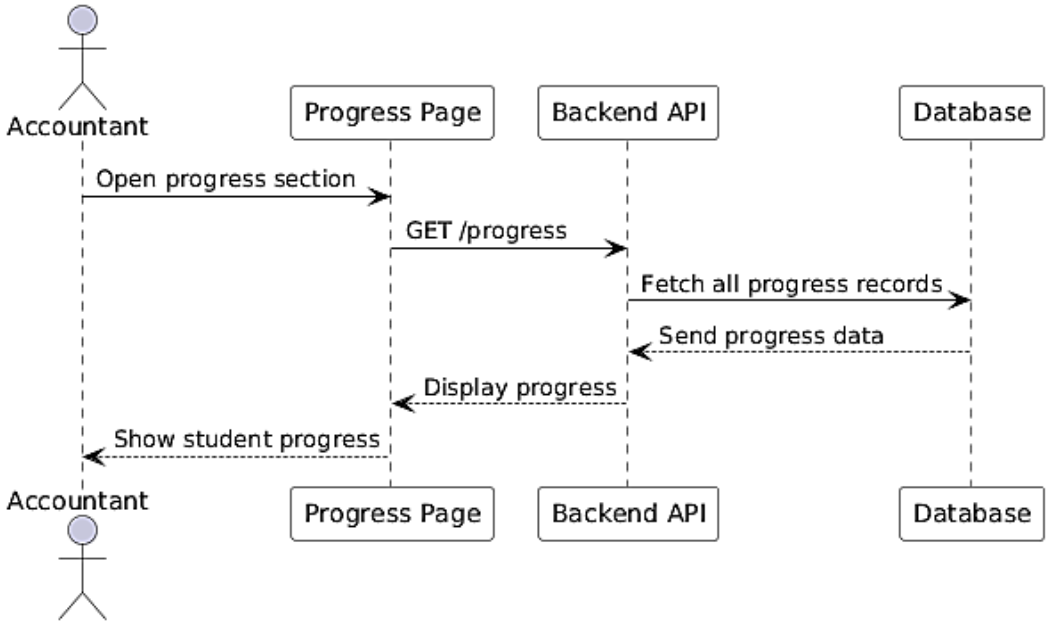


Figure 111 Accountant View Progress Sequence Diagram

4.8.18 Teacher Marks Attendance Sequence Diagram

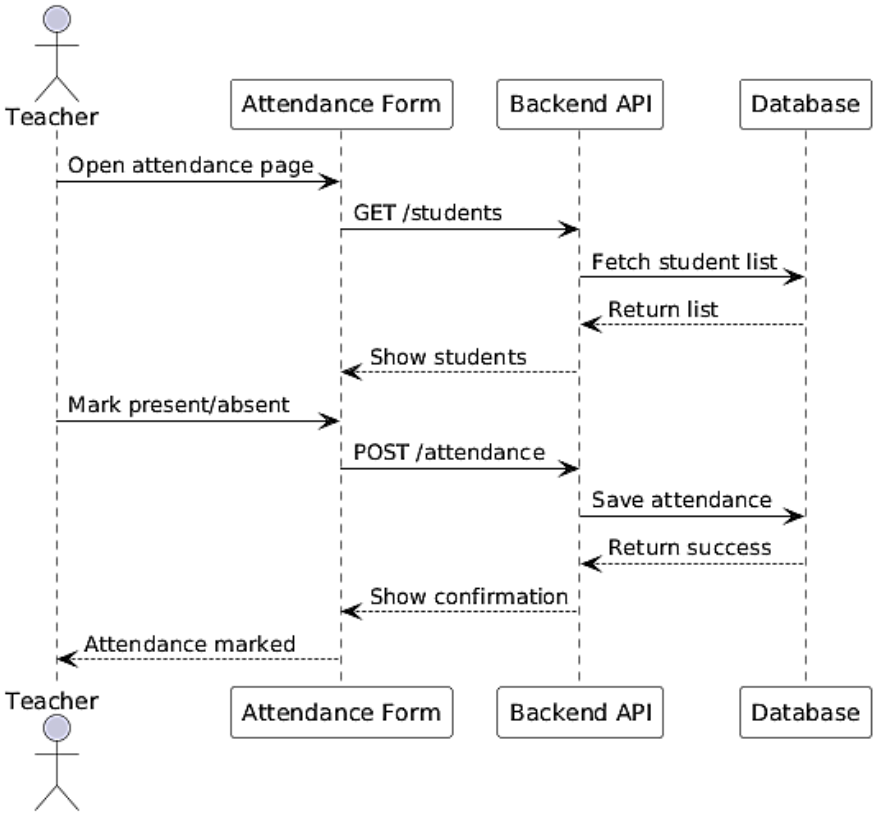


Figure 112 Teachers Marks Attendance Sequence Diagram

4.8.19 Teacher Add Diary Sequence Diagram

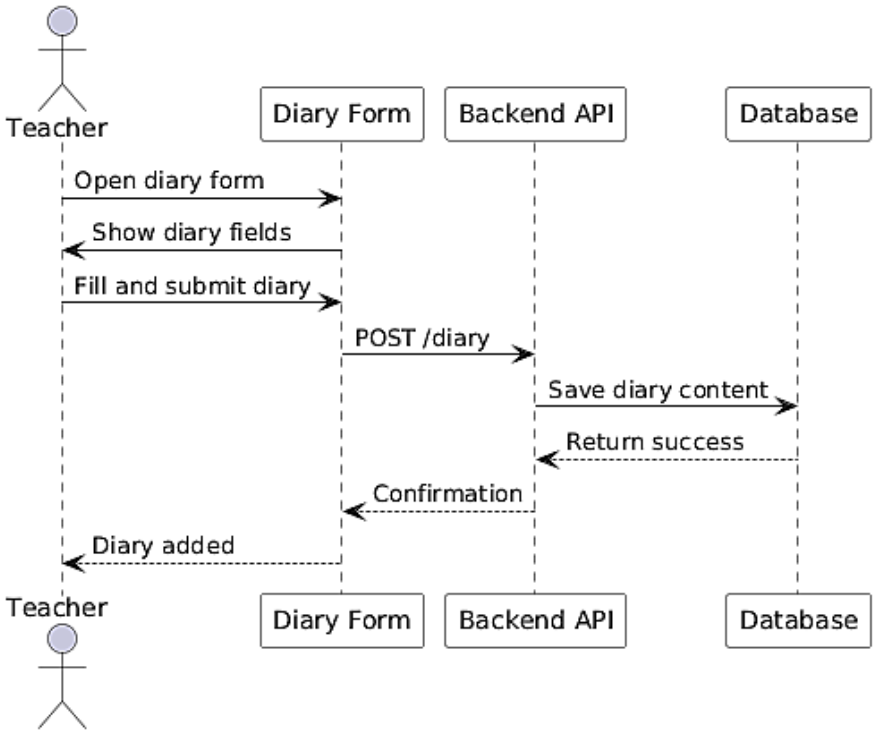


Figure 113 Teachers Add Diary Sequence Diagram

4.8.20 Teacher Update Sabaq Sequence Diagram

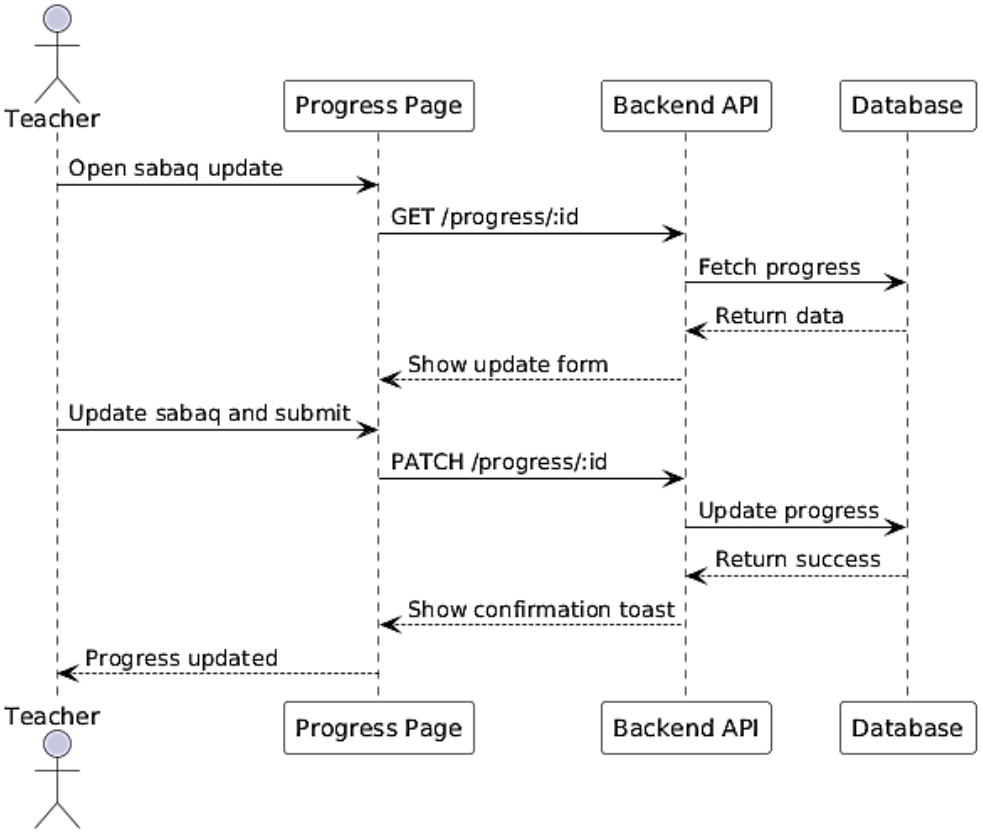


Figure 114 Teachers Update Sabaq Sequence Diagram

4.8.21 Teacher View Progress Sequence Diagram

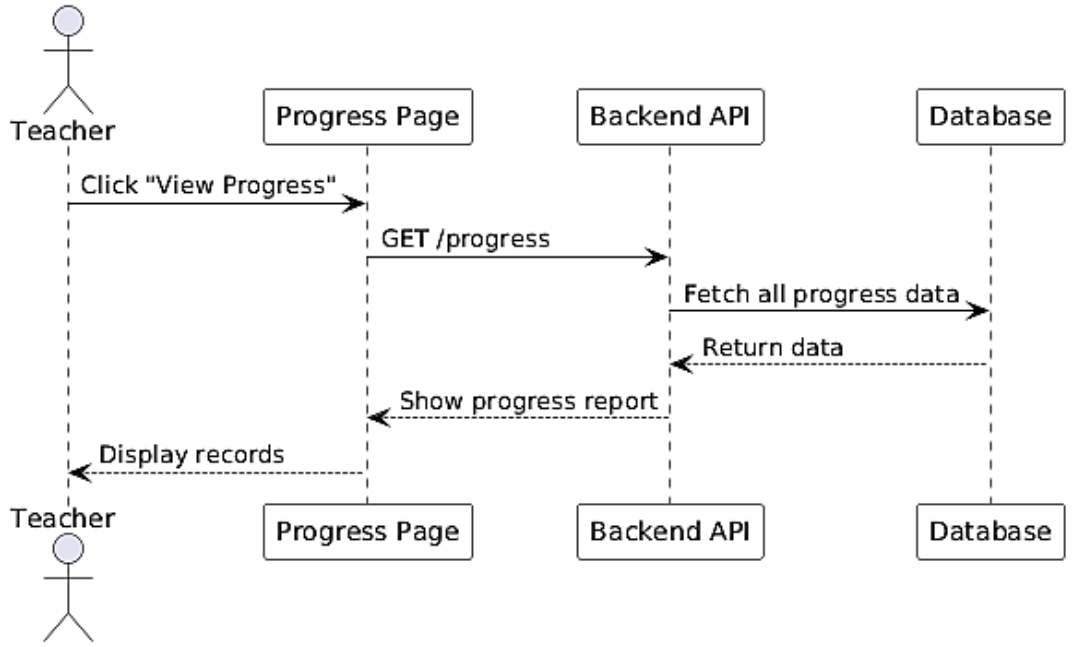


Figure 115 Teachers View Progress Sequence Diagram

4.8.22 Parent View Attendance Sequence Diagram

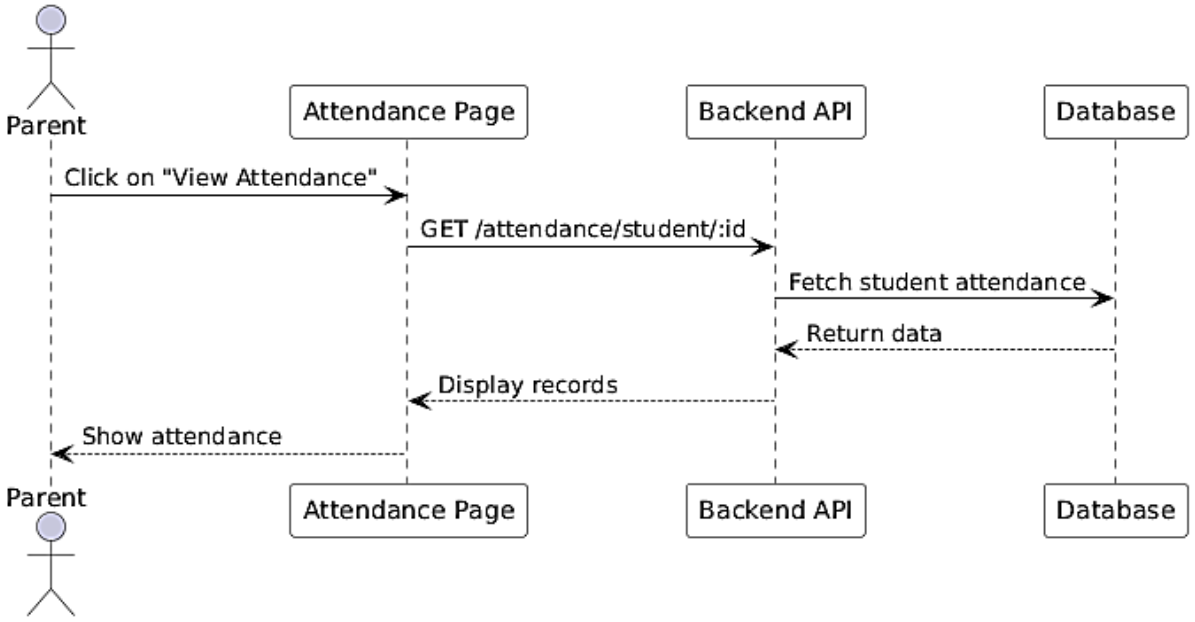


Figure 116 Parent View Attendance Sequence Diagram

4.8.23 Parent View Diary Sequence Diagram

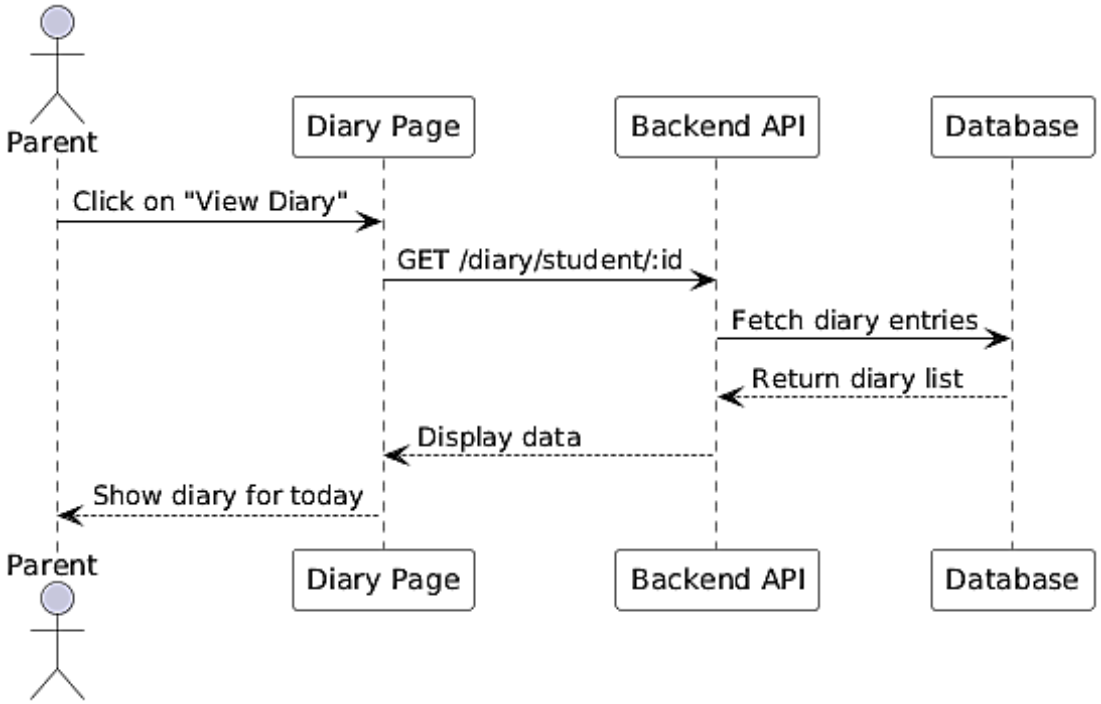


Figure 117 Parent View Diary Sequence Diagram

4.8.24 Parent View Progress Sequence Diagram

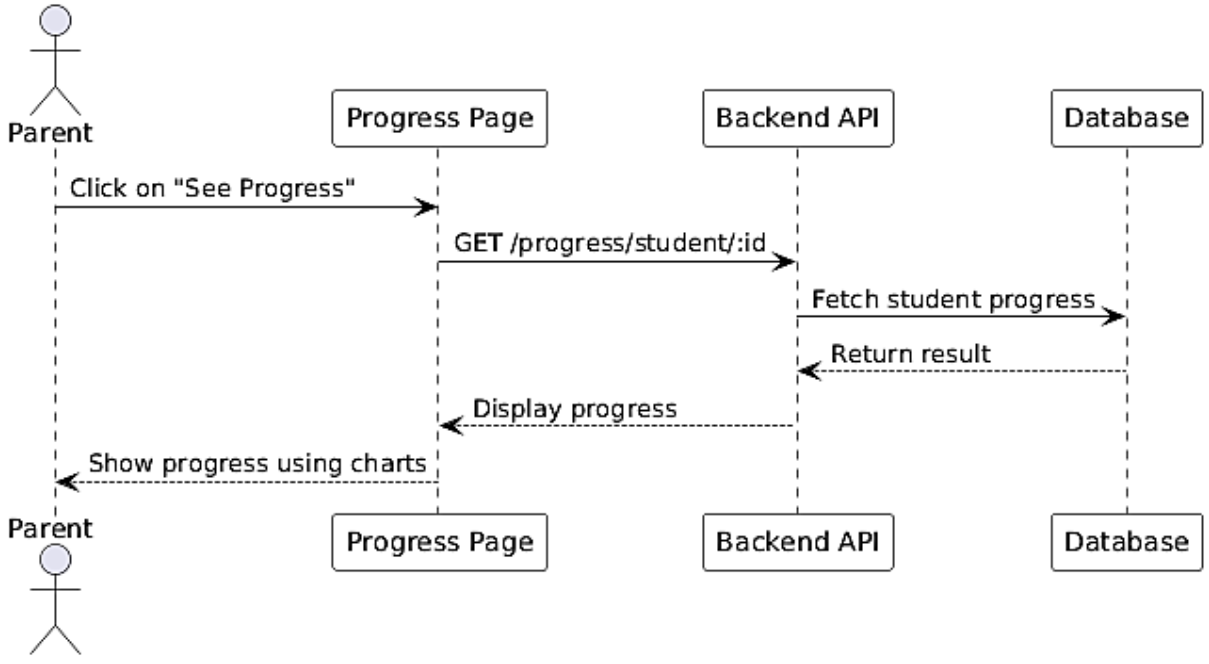


Figure 118 Parent View Progress Sequence Diagram

CHAPTER 5
IMPLEMENTATION

5.1 Development Tools and Environment

In this project, simple and useful tools were used to build and test the system. Below are the main tools and technologies that were used:

1. **Visual Studio Code**

   VS Code was used as the main code editor. It made it easy to write and manage both the frontend and backend code. It also supports extensions, which helped make coding faster and better.

2. **Postman and Thunder Client**

   For testing APIs, Postman and Thunder Client were used. These tools helped send requests to the backend and check if the API responses were correct.

3. **MERN Stack**

   Project implemented using MERN stack to build the full project:

   - MongoDB: For storing all data (users, attendance, records).
   - Express.js: For backend logic and APIs.
   - React.js: For frontend design and user interface.
   - Node.js: For running the backend server.

4. **Redux Toolkit and RTK Query**

   Redux Toolkit and RTK Query were used to manage app state and API data in React. RTK Query made data fetching simple and automatic, while Redux Toolkit helped keep the state organized.

5. **Git and GitHub**

   Git was used to save and manage changes in the code. GitHub was also used to store the project online and keep it safe.

6. **Browser Developer Tools**

   The browser dev tools (like in Chrome) were used to debug the frontend, check console errors, and improve performance.

7. **MongoDB Compass and Atlas**

   MongoDB Compass was used to view and manage database records. It helped to see all data clearly in tables and collections. All data was also stored in the MongoDB cloud, which is MongoDB Atlas, allowing access to data without needing it on a local system.

5.2 Modules Implemented

Module Name	Description
Login	User enters email and password to open the system.
Forget Password	User enters email to get OTP for resetting password.
Verify OTP	User enters the 6-digit OTP code received on email.
Reset Password	If OTP is correct, user can set a new password.
Admin Module	Admin can add, edit, delete, and view all accountants and other admins.
Accountant Module	Accountant manages students, teachers, parents and checks student progress.
Teacher Module	Teacher marks attendance, adds sabaq, writes diary, and sees student progress.
Parent Module	Parent can view child's diary, attendance, sabaq, and progress.
Quran Module	Allows the user to read and listen to the Quran.
Attendance module	Teacher can mark attendance, parents can view it.
Diary module	Teacher adds diary and notes, parents can read them.
Progress Module	Shows the student's progress to parent, teacher, and accountant.
Register Module	User can register new admin, accountant, teacher, student, and parent.
Update Module	Used to update details of admin, accountant, teacher, student, or parent.
Delete Module	Used to delete any admin, accountant, teacher, student, or parent.
Fee Module	Used to pay fee from parent pannel

Table 55 Modules Implemented

CHAPTER 6
SYSTEM TESTING

6.1 Testing Techniques Used

In this project, different testing techniques were used to check the working and behavior of the system. These methods helped to make sure that the system works correctly and without errors. The testing was done in the following ways:

1. **Functional Testing**

   This testing checks if every function in the project is working as expected. For example, login, register, and form submissions were tested by entering different inputs.

2. **Non-Functional Testing**

   This checks how the system behaves. Like speed, response time, and how easy it is to use and observed if the pages load fast and work smoothly.

3. **Unit Testing**

   Small parts of the project, like components and functions, were tested one by one to see if they gave correct results. For example, input validation and button clicks were tested.

4. **Integration Testing**

   After unit testing, different parts were connected together and tested to see if they worked properly as a group. For example, it was checked if the data went to the database after registration and if the admin could see it.

5. **System Testing**

   The full system was checked after completion. Login, registration, attendance, diary, and progress modules were tested to ensure they were working together.

6. **Acceptance Testing**

   The full project was checked like a normal user to see if it was easy to use and working correctly. This helped to determine if the project was ready for use.

7. **Stress Testing**

   If multiple dashboards are opens at same time the system not goes down or crashed.

8. **Load Testing**

   This project handles a small number of users due to limited storage on MongoDB Atlas and restricted image upload capacity on Cloudinary. Upgrading to a paid plan would allow for the storage of a larger number of user data.

9. **Security Testing**

   Whenever user trigger any event whole page refresh and a new request goes to backend and match token assigned to it, this make system more secure from unauthorized user instead of storing token into browser sessional storage.

6.2 Test Plans

6.2.1 GUI Testing

Test Objective	To check that the user interface (UI) looks good and works properly for the user.
Technique	Manual testing by using the app, will check buttons, input fields, layout, colors, and design elements.
Completion Criteria	All GUI elements are visible, clickable, and working fine. The design should match the planned layout.
Special Considerations	Check on different screen sizes (mobile, tablet, desktop). Make sure fonts and colours are easy to read. Ensure buttons and links go to the correct pages. Test for responsive design and alignment.

Table 56 GUI Testing

6.2.2 Functional Testing

Test Objective	To check that each feature of the system works correctly as expected.
Technique	Manual testing using test cases made from the project's requirements. Each function will be tested step by step.
Completion Criteria	All functions work properly without any error. The output should match what is expected from the input.
Special Considerations	Focus on important features like login, registration, and data saving. Test both correct and wrong inputs. Check system behaviour when user leaves fields empty or enters wrong data.

Table 57 Functional Testing

6.2.3 Performance Testing

Test Objective	To check how fast and smooth the app works when many users use it or when there is a lot of data.
Technique	Using tools like JMeter or LoadRunner to apply load and test speed, response time, and stability.
Completion Criteria	The app should load within a few seconds and should not crash or slow down under heavy load.
Special Considerations	Test during login, form submissions, and data fetching. Check how much load the server can handle. Make sure the app stays fast even with many users. Watch memory and CPU usage under load.

Table 58 Performance Testing

6.2.4 Load Testing

Test Objective	To check how the system works when many users use it at the same time.
Technique	Use tools (like JMeter) to send many requests to the system at once and check the response time.
Completion Criteria	The system should not crash and should respond within a reasonable time, even with heavy usage.
Special Considerations	Focus on login, registration, and other important features. Test during peak usage hours. Monitor CPU, memory, and server speed during the test.

Table 59 Load Testing

6.2.5 Configuration Testing

Test Objective	To make sure the app works properly on different devices, browsers, and operating systems.
Technique	Install and run the app on different setups (like Windows, Android, Chrome, Firefox, etc.) and check if everything works fine.
Completion Criteria	The app should run smoothly without any issues on all supported devices and platforms.
Special Considerations	Test on both old and new versions of browsers. Check mobile and desktop versions. Look for layout or button issues on different screens. Make sure there is no crash or error due to settings.

Table 60 Configuration Testing

6.2.6 Browser Testing

Test Objective	To check if the website or web app works properly on different browsers like Chrome, Firefox, Edge, etc.
Technique	Open the website on different browsers and test all pages and features to see if anything looks broken or doesn't work.
Completion Criteria	The website should look good and work the same way on all major browsers.
Special Considerations	Test on both desktop and mobile browsers. Check for layout issues, button clicks, and loading problems. Make sure styles (CSS) and scripts (JavaScript) run smoothly on all browsers.

Table 61 Browser Testing

6.3 Test Cases

6.3.1 Login Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-Login-01	Valid Email, Valid Password	User should be redirected to dashboard	Check login with correct email & password	Pass
TC-Login-02	Valid Email, Wrong Password	Error message: "Incorrect password"	Check login with wrong password	Pass/Fail
TC-Login-03	Invalid Email, Any Password	Error message: "User not registered"	Check login with unregistered email	Pass/Fail
TC-Login-04	Blank Email and Password	Show message: "Fields cannot empty"	Check login with empty fields	Pass/Fail
TC-Login-05	Invalid Email Format	Show message: "Enter a valid email address"	Check login with invalid email format	Pass/Fail

Table 62 Login Test Case

6.3.2 OTP Verification Test Case:

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-OTP-01	Valid OTP	Access to reset password	Correct OTP entered	Pass/Fail
TC-OTP-02	Invalid OTP	Show error: "Invalid OTP"	Wrong OTP entered	Pass/Fail
TC-OTP-03	Blank OTP	Show message: "Please enter OTP"	Empty OTP field	Pass/Fail
TC-OTP-04	Old OTP	Show message: "OTP expired"	Expired OTP	Pass/Fail
TC-OTP-05	Letters in OTP field	Show message: "Enter 6-digit code send to provided email"	OTP format check	Pass/Fail

Table 63 OTP Verify Test Case

6.3.3 Reset Password Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-RP-01	New = "abc123", Confirm =	Password should reset successfully	Correct new password and confirm password	Pass/Fail

	"abc123"			
TC-RP-02	New = "abc123", Confirm = "xyz456"	Show error: "Passwords do not match"	Password and confirm password do not match	Pass/Fail
TC-RP-03	Blank values	Show message: "Fields cannot be empty"	Empty password fields	Pass/Fail
TC-RP-04	New = "123"	Show message: "Password must be strong"	Weak password entered	Pass/Fail
TC-RP-05	Expired OTP + valid new password	Show error: "OTP expired, try again"	Reset after OTP expired	Pass/Fail

Table 64 Reset Password Test Case

6.3.4 Register User Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-RU-01	Name, email, password, role	User should be registered successfully	All valid fields	Pass/Fail
TC-RU-02	Blank name or email	Show message: "All fields are required"	Missing required fields	Pass/Fail
TC-RU-03	Email = "abc.com"	Show message: "Enter a valid email"	Invalid email format	Pass/Fail
TC-RU-04	Password = "123"	Show message: "Password too weak"	Password too short or weak	Pass/Fail
TC-RU-05	Email already exists	Show message: "Email already in use"	Duplicate email	Pass/Fail

Table 65 Register User Test Case

6.3.5 Update User Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-UU-01	Name, email, password	User data should be updated successfully	Update with valid data	Pass/Fail
TC-UU-02	Email = "abc"	Show message: "Invalid email format"	Update with invalid email	Pass/Fail

TC-UU-03	Blank fields	Show message: "Fields cannot be empty"	Update with empty fields	Pass/Fail
TC-UU-04	No changes made	Show message: "No changes made"	Update without change any field	Pass/Fail

Table 66 Update User Test Case

6.3.6 Delete User Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-DU-01	Valid user ID	User should be deleted successfully	Delete with valid user ID	Pass/Fail
TC-DU-02	User ID does not exist	Show message: "User not found"	Delete with invalid user ID	Pass/Fail
TC-DU-03	Normal user tries to delete	Show message: "You are not allowed"	Unauthorized delete attempt	Pass/Fail
TC-DU-04	No changes made	Show message: "No changes made"	Try to delete admin by mistake	Pass/Fail

Table 67 Delete User Test Case

6.3.7 Get All Users Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-GU-01	API call	Show list of all registered users	Fetch all users	Pass/Fail
TC-GU-02	Not verified	Show message: "Unauthorized"	Unauthorized access	Pass/Fail
TC-GU-03	Server down	Show message: "Something went wrong"	Server error	Pass/Fail
TC-GU-04	No registration	Show message: "No user found "	If no user registered in database	Pass/Fail

Table 68 Get All Users Test Case

6.3.8 Add Diary Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-AD-01	Student ID, date, diary note	Show message: "Diary added successfully"	Add diary for a student	Pass/Fail
TC-AD-02	No note given	Show error: "Note is required"	Add diary with missing data	Pass/Fail

TC-AD-03	No login token	Show error: "Unauthorized access"	Unauthorized diary add	Pass/Fail

Table 69 Add Diary Test Case

6.3.9 Mark Attendance Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-MA-01	Student ID, status = present	Show message: "Attendance marked"	Mark student present	Pass/Fail
TC-MA-02	Student ID, status = absent	Show message: "Attendance marked"	Mark student absent	Pass/Fail
TC-MA-03	Same student & date	Show error: "Attendance already marked"	Already marked attendance	Pass/Fail

Table 70 Mark Attendance Test Case

6.3.10 Add Daily Sabaq Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-AS-01	Student ID atkan,sabqi, manzil	Show message: "Progress saved"	Add student progress	Pass/Fail
TC-AS-02	No record added	Show error: "All fields required"	Missing any field	Pass/Fail
TC-AS-03	Record already added	Show error: "Sabaq already added"	Sabaq already added	Pass/Fail

Table 71 Add Daily Sabaq Test Case

6.3.11 Read and Listen Quran Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-QR-01	List of Surah	Show verses in Arabic	Read Quran online	Pass/Fail
TC-QR-02	Ayah or Surah	Play Quran audio of each Ayah	Listen Quran audio	Pass/Fail
TC-QR-03	No connection	Show message: "No internet connection"	Internet not available	Pass/Fail

Table 72 Read and Listen Quran Test Case

6.3.12 View Diary Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-VD-01	Date = Today	Show diary content	Show diary content	Pass/Fail
TC-VD-02	Wrong date	Show message: "No diary available"	Show message: "No diary available"	Pass/Fail
TC-VD-03	Parent not linked	Show message: "Access denied"	Show message: "Access denied"	Pass/Fail

Table 73 View Diary Test Case

6.3.13 View Attendance Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-VA-01	Student ID	Show present/absent status with dates	View student attendance	Pass/Fail
TC-VA-02	Wrong ID	Show message: "No attendance found"	No record found	Pass/Fail

Table 74 View Attendance Test Case

6.3.14 View Progress Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-VP-01	Student ID	Show weekly or monthly progress details	View progress report	Pass/Fail
TC-VP-02	Wrong ID	Show message: "No progress record"	No progress available	Pass/Fail

Table 75 View Progress Test Case

6.3.15 Logout Test Case

Test Case ID	Input	Expected Output	Test Case Description	Status
TC-LO-01	Click logout	Redirect to login page	Logout successfully	Pass/Fail
TC-LO-02	Token expired	Show message: "Session ended"	Session expired logout	Pass/Fail

Table 76 Logout Test Case

6.4 Results and Bug Reports

Bug ID	Module	Bug Description	Cause	Status
BG-01	Form	Submit button not working properly	Backend Api call not hit the desired route	Fixed
BG-02	Formik Yup	Form errors not showing correctly	Wrong field name used in validation schema	Fixed
BG-03	Redux Toolkit	State not updating after login	Didn't dispatch the login action properly	Fixed
BG-04	RRD	Page not navigating after clicking a link	Route path was incorrect	Fixed
BG-05	React Toastify	Toast not showing success message	Forgot to import CSS file of react-toastify	Fixed
BG-06	Express	User not able to register	Body-parser middleware was not used	Fixed
BG-07	JWT	Token expired after every visit	No token expiry time set in backend	Fixed
BG-08	Nodemailer	OTP not received on email	SMTP settings were incorrect	Fixed
BG-09	Validator.js	Validation not applied to fields	Validator not import and used	Fixed
BG-10	Cloudinary	Image not uploading	API key was not properly set in .env file	Fixed
BG-11	React Magnifier	Magnifier not work on images	Required tag not used	Fixed
BG-12	CORS	API request blocked	Forgot to allow frontend origin in CORS settings	Fixed
BG-13	Nodemon	Server not restarting on file	Nodemon not conFigureured	Fixed
BG-14	Redux	State is not reflecting changes	Slice did not export actions	Fixed
BG-15	Mongoose	Data not saving in MongoDB	Required fields missing or schema not matching properly	Fixed

Table 77 Bug Report Table

CHAPTER 7
RESULTS AND DISCUSSION

7.1 Performance Metrics

To know if the EMadrasa System is working well or not, some important points were checked. These points are called performance metrics. They help us understand how fast, smooth, and useful the system is. Below are the main things that were tested:

7.1.1 Page Load Time

One of the most important things is how quickly a page opens. When teachers, parents, or admins click on something, they should not have to wait. During testing, almost all pages opened in less than 2 seconds. This means the system is fast and easy to use, even if someone is using it on slow internet.

Page Name	Avg. Load Time	Acceptable Time	Status
Login Page	1 sec	< 1 sec	Good
Dashboard	1.2 sec	< 1.5 sec	Good
Student Progress	1.5 sec	< 2 sec	Good

Table 78 Page Load Time

7.1.2 Data Saving Time

Another important thing is how fast the data is saved. When a teacher marks attendance or adds sabaq, manzil, or atkan it should not take long. In this system, the data gets saved almost instantly (in less than 1 second). This helps save time and avoids mistakes, especially when there are many students.

Action	Avg. Save Time	Acceptable Limit	Status
Save Attendance	1 sec	< 2 sec	Good
Update Sabaq	1.3 sec	< 2 sec	Good
Edit Student Info	1.5 sec	< 2 sec	Good

Table 79 Data Saving Time

7.1.3 System Uptime

System uptime means how much time the system stays working without crashing or stopping. During testing, the system was live and working 99% of the time. There were no big problems or errors. This shows that the system is reliable and can be used daily without worries.

Month	Total Uptime	Downtime	Status
January	99.4%	8 hours	Excellent
February	99.2%	8 hours	Excellent
April	99.3&	8 hours	Excellent
May	99.5%	8 hours	Excellent
June	99.6%	8 hours	Excellent

Table 80 System Uptime

7.1.4 Multiple Users at the Same Time

The system was also tested by using it with multiple users at the same time. For example, if a teacher is marking attendance, an admin is checking reports, and a parent is logging in — the system should still work properly. During this test, more than 10 users used the system together, and there were no issues. Everything worked smoothly.

No. of Users	Avg. Response Delay	System Errors	Status
3	0.1 sec	None	Smooth
5	0.3 sec	None	Smooth
7	0.6 sec	timeout	Acceptable
10	1.0 sec	Slow request	Near limit

Table 81 Multiple User at same Time

7.1.5 Database Speed

All the data in this system is stored in a database. If the database is slow, then the whole system becomes slow. But in this case, even when there were many records, like attendance, sabaq, and student info the system still worked fast. This means the database is set up well and can handle real-life usage.

Operation	Max Query Time	Avg. Query Time	Acceptable Time
Fetch Attendance Record	1.2 sec	0.8 sec	< 2 sec
Update Student Profile	1.0 sec	0.7 sec	< 2 sec
View Progress	0.9 sec	0.6 sec	< 2 sec

Table 82 Multiple User at same Time

7.1.6 Mobile and Desktop Compatibility

The system was checked on different devices, like mobile phones, tablets, and laptops. It worked fine on all screens. This is very important because many parents and teachers use mobile phones.

Operation	Tested Browsers	Result
Desktop (Windows)	Chrome, Edge, Firefox	Fully Working
Laptop (Mac)	Safari, Chrome	Fully Working
Android Mobile	Chrome, Firefox, Opera	Fully Working
iPhone (iOS)	Safari, Chrome	Fully Working

Table 83 Database Speed

7.1.7 Low Internet Handling

The system was also tested on slow internet. Pages still loaded fine, and data was saved properly. This is a useful feature for madrasas in areas with weak internet connections. Loaders are gracefully implemented. Offline pop up also implement for better visualization.

Summary

All these performance points show that the EMadrasa System is fast, reliable, and easy to use. It works well even with many users, on slow internet, and with large data. These results prove that this system is ready for real madrasa use and can really help teachers, admins, and parents in a simple way.

7.2 Evaluation of Results

The EMadrasa Management System was carefully tested after development to make sure that all features were working properly. Since the system is still under development and not yet deployed in a real madrasa, testing was done in a controlled environment rather than by real users.

All major features were checked one by one. These included:

- Login and registration
- Attendance marking and viewing
- Tracking of sabaq, manzil, and atkan
- Rolebased access for admin, teachers, and parents
- Automatic progress reports

Each function was tested manually, and the system gave the correct response every time. When student data was added, it was saved correctly, and the progress reports showed updated information without errors.

The system was also tested on both desktop and mobile devices to check compatibility. It worked smoothly on both, and all pages loaded quickly. Forms, buttons, and tables worked without any issues.

To test system strength, multiple tabs and users were used at the same time. The system handled all actions without crashing or slowing down. Basic internet limitations were also tested, and the system remained functional with only slight delays in loading.

In conclusion, the results showed that the EMadrasa System is stable and performs well. All core features worked as expected. More improvements can be made in the future after testing it with real users in an actual madrasa setting.

CHAPTER 8
CONCLUSION & FUTURE WORK

The eMadrasa Management System was made to solve the daily problems of madrasas. It helped build a complete system where teachers can mark attendance, track Quran lessons like sabaq and manzil, and parents can see their child's progress easily. Different login roles were created for admin, teacher, accountant, and parent and each person sees only what they need on a simple dashboard. The system works well on both desktop and mobile, and it was tested with many users at the same time without any problem. This shows the system is useful and ready for small to medium-sized madrasas.

However, there were also many challenges. Since this kind of system did not exist before, everything had to be made from scratch. It was hard to understand madrasa-specific terms like sabaq and atkan and set them up in the database. Giving each user a different view and access also took extra planning.

The eMadrasa Management System is working fine with basic features like attendance, daily sabaq tracking, and rolebased logins. But many new features can still be added in the future to make it more useful and complete. Some ideas for future work are:

A built-in chat system can be added so that teachers, parents, and admins can easily communicate without needing WhatsApp or any other app. An online video class or meeting feature can also be useful for times when students or teachers are unable to attend in person. A Learning Management System (LMS) can be added to share notes, video lectures, quizzes, and to track each student's progress.

A proper system for sending daily or weekly reports to parents through SMS or app notifications can help them stay updated about their child's performance. A fee management module can be added to handle student fees, generate receipts, and track any pending dues. A mobile app for both Android and iOS can also be created so that parents and teachers can easily access the system on their phones.

Since madrasas mostly use Urdu and Arabic, the system can support multiple languages to make it easier for users. An online admission system can also be added to make the process of new student registration faster and more organized. There can be a backup and restore system that automatically saves the database and can restore it in case any data is lost.

To make attendance easier and faster, QR code scanning or face detection features can be introduced to reduce manual errors. A complete exam and test management system can also be created where teachers can upload marks, generate result cards, and analyze student performance. Lastly, a digital library can be included where Islamic books, tajweed guides, and other learning resources are uploaded so that students and teachers can access them anytime.

References

[1] OpenAI, *ChatGPT - AI language model*. [Online]. Available: https://chat.openai.com/
[2] Blackbox AI, *AI code suggestion and documentation tool*. [Online]. Available: https://www.useblackbox.io/
[3] DeepSeek, *AI-powered coding and research assistant*. [Online]. Available: https://deepseek.com/
[4] PlantUML, *Tool to generate UML diagrams from text*. [Online]. Available: https://plantuml.com/
[5] Airbnb, *Book unique homes and experiences around the world*. [Online]. Available: https://www.airbnb.com/
[6] Axios, *Promisebased HTTP client for the browser and Node.js*. [Online]. Available: https://axios-http.com/
[7] Bootstrap, *Front-end open-source toolkit*. [Online]. Available: https://getbootstrap.com/
[8] Cloudinary, *Media management platform for web and mobile developers*. [Online]. Available: https://cloudinary.com/
[9] Draw.io, *Free online diagram editor*. [Online]. Available: https://www.draw.io/
[10] Express.js, *Fast, unopinionated, minimalist web framework for Node.js*. [Online]. Available: https://expressjs.com/
[11] Fedena, *School management software*. [Online]. Available: https://fedena.com/
[12] Formik, *Forms in React, without the tears*. [Online]. Available: https://formik.org/
[13] GitHub, *Code hosting platform for version control and collaboration*. [Online]. Available: https://github.com/
[14] JWT.io, *JSON Web Tokens - Introduction and examples*. [Online]. Available: https://jwt.io/
[15] Lucidchart, *Collaborative diagramming and charting tool*. [Online]. Available: https://www.lucidchart.com/
[16] Material UI, *React UI framework*. [Online]. Available: https://mui.com/
[17] MongoDB, *The developer data platform*. [Online]. Available: https://www.mongodb.com/
[18] Netlify, *Cloud platform for web deployments*. [Online]. Available: https://www.netlify.com/
[19] Node.js, *JavaScript runtime built on Chrome's V8 JavaScript engine*. [Online]. Available: https://nodejs.org/
[20] NPM, *Package manager for JavaScript*. [Online]. Available: https://www.npmjs.com/
[21] React, *A JavaScript library for building user interfaces*. [Online]. Available: https://reactjs.org/
[22] Redux, *A predictable state container for JavaScript apps*. [Online]. Available: https://redux.js.org/
[23] Redux Toolkit, *The official, recommended way to write Redux logic*. [Online]. Available: https://redux-toolkit.js.org/
[24] RTK Query, *Data fetching tool from Redux Toolkit*. [Online]. Available: https://redux-toolkit.js.org/rtk-query/overview
[25] Toastify, *React toast notifications*. [Online]. Available: https://fkhadra.github.io/react-toastify/
[26] University of the Punjab, *FYP Guidelines for Affiliated Colleges – BSCS*, 2023. [Online]. Available: http://pu.edu.pk
[27] Visual Studio Code, *Code editor optimized for development*. [Online]. Available: https://code.visualstudio.com/
[28] Yup, *JavaScript schema builder for value parsing and validation*. [Online]. Available: https://github.com/jquense/yup

Supervisor First Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: December 2, 2024

Meeting Day: Monday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)

Agenda:

1. Project introduction and initial planning
2. Finalizing supervisor and group members
3. Overview of documentation structure

Meeting Summary:
The initial meeting introduced the project concept. Supervisor approval and documentation guidelines were discussed.

_____
Project Developer

_____
Project Supervisor

Supervisor Second Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: December 16, 2024

Meeting Day: Monday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)

Agenda:

1. Software Requirements Discussion
2. Discussion on user roles and functionality
3. Assigning tasks for Designing

Meeting Summary:
Basic requirements were finalized. User roles (Admin, Teacher, Student) discussed. Assigned UI sketch tasks.

_____
Project Developer

_____
Project Supervisor

Supervisor Third Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: January 1, 2025

Meeting Day: Wednesday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)

Agenda:

1. Review of SRD
2. Initial database design discussion
3. Confirmation of tech stack

Meeting Summary:
SRS was reviewed and finalized. Decided to use MERN stack. Began rough database schema planning.

_____
Project Developer

_____
Project Supervisor

Supervisor Fourth Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: January 15, 2025

Meeting Day: Wednesday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)

Agenda:

1. Finalizing database schema
2. Start frontend development
3. Set milestone for login system

Meeting Summary:
Database schema was locked in. Work began on login/signup screens. Milestone set for authentication.

_____
Project Developer

_____
Project Supervisor

Supervisor Fifth Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: February 3, 2025

Meeting Day: Monday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)

Agenda:

1. Progress check on authentication
2. Start admin panel UI
3. Backend routes planning

Meeting Summary:
Frontend login system completed. Started admin dashboard layout. Outlined Express backend APIs.

_____
Project Developer

_____
Project Supervisor

Supervisor Sixth Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: February 17, 2025

Meeting Day: Monday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)

Agenda:

1. Backend API development review
2. Connect frontend to backend
3. Test user registration

Meeting Summary:
Completed user APIs. Connected frontend login/register. Started testing user flows.

_____
Project Developer

_____
Project Supervisor

Supervisor Seventh Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: March 3, 2025

Meeting Day: Monday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)

Agenda:

1. CRUD for student attendance
2. Discuss Diary module
3. Plan rolebased access control

Meeting Summary:
Attendance module completed with basic CRUD. Planned Leave module. Assigned RBAC implementation task.

_____
Project Developer

_____
Project Supervisor

Supervisor Eighth Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: March 17, 2025

Meeting Day: Monday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)

Agenda:

1. Diary Module UI integration
2. Middleware for RBAC
3. Start work on Progress Module

Meeting Summary:
RBAC added to routes. Leave request page completed. Admin controls added to attendance.

_____
Project Developer

_____
Project Supervisor

Supervisor Nineth Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: April 1, 2025

Meeting Day: Tuesday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)

Agenda:

1. Show Development work
2. Show Documentation
3. UI enhancements discussion

Meeting Summary:
Users controls almost complete. Suggested UI improvements.

_____
Project Developer

_____
Project Supervisor

Supervisor Tenth Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: April 15, 2025

Meeting Day: Tuesday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)

Agenda:

1. Bug fixing and optimization
2. Further Documentation Enhancement

Meeting Summary:
Bugs in components fixed.

_____
Project Developer

_____
Project Supervisor

Supervisor Eleventh Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: May 1, 2025

Meeting Day: Thursday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)

Agenda:

1. Final system testing
2. Discuss deployment options
3. Plan for viva and final demo

Meeting Summary:
System testing in progress. Discussed deploying on Render. Prepared checklist for final demo.

_____
Project Developer

_____
Project Supervisor

Supervisor Twelfth Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: June 2, 2025

Meeting Day: Monday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)

Agenda:

1. Collect feedback from supervisor
2. Final documentation review
3. Start PPT for viva

Meeting Summary:
Supervisor shared feedback. Minor fixes suggested. Documentation proofreading started. Began creating final presentation.

_____
Project Developer

_____
Project Supervisor

Board First Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: November 5, 2024

Meeting Day: Tuesday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)
- Mr Atif Javed (Board Member)
- Mr Arshad Nadeem (Board Member)
- Miss Kainat Mehak (Board Member)

Agenda:

1. Project Overview Presentation

Meeting Summary:
Project concept appreciated. Board suggested including teacher panel separately and improving login flow structure.

_____
Project Developer

_____
Project Supervisor

Board Second Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: December 5, 2024

Meeting Day: Thursday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)
- Mr Atif Javed (Board Member)
- Mr Arshad Nadeem (Board Member)
- Miss Kainat Mehak (Board Member)

Agenda:

1. Database Schema Review
2. UI Layout Demo

Meeting Summary:
UI design feedback was noted for dashboard improvements.

_____
Project Developer

_____
Project Supervisor

Board Third Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: January 6, 2024

Meeting Day: Monday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)
- Mr Arshad Nadeem (Board Member)
- Miss Kainat Mehak (Board Member)

Agenda:

1. Frontend Layout Designs
2. Role Management
3. API Integration Review

Meeting Summary:
Board suggested creating separate middleware for rolebased access and refining API response formats.

_____
Project Developer

_____
Project Supervisor

Board Fourth Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: February 5, 2024

Meeting Day: Wednesday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)
- Mr Arshad Nadeem (Board Member)
- Miss Kainat Mehak (Board Member)

Agenda:

1. Accountant Module Implementation
2. Documentation Review

Meeting Summary:
Showing accountant dashboard and their respective tasks.

_____
Project Developer

_____
Project Supervisor

Board Fifth Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: March 5, 2024

Meeting Day: Wednesday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)
- Mr Arshad Nadeem (Board Member)
- Miss Kainat Mehak (Board Member)

Agenda:

1. Teacher Module Implementation
2. Documentation Review

Meeting Summary:
Showing teacher dashboard and their respective tasks.

_____
Project Developer

_____
Project Supervisor

Board Sixth Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: April 4, 2024

Meeting Day: Friday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)
- Mr Arshad Nadeem (Board Member)
- Miss Kainat Mehak (Board Member)

Agenda:

1. Student Module Implementation
2. Documentation Review

Meeting Summary:
Showing student dashboard and Further Discussion on Parent module.

_____
Project Developer

_____
Project Supervisor

Board Seventh Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: April 4, 2024

Meeting Day: Friday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)
- Mr Arshad Nadeem (Board Member)
- Miss Kainat Mehak (Board Member)

Agenda:

1. Student Module Implementation
2. Documentation Review

Meeting Summary:
Showing student dashboard and Further Discussion on Parent module.

_____
Project Developer

_____
Project Supervisor

Board Eighth Meeting Minutes

Project Title: E Madrasa Management System

Project Supervisor: Mr. Faraz Siddiqi

Meeting Date: May 30, 2024

Meeting Day: Friday

Meeting Start Time: 10:00 AM

Meeting End Time: 11:00 AM

Venue: BSCS Lab

Attendees:

- Huzaifa Zafar (Development + Documentation)
- Mr. Faraz Siddiqi (Supervisor)
- Miss Rafia Khalil (Co-Supervisor)
- Mr Arshad Nadeem (Board Member)
- Miss Kainat Mehak (Board Member)

Agenda:

1. Quran Module Implementation
2. Final Documentation Review

Meeting Summary:
Showing Quran module and complete documentation review.

_____
Project Developer

_____
Project Supervisor